

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

УДК 004.023

ОТЧЕТ

по исследовательскому курсовому проекту

на тему

**«Оптимизация алгоритмов маршрутизации на примере задачи
нескольких коммивояжеров»**

по направлению подготовки бакалавров
09.03.04 «Программная инженерия»

Выполнил
студент группы БПИ246
образовательной программы
09.03.04 «Программная инженерия»

Купцов Денис Дмитриевич

Научный руководитель
Ермаков Андрей Максимович

Москва 2026

РЕФЕРАТ

Отчет 104 с., 17 рис., 24 табл., 53 источн., 8 прил.

Объект исследования — эвристические алгоритмы решения задачи о нескольких коммивояжерах (multiple Traveling Salesman Problem, mTSP) на инстансах до 100 000 узлов в условиях ограниченного лимита времени.

Цель работы — спроектировать, реализовать и экспериментально оценить собственный решатель на базе адаптивного поиска в больших окрестностях, сравнить его с открытыми эталонами (LKH-3, OR-Tools, FILO2) на стратифицированной сетке инстансов и подтвердить выводы статистически.

Ключевые результаты.

- **Большие инстансы** ($N = 25\text{--}100$ тыс.). ALNS-mTSP строит маршруты, равномерно распределённые между коммивояжерами; LKH-3 default отдаёт почти всю работу одному коммивояжеру — формально допустимое, но непригодное для прикладной маршрутизации решение.
- **Малые инстансы (fair-baseline)**. При балансирующей настройке LKH-3 ALNS-mTSP лучше по MINSUM на 13–18 %; этот разрыв служит контролем против артефакта default-LKH-3, а не основным доказательством.
- **Лимит времени**. ALNS-mTSP укладывается в $T \leq 350$ с на всех проверенных инстансах; LKH-3 в балансирующих режимах на $N = 25K$ — 0/12 валидных за 30–60 мин; FILO2 в CVRP-адаптации — 2/18; OR-Tools в стандартной конфигурации не уложился в бюджет с практически приемлемым решением на $N \geq 5K$.
- **Прямое сравнение на $N = 100K$ — против FILO2**. На 12 ячейках $N \in \{50K, 100K\}$ ALNS-mTSP выигрывает 9/12, в т. ч. **6/6 на $N = 100K$** с разрывом 5–30 % по MINSUM.
- **Сравнение со специализированными SOTA для MINMAX**. На mTSPLib ($n \leq 100$) HGA [35] и He-Hao MA [29] доминируют по makespan; на $N \geq 25K$ их эталонные реализации не публикуют результатов и в работе не запускались.

Основной результат. В заявленном сценарии (CPU без GPU, $T \leq 350$ с, N до 100 000, $n_r = m$ нетривиальных маршрутов, $\text{Gini} \leq 0.05$) ALNS-mTSP проходит полный практический фильтр. Воспроизведённые открытые baseline нарушают хотя бы одно из условий: LKH-3 default — (i), (iii); LKH-3 balanced на $N = 25K$ — (i); OR-Tools — бюджет на $N \geq 5K$; FILO2 — (i), (ii). Внутри допустимой области ALNS-mTSP лучше *2opt+greed* (единственного открытого baseline, проходящего фильтр) на 1.4–4.8 % по MINSUM (медиана ≈ 3.6 %, табл. 5а).

Экспериментальная инфраструктура. Собрана воспроизводимая система: генератор синтетических инстансов (6 семейств), единая C++-обёртка решателей, адаптеры WSL2 для LKH-3 и FILO2, Python-статистика. Метрики качества и баланса — по обзору [28]; парные сравнения — непараметрическими методами с поправкой на множественные сравнения [39] и bootstrap-CI [38]; величина эффекта — по [44]. Всего — 15 экспериментов, ≥ 2157 валидных запусков на 13 конфигурациях решателей.

Ограничения. Часть SOTA-методов (ITSHA, MILS, нейросетевые Equity-Transformer, DPN, GLOP, DualOpt, UDC, iMTSP) не запускалась — причины обсуждаются в главе 5; направления дальнейшей работы — в главе 6.

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	6
ВВЕДЕНИЕ	9
1 Обзор алгоритмов решения mTSP и TSP	15
1.1 Постановка задачи mTSP	15
1.2 Точные методы и Concorde	16
1.3 LKH-3 (Lin-Kernighan-Helsgaun)	17
1.4 Прочие подходы и внешние ориентиры	19
1.5 Собственный алгоритм ALNS-mTSP	20
1.6 Теоретическая оценка сложности	24
2 Сравнение архитектур исследуемых решателей	25
2.1 LKH-3 в режиме MINSUM	25
2.2 ALNS-mTSP	25
2.3 OR-Tools (Routing + GLS)	26
2.4 Классические эвристики	26
3 Описание экспериментальной программы	27
3.1 Подготовка набора инстанций	27
3.2 Главная программа сравнения	27
3.3 Метрики качества и времени	28
3.4 Особенности работы исследуемых решателей	29
3.5 Стратегия стратифицированного бенчмарка	30
3.6 Сводка экспериментальной программы	31
4 Результаты и проверка гипотез	33
4.1 Малые инстансы ($N = 100 \dots 1000$)	33
4.2 Слой верификации ($N = 25\,000$)	35
4.3 Большие данные ($N = 50\,000$ и $N = 100\,000$)	35
4.4 Эмпирическая проверка гипотез H1–H6	38
4.5 Ablation: вклад компонентов ALNS-mTSP	40
4.6 Fair-baseline сравнение с LKH-3 в балансирующем режиме . .	41

4.7	Сравнение с FILO2 на больших инстансах	43
4.8	Сравнение со специализированными SOTA для MINMAX-mTSP (HGA, He-Nao MA)	49
4.9	Статус гипотез работы	50
5	Достоверность, ограничения и риски	53
6	Заключение	56
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	60
А	Детали архитектуры ALNS-mTSP	66
A.1.	Структура общего ядра	66
A.2.	Описание четырех solver-оберток	67
A.3.	ALNS-операторы и acceptance	69
Б	Вырожденные mTSP-решения в чистом MINSUM	70
В	Воспроизводимость статистических расчётов	72
B.1.	Артефакты и команды воспроизведения	73
B.2.	Среда и параметры воспроизводимости	74
Г	Постпроцессинговый анализ	75
G.1	Pareto-анализ cost vs balance	75
G.2	Anytime-кривые сходимости	76
G.3	Sensitivity-анализ бюджета времени	77
G.4	Профилирование по фазам <i>alns_minsum</i>	80
Д	Эволюция <i>1kh-wrapper</i>-линии (v1–v21)	83
Е	Матрица «решатель × масштаб»	95
E.1.	Покрытие	95
E.2.	Качество	96
E.3.	Заметки по интерпретации	98
Ж	Полная теоретическая сложность	99
J.1.	Точные методы	99

Ж.2. LKH-3	99
Ж.3. OR-Tools GLS	99
Ж.4. FILO2	99
Ж.5. ALNS-mTSP	99
Ж.6. Сводная таблица	100
3 Обзор внешних SOTA-решателей	101
3.1. Классические min-max mTSP	101
3.2. Нейросетевые SOTA	101

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

Используемые термины и обозначения:

Термин	Определение
TSP (Traveling Salesman Problem)	Классическая задача о коммивояжере: найти кратчайший гамильтонов цикл, проходящий через каждую точку ровно один раз. NP-трудная задача.
mTSP (multiple TSP)	Обобщение TSP: имеется m коммивояжеров, выходящих и возвращающихся в общий депот; каждый клиент посещается ровно одним коммивояжером.
MINSUM	Целевая функция mTSP — сумма длин всех маршрутов.
MINMAX	Альтернативная целевая функция — длина самого длинного маршрута; используется для балансировки.
Депот (depot)	Стартовая и конечная вершина каждого маршрута.
Маршрут (route)	Упорядоченная последовательность вершин одного коммивояжера, начинающаяся и заканчивающаяся в депоте.
Тривиальный маршрут	Маршрут вида [depot, depot] (пустой) или [depot, c , depot] (один клиент); такие маршруты практически бесполезны для маршрутизации флота.
Вырожденное решение	Решение mTSP, в котором один коммивояжер посещает почти все клиенты, а остальные имеют тривиальные маршруты.

balance_max_avg	Диагностическая метрика равномерности маршрутов: $\text{balance} = \max_s \text{len}(R_s) / \overline{\text{len}(R_s)} = m \cdot F_{\max} / F_{\text{sum}}$. Идеал — 1; при одном коммивояжере, несущем почти всю нагрузку, стремится к m.
makespan	$F_{\max}(R) = \max_s \text{len}(R_s)$ — длина самого длинного маршрута; критерий минимизации максимума.
range	$\max_s \text{len}(R_s) - \min_s \text{len}(R_s)$ — размах распределения длин.
std (длин маршрутов)	$\sqrt{\frac{1}{m} \sum_s (\text{len}(R_s) - \overline{\text{len}})^2}$ — стандартное отклонение длин.
Gini coefficient	$G = \frac{1}{2m^2\bar{x}} \sum_i \sum_j x_i - x_j$, где $x_i = \text{len}(R_i)$. Принимает значения $[0, 1 - 1/m]$. Стандартная метрика неравномерности [28]. В отличие от balance_max_avg, учитывает весь профиль распределения, не только экстремум.
n_empty	Количество маршрутов, не содержащих ни одного клиента.
LKH (Lin-Kernighan-Helsgaun)	Метаэвристический решатель TSP/mTSP/CVRP/CVRPTW авторства Keld Helsgaun; современная версия LKH-3.
TIME_LIMIT	Параметр LKH-3, задающий предельное время работы; на больших инстанциях выполняется приближенно.
ALNS	Adaptive Large-Neighborhood Search — метаэвристика «разрушить-восстановить» с адаптивным выбором операторов [23,24].

GLS	Guided Local Search — локальный поиск со штрафами на ребра с большой стоимостью для побега из локальных оптимумов.
POPMUSIC	Метод построения candidate-сетов через маленькие подпроблемы.
OR-Tools	Open-source решатель Google для задач маршрутизации (CVRP, TSP, mTSP).
Budget (бюджет времени)	Лимит времени, выделенный решателю; единица сравнения по производительности.
n, m	Число клиентов (включая депо в n) и число коммивояжеров.
OMP	OpenMP — система параллелизации для C/C++.
WSL	Windows Subsystem for Linux; для запуска LKH-3 и FILO2.
Wilcoxon signed-rank	Непараметрический парный критерий значимости (<code>scipy.stats.wilcoxon</code>); рекомендован [34] для сравнения routing-методов.
Bootstrap CI	Percentile-bootstrap CI, 10 000 ресэмплов, $\alpha = 0.05$ (<code>scipy.stats.bootstrap</code>).

ВВЕДЕНИЕ

Прикладной контекст. Современная логистика и планирование маршрутов курьерских и сервисных бригад требуют эффективных инструментов работы с большими массивами точек обслуживания. Задача о нескольких коммивояжерах (multiple Traveling Salesman Problem, mTSP) — обобщение классической задачи коммивояжера на случай m агентов, выходящих из общего депо, посещающих непересекающиеся подмножества клиентов и возвращающихся обратно. Она встречается в курьерской доставке, в логистике последней мили, в планировании работы мобильных бригад, в маршрутизации дронов и беспилотных транспортных средств. Особенно остро задача стоит при росте числа точек обслуживания до десятков и сотен тысяч — такие масштабы характерны для крупных городских и региональных операторов доставки.

Существующие решатели и их ограничения. Задача о коммивояжере относится к классу NP-трудных [49], и при переходе к нескольким исполнителям сложность только возрастает. На инстансах с десятками и сотнями тысяч узлов точные методы (Concorde, Held–Karp DP [30]) становятся неприменимыми, и единственным практичным подходом остаются эвристики и метаэвристики. Среди открытых решателей выделяются три эталонных:

- **LKH-3** (Lin–Kernighan–Helsgaun) — k -opt-метаэвристика с расширением на mTSP через штрафные функции;
- **OR-Tools** от Google — индустриальный решатель на основе Guided Local Search;
- **FILO2** (Accorsi, Vigo, 2024) — сильный CVRP-решатель, адаптируемый на mTSP через ограничение вместимости маршрута.

Большинство публикаций сравнивают эти решатели на $N \leq 10\,000$; на TSP-100K даже LKH-3 требует около 8 часов на один прогон, что нереально для прикладных условий.

Перспективное направление: ALNS. Адаптивный поиск в больших окрестностях (Adaptive Large-Neighborhood Search, ALNS) [23,24] в сочетании с алгоритмом имитации отжига и candidate-set-фильтрацией

хорошо показывает себя на средних масштабах; обзоры структуры операторов и стратегий их выбора — [36,42]. Однако значительная часть существующих ALNS-реализаций ориентирована на $N \sim 1\,000$ и не масштабируется до $N = 100\,000$ без специальных инженерных приёмов: KD-tree-поиск, candidate-set, region-granular-операторы, параллельный multi-start. Таким образом, разработка собственного ALNS-решателя для крупных mTSP-инстансов и его сравнительный анализ с открытыми эталонами — актуальная задача, имеющая практическое и научное значение.

Объект исследования — эвристические алгоритмы решения задачи о нескольких коммивояжерах (mTSP) на инстанциях с N до 100 000 узлов в условиях ограниченного бюджета времени.

Предмет исследования — собственный решатель ALNS-mTSP (модульная архитектура src/v21/) и его относительная эффективность по сравнению с открытыми эталонными решателями LKH-3, OR-Tools и FILO2 на стратифицированной сетке инстансов.

Цель работы — спроектировать, реализовать и экспериментально оценить собственный ALNS-решатель крупных mTSP-инстансов; провести систематическое сравнение с тройкой эталонных открытых решателей и определить диапазоны параметров (N, m , геометрия), на которых ALNS-mTSP дает наилучший компромисс качество/время.

Задачи исследования:

1. Реализовать модульный ALNS-решатель с четырьмя специализированными вариантами (alns_minsum, alns_cap, alns_depot2m, alns_minmax), поддерживающий N до 100 000 и m до 100.
2. Собрать воспроизводимую экспериментальную инфраструктуру: генератор синтетических инстансов из 6 геометрических семейств, единая C++-обертка для всех решателей, адаптеры WSL2 для LKH-3 и FILO2, Python-скрипты статистики.
3. Провести стратифицированный бенчмарк трех масштабов ($N \leq 1\,000$; $N = 25\,000$; $N \in \{50\,000, 100\,000\}$) с корректной парной статистикой: Wilcoxon signed-rank, BCa-bootstrap CI, Holm-коррекция, Cliff's δ .
4. Сравнить ALNS-mTSP с LKH-3 в трех режимах (default-MINSUM, balanced-MINSUM MIN_SIZE=-1, balanced-MINMAX), с OR-Tools

(PATH_CHEAPEST_ARC + GUIDED_LOCAL_SEARCH) и с FILO2 в CVRP-адаптации.

5. Кросс-валидировать результаты на стандартном наборе mTSPLib (Necula et al.) и со специализированными SOTA для MINMAX-mTSP (HGA Mahmoudinazlou–Kwon, He–Hao MA).
6. Сформулировать рекомендации по применимости 13 исследованных конфигураций решателей и зафиксировать ограничения работы.

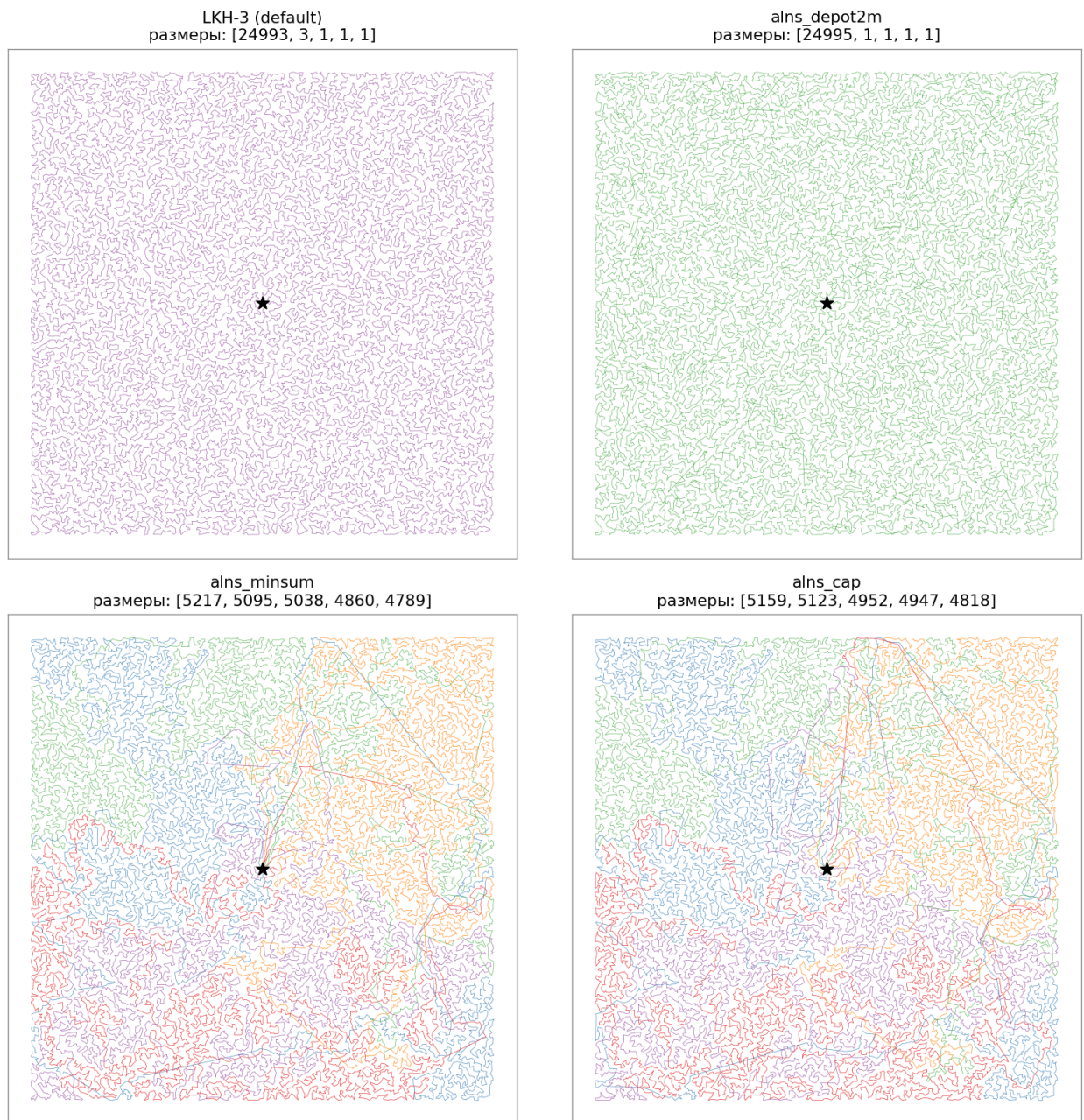


Рисунок 1 — Структура маршрутов на $N = 25\,000$, $m = 5$ (uniform): *слева* — вырожденные решения (LKH-3 default и alns_depot2m), *справа* — сбалансированные (alns_minsum и alns_minsum_cap).

Чтение рисунка 1. В вырожденных решениях один коммивояжёр посещает почти все 25 000 точек, остальные четыре маршрута тривиальны. В сбалансированных решениях пять непересекающихся зон сравнимого размера. Это — центральный визуальный аргумент работы: формально-минимальная по MINSUM конфигурация (слева) непригодна для практической маршрутизации флота, тогда как сбалансированная (справа) удовлетворяет требованию (iii) практического фильтра.

Гипотезы работы. При ограниченном бюджете времени до 350 секунд на CPU собственный решатель ALNS-mTSP даёт практически применимое решение задачи о нескольких коммивояжерах на инстансах с N до 100 000 узлов — то есть решение, удовлетворяющее одновременно трём практическим требованиям: (i) укладывание в заявленный лимит времени; (ii) ровно m нетривиальных маршрутов; (iii) приемлемая балансировка длин маршрутов между коммивояжерами. Утверждение формулируется на двух уровнях.

- **Главная гипотеза.** В рамках бюджета времени до 350 секунд ALNS-mTSP возвращает решения, удовлетворяющие (i)–(iii), на инстансах с N до 100 000 узлов. Открытые эталонные решатели (LKH-3, OR-Tools, FILO2) в тех же условиях либо нарушают (i) — не укладываются в бюджет, либо нарушают (ii)–(iii) — возвращают вырожденные решения или систематически отклоняются от заданного m .
- **Итоговая гипотеза.** В заявленном сценарии (mTSP на CPU без GPU, лимит времени до 350 секунд, инстансы до 100 000 узлов) ALNS-mTSP — конкурентоспособная альтернатива воспроизведённым в работе открытым baseline. Сравнение с нейросетевыми и частью специализированных SOTA-методов ограничено анализом применимости по опубликованным масштабам, требованиям к GPU и доступности исходного кода — прямое сравнение в работе не выполнено.

Операциональный критерий приемлемой балансировки. Под «приемлемой балансировкой» в условии (iii) понимается рабочий порог $Gini \leq 0.05$ или $balance_max_avg \leq 1.10$ на основных больших инстансах

($N \in \{25K, 50K, 100K\}$). Пороги используются как диагностическое правило для отделения вырожденных решений (один маршрут несёт почти всех клиентов) от равномерных. На малых инстансах ($N \leq 1\,000$) критерий смягчён до $\text{balance} \leq 1.5$ — MINSUM-оптимум на этом масштабе сам допускает тривиальные маршруты.

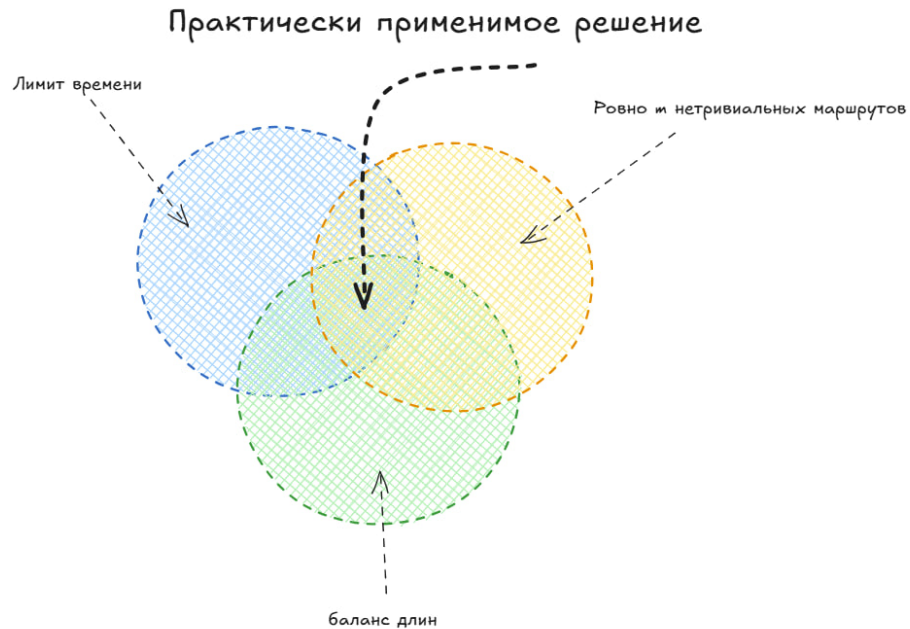


Рисунок 2 — «Практически применимое решение» как пересечение трёх требований практического фильтра: (i) $t \leq T_{\text{budget}}$, (ii) $n_r = m$ нетривиальных маршрутов, (iii) приемлемая балансировка длин.

Главная и итоговая гипотезы формулируются как утверждения о попадании ALNS-mTSP в центральную область пересечения — недостижимую для рассмотренных открытых эталонов в целевом сценарии.

Сопутствующие гипотезы H1–H6 (вырождение чистого MINSUM, поведение OR-Tools на $N \geq 5K$, адаптивность весов ALNS, рост преимущества с m , квази-линейное масштабирование по времени, симметрия вырождения) проверяются отдельно в разделе 4.4.

Методологический дисклеймер о LKH-3. Часть таблиц использует LKH-3 в default-конфигурации ($\text{MTSP_MIN_SIZE} = 1$, $\text{MTSP_OBJECTIVE} = \text{MINSUM}$), при которой допустимы вырожденные решения с маршрутами длиной 0 или 1 клиент.

Раздел 4.6 содержит отдельный fair-baseline-прогон LKH-3 с балансирующими параметрами ($\text{MIN_SIZE} = -1$ и MINMAX). Этот fair-baseline трактуется не как окончательное доказательство

превосходства ALNS, а как (а) демонстрация цены балансировки для LKH-3 и (б) контроль того, что преимущество ALNS на малых инстансах не сводится исключительно к артефакту default-конфигурации; default-режим LKH-3 — как иллюстрация эффекта вырождения чистого MINSUM.

Структура работы.

- *Глава 1* — обзор алгоритмов решения mTSP и TSP, постановка задачи и теоретические оценки сложности.
- *Глава 2* — сравнение архитектур исследуемых решателей.
- *Глава 3* — экспериментальная программа: подготовка инстансов, метрики, статистические инструменты.
- *Глава 4* — результаты стратифицированного бенчмарка, проверка гипотез H1–H6, ablation, fair-baseline-сравнение, mTSPLib, FILO2 и SOTA.
- *Глава 5* — достоверность, ограничения и риски.
- *Глава 6* — заключение и направления дальнейшей работы.
- *Приложения A–З* — детали архитектуры ALNS-mTSP, обоснование вырождения чистого MINSUM, постпроцессинговые артефакты, эволюция исторической lkh-wrapper-линии (v1...v21), полная теоретическая сложность и расширенный обзор внешних SOTA-решателей.

1. ОБЗОР АЛГОРИТМОВ РЕШЕНИЯ mTSP И TSP

В главе рассмотрены ключевые подходы к решению mTSP: точные методы, классические эвристики, метаэвристики и нейросетевые подходы. Отмечены особенности каждого класса на больших инстансах ($N \geq 25\,000$), существенные для экспериментального сравнения. Глава завершается оценкой временной сложности.

1.1. Постановка задачи mTSP

Обозначим через 0 депот, через $C = \{1, \dots, n - 1\}$ — множество клиентов, и положим $V = \{0\} \cup C$, $|V| = n$. На полном множестве пар $V \times V$ задана весовая функция $w : V \times V \rightarrow \mathbb{R}_{\geq 0}$; в настоящей работе $w(u, v) = \|p_u - p_v\|_2$, где $p_u \in \mathbb{R}^2$ — координата вершины u на плоскости $[0, 100]^2$. Каждой инстанции соответствует полный взвешенный граф

$$G = (V, E, w), \quad E = \{(u, v) : u, v \in V, u \neq v\}. \quad (1.1)$$

Дополнительно задано число агентов $m \in \mathbb{N}$, $m \leq n - 1$.

Под *маршрутом* агента $s \in \{1, \dots, m\}$ понимается упорядоченная последовательность

$$R_s = (v_0^{(s)}, v_1^{(s)}, \dots, v_{\ell_s}^{(s)}, v_{\ell_s+1}^{(s)}), \quad v_0^{(s)} = v_{\ell_s+1}^{(s)} = 0,$$

длина которой определяется как

$$\text{len}(R_s) = \sum_{i=0}^{\ell_s} w(v_i^{(s)}, v_{i+1}^{(s)}). \quad (1.2)$$

Определение 1 (допустимое решение mTSP). *Набор $R = \{R_1, \dots, R_m\}$ называется допустимым решением задачи mTSP, если: (а) каждый маршрут начинается и заканчивается в депоте; (б) каждый клиент $c \in C$ принадлежит ровно одному маршруту; (в) внутри каждого маршрута клиенты не повторяются.*

В работе рассматриваются две основные целевые функции:

$$F_{\text{sum}}(R) = \sum_{s=1}^m \text{len}(R_s) \longrightarrow \min \quad (\text{MINSUM}), \quad (1.3)$$

$$F_{\text{max}}(R) = \max_{s=1, \dots, m} \text{len}(R_s) \longrightarrow \min \quad (\text{MINMAX}). \quad (1.4)$$

Если все длины $\text{len}(R_s)$ равны, обе целевые функции совпадают с точностью до множителя $1/m$; в общем случае оптимумы (1.3) и (1.4) разнесены, причём степень их рассогласования зависит от геометрии инстанса и числа агентов — количественный worst-case-анализ соответствия min-sum и min-max в маршрутизации приведён в [27].

О вырождении в чистом MINSUM. Определение 1 не требует $\text{len}(R_s) > 0$, т.е. допускает маршруты вида $R_s = (0, 0)$ (пустой) или $R_s = (0, c, 0)$ (синглтон). Без дополнительных ограничений (MIN_SIZE, MAX_SIZE, capacity) минимум (1.3) часто достигается на конфигурации, в которой один маршрут проходит по всем клиентам, а остальные $m - 1$ маршрутов тривиальны — такое решение формально допустимо, но непригодно для маршрутизации флота. Эмпирическая проверка этого свойства приведена в разделе 4.4 (H1) и обоснование — в Прил. Б.

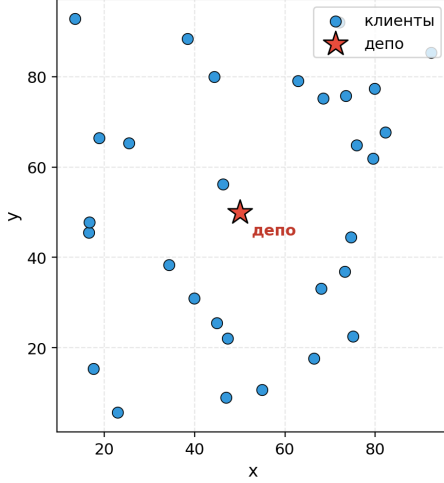
1.2. Точные методы и Concorde

Программа Concorde [3,4] (Applegate, Vixby, Chvátal, Cook) — золотой стандарт точных решателей TSP, работающий на стандартных инстанциях TSPLIB [26]. Concorde находит доказуемо оптимальные решения. Однако его применимость ограничена малыми и средними N : по публичным данным, на инстанции с $n = 85\,900$ узлов Concorde тратит порядка 136 CPU-лет; на случайных евклидовых инстанциях с $n > 10\,000$ он, как правило, не завершается в разумное время. Аналогично динамическое программирование Held–Karp [30] требует $\Theta(n^2 \cdot 2^n)$ времени и $\Theta(n \cdot 2^n)$ памяти, что делает его применимым только до $n \approx 25$.

Применимость в работе. Точные методы непригодны для рассматриваемой задачи (mTSP с N до 100 000 при лимите 350 секунд CPU), и в основной сетке экспериментов не запускались. Они используются в работе только как теоретический ориентир: невозможность точного

Задача mTSP: вход и допустимое решение ($n = 30$ клиентов, $m = 4$, $F_{\text{sum}} = \sum_k \text{len}(R_k)$, $F_{\text{max}} = \max_k \text{len}(R_k)$)

(а) Вход: $n = 30$ клиентов + 1 депо, $m = 4$ коммивояжёра



(б) Допустимое решение: 4 маршрута, каждый стартует и финиширует в депо

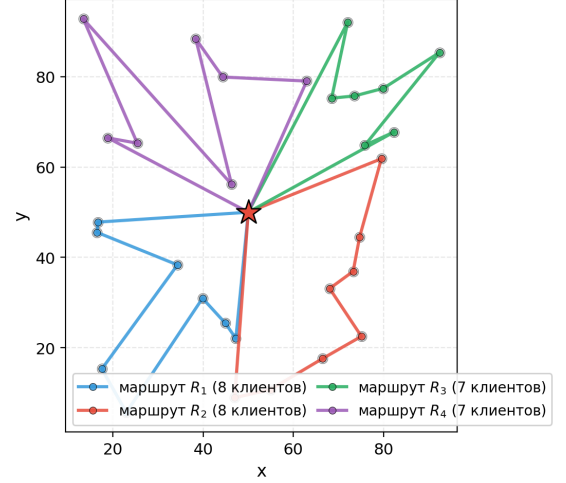


Рисунок 3 — Иллюстрация постановки mTSP на маленькой инстанции ($n = 30$ клиентов, $m = 4$ коммивояжера, депо в центре поля 100×100). Слева (а): вход — множество $V = \{0, 1, \dots, 30\}$, где 0 — депо (красная звезда), $1 \dots 30$ — клиенты (синие точки). Справа (б): одно из допустимых решений — 4 маршрута R_1, R_2, R_3, R_4 , каждый начинается и заканчивается в депо, посещает 7...8 клиентов; цвет маршрута соответствует своему коммивояжеру. Целевые функции: $F_{\text{sum}}(R) = \sum_{k=1}^m \text{len}(R_k)$ (MINSUM) и $F_{\text{max}}(R) = \max_k \text{len}(R_k)$ (MINMAX).

решения за разумное время и определяет необходимость метаэвристических подходов на крупных инстансах.

1.3. LKH-3 (Lin-Kernighan-Helsgaun)

Программа LKH-3 [1,2,32], разрабатываемая Keld Helsgaun-ом с 1990-х годов, реализует k -opt-метаэвристику на основе классического алгоритма Lin-Kernighan [18] с многочисленными расширениями: candidate-сетью на основе α -меры или POPMUSIC [25], штрафные функции для расширения на CVRP/PDP/mTSP/CVRPTW, сегментные деревья для быстрых перестановок. LKH-3 поддерживает mTSP штатно через ключевые слова SALESMEN, MTSP_OBJECTIVE = MINSUM | MINMAX, MTSP_MIN_SIZE, MTSP_MAX_SIZE, INITIAL_TOUR_ALGORITHM = MTSP. По данным авторов GLOP [7] (AAAI 2024), на TSP100K в режиме одного trial LKH-3 тратит около 8.1 часов на одно решение.

В работе LKH-3 запускался в *default-MINSUM* и в двух балансирующих режимах — *balanced-MINSUM* (MTSP_OBJECTIVE = MINSUM, MTSP_MIN_SIZE = -1) и *balanced-MINMAX* (MTSP_OBJECTIVE = MINMAX с явными MIN_SIZE/MAX_SIZE-ограничениями). На малых инстансах

LKH-3 соблюдает заданный лимит времени; на инстансах с $N \geq 25\,000$ может превышать его в $10\times$ и более — это специально измерено в работе. Параметры запуска и пути к собранному binary вынесены в Прил. Г.

Применимость в работе. LKH-3 используется как основной эталонный решатель на малых инстансах ($N \leq 1000$) и на слое верификации ($N = 25\,000$). В условиях, заявленных в работе (фиксированный лимит 350 секунд, N до 100 000, требование ровно m нетривиальных маршрутов и приемлемой балансировки), LKH-3 в использованных конфигурациях оказался непригоден по двум причинам: (1) в default-MINSUM-режиме на $N \geq 25\,000$ превышает заданный лимит в 9–12 раз и возвращает вырожденные решения ($Gini \rightarrow 1 - 1/m$), не удовлетворяющие требованию балансировки; (2) в балансирующих режимах (MTSP_OBJECTIVE = MINMAX или MTSP_MIN_SIZE = -1) LKH-3 на $N = 25\,000$ не сходится за 30–60 минут (0 валидных запусков из 12). Подробнее — разделы 4.6 и 4.2.

В основе LKH-3 и большинства метаэвристик маршрутизации лежит *2-opt-перестановка* [21]: реверс сегмента маршрута между двумя ребрами, принимаемый при уменьшении суммарной длины (рис. 4). Родственная операция *Or-opt* [47] переносит сегмент длины $k \in \{1, 2, 3\}$ в новую позицию без реверса. Прямолинейный 2-opt требует $\Theta(n^2)$ на проход; стандартное ускорение — *candidate-set* (k -NN-окрестность), сокращающий перебор до $\Theta(n \cdot k)$. LKH-3 использует POPMUSIC/ α -меру с $k = 5$ –10; собственный ALNS-mTSP — k -NN из KD-tree с $k = 10$. Эмпирический эффект — ускорение в 10 – $50\times$ при практически той же зоне сходимости.

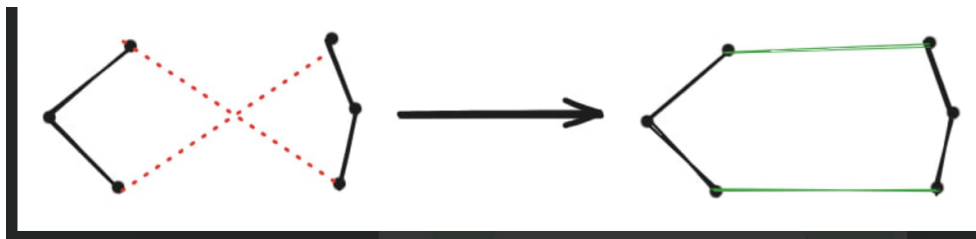


Рисунок 4 — Принцип 2-opt-перестановки. Слева — два маршрута с пересекающимися рёбрами (красный пунктир). Справа — после реверса сегмента между точками обмена новые рёбра (зелёные) короче, пересечение устранено, суммарная длина уменьшается.

Сам *candidate-set* строится поверх k -d tree [48] — структуры рекурсивного разбиения плоскости чередующимися осевыми гиперплоскостями: на каждом уровне точки делятся пополам по медиане

одной координаты, на следующем — по другой. В получившемся дереве (рис. 5) запрос «найти k ближайших соседей точки» обрабатывается за $\Theta(\log n + k)$ амортизированно вместо $\Theta(n)$ при наивном переборе. Построение дерева — $\Theta(n \log n)$, выполняется один раз в начале работы решателя; все последующие 2-opt, repair и destroy-операторы пользуются предвычисленным candidate-set, что и даёт основное ускорение метаэвристики на крупных инстансах.

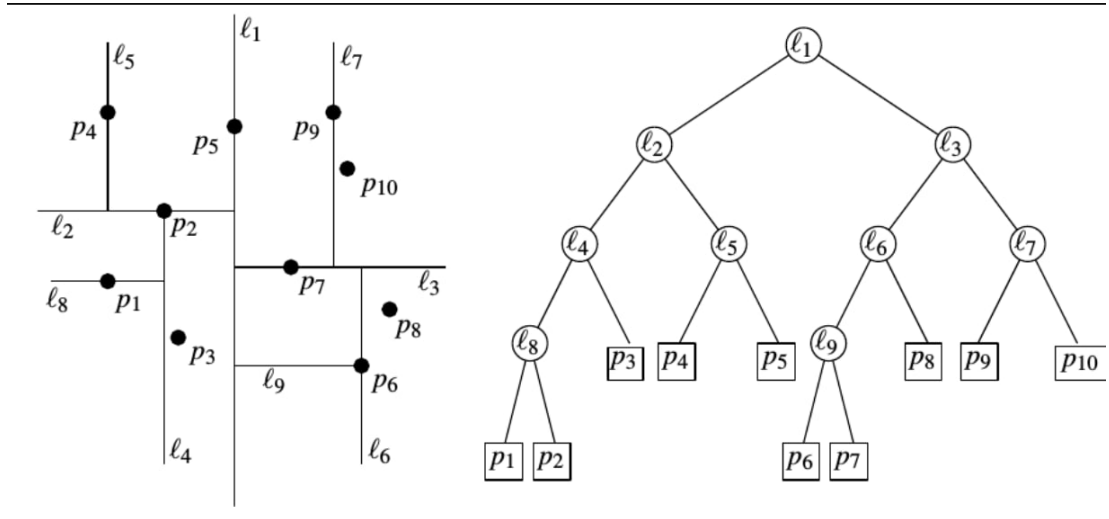


Рисунок 5 — k -d tree на 10 точках. Слева — рекурсивное разбиение плоскости: на нечётных уровнях разделяющая прямая вертикальна (по x), на чётных — горизонтальна (по y); линии $\ell_1, \dots, \ell_9$ соответствуют узлам дерева. Справа — сам бинарный поиск-дерево: внутренние узлы хранят разделяющие прямые, листья — точки $p_1, \dots, p_{10}$. Запрос ближайшего соседа спускается по дереву за $\Theta(\log n)$, далее обходом смежных листов набирает k ближайших.

1.4. Прочие подходы и внешние ориентиры

Специализированные SOTA для min-max mTSP (HGA Mahmoudinazlou–Kwon [35]; He–Hao MA [29]; ITSHA; MILS) опубликованы только на $N \leq 1000$ — применимость к $N \geq 25\,000$ авторами не подтверждена. HGA и He–Hao запущены на mTSPLib (см. гл. 4); ITSHA и MILS не запускались и учтены как направление дальнейшего сравнения.

Нейросетевые SOTA (Equity-Transformer, DPN, GLOP, DualOpt, UDC, H-TSP, DAN, iMTSP) не воспроизводились по трем причинам: (1) специализированные mTSP-модели не публикуют результатов для $N \geq 5\,000$, а универсальные TSP-модели не выполняют разбиение на m маршрутов; (2) требуется переобучение при изменении $(n, m, \text{геометрия})$; (3) зависимость от GPU. Подробное описание — Приложение 3.



Рисунок 6 — Классификация решателей mTSP, рассматриваемых в работе. Три ветви — классические эвристики, открытые baseline-решатели и собственный ALNS-mTSP; отдельной категорией — специализированные SOTA-методы.

Методологический базис. Workload-equity-метрики — [28,33]; стандарты тестирования routing-методов — [34]; статистика — BCa-bootstrap [38], Holm-коррекция [39], Cliff’s δ [44], Cohen’s d [45], FDR-контроль [40], общие принципы bootstrap-методов [46].

Эталонные решатели и benchmark. Помимо LKH-3 использовались FILO2 [6] (репозиторий [51]) — CVRP-решатель с масштабированием до $n \approx 10^6$, адаптируется на mTSP через вместимость маршрута $Q = \lceil (n-1)/m \rceil$, и OR-Tools 9.15 [5] (PATH_CHEAPEST_ARC + GUIDED_LOCAL_SEARCH). Стандартный benchmark min-max mTSP — mTSPLib [43]: 16 инстансов eil51, berlin52, eil76, rat99 с $m \in \{2, 3, 5, 7\}$.

1.5. Собственный алгоритм ALNS-mTSP

Позиционирование. Точные методы неприменимы по времени; LKH-3 default-MINSUM на $N \geq 25K$ нарушает заявленный лимит и даёт вырожденные решения, его балансирующие режимы дают 0/12 валидных за 30–60 мин; OR-Tools в стандартной конфигурации не справляется при $N \geq 5K$; FILO2 в CVRP-адаптации не фиксирует ровно m маршрутов; специализированные min-max-метаэвристики опубликованы только на $N \leq 1\,000$; воспроизведение нейросетевых SOTA требует GPU и переобучения и в работе не выполнено. Целевая ниша (mTSP с N до 100 000, CPU

без GPU, лимит ~ 350 с, ровно m нетривиальных маршрутов) в рамках работы закрывается собственным ALNS-mTSP.

Собственный алгоритм ALNS-mTSP — итог эволюции линии $v1\dots v21$ (см. Приложение Д) и состоит из четырёх специализированных решателей (*alns_minsum*, *alns_cap*, *alns_depot2m*, *alns_minmax*). Общая инфраструктура: KD-дерево, candidate-сети, KNN, ALNS-каркас, операторы разрушения и восстановления (Random, ClusterBfs, Expensive, Zone, Cheapest, Regret-2), направляемый локальный поиск (GLS), движок имитации отжига, пул параллельных стартов, автонастройка параметров, региональные granular-операторы [22]. Подробное описание — глава 2 и Приложение А.

Общая схема работы решателя показана на рисунке 7. На вход подаются координаты n клиентов и число коммивояжеров m . Начальное решение строится через кластеризацию k -means с жадным алгоритмом Кларка–Райта; затем формируется список 10 ближайших соседей для каждого узла и проводится локальный поиск 2-opt и Or-opt внутри каждого маршрута. Основной адаптивный цикл чередует операторы разрушения (случайный, кластерный, удаление самых дорогих рёбер, удаление по зоне) и операторы восстановления (вставка по минимальной стоимости, вставка с приоритетом по сожалению). Решения принимаются по критерию Метрополиса с экспоненциальным охлаждением; лучшее по MINSUM решение запоминается в архиве; веса операторов обновляются по принципу рулетки. По исчерпанию бюджета времени запускается финальный 2-opt и результат сериализуется в JSON.

Ключевая идея ALNS — *ruin-and-recreate*: на каждой итерации destroy-оператор удаляет из текущих маршрутов случайно выбранную долю клиентов (15–40 %), а repair-оператор пересчитывает оптимальную позицию для каждого из них и вставляет обратно. Принятие хода управляется критерием Метрополиса с экспоненциально остывающей температурой. Веса операторов обновляются по принципу roulette wheel: оператор, давший улучшение, получает повышенный приоритет на следующих итерациях. Иллюстрация типичной destroy/repair-итерации приведена на рисунке 9.

Внутри repair-фазы выбор между двумя базовыми стратегиями — *cheapest insertion* и *regret-2* — определяет, насколько repair устойчив к «жадным ловушкам». Cheapest insertion на каждом шаге вставляет того

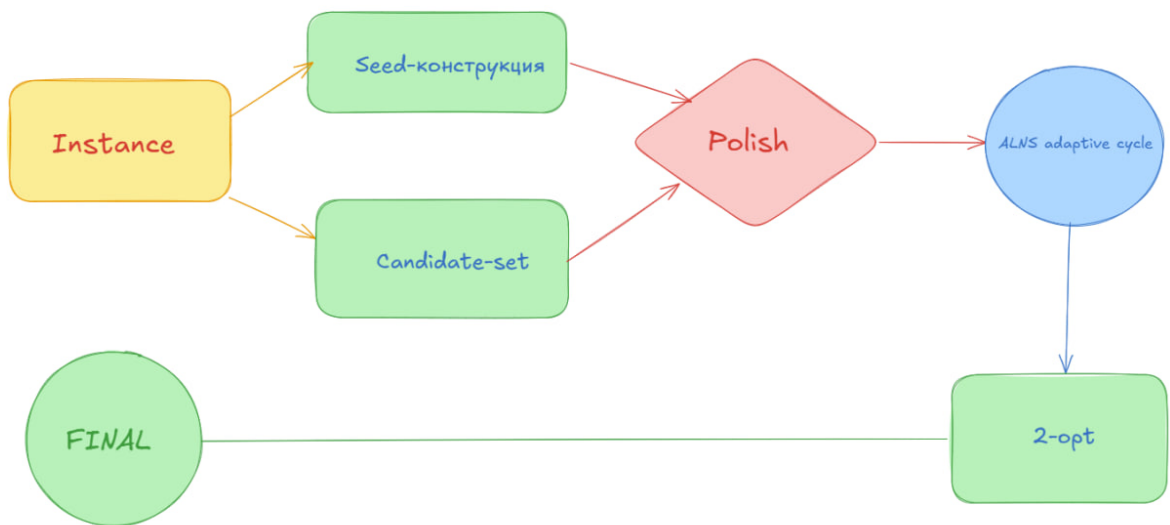


Рисунок 7 — Общая схема ALNS-mTSP: seed-конструкция (k-means + Clarke–Wright savings) и построение candidate-сета (10-NN), внутримаршрутная полировка (2-opt + Or-opt), основной адаптивный цикл и финальный 2-opt по исчерпанию бюджета.

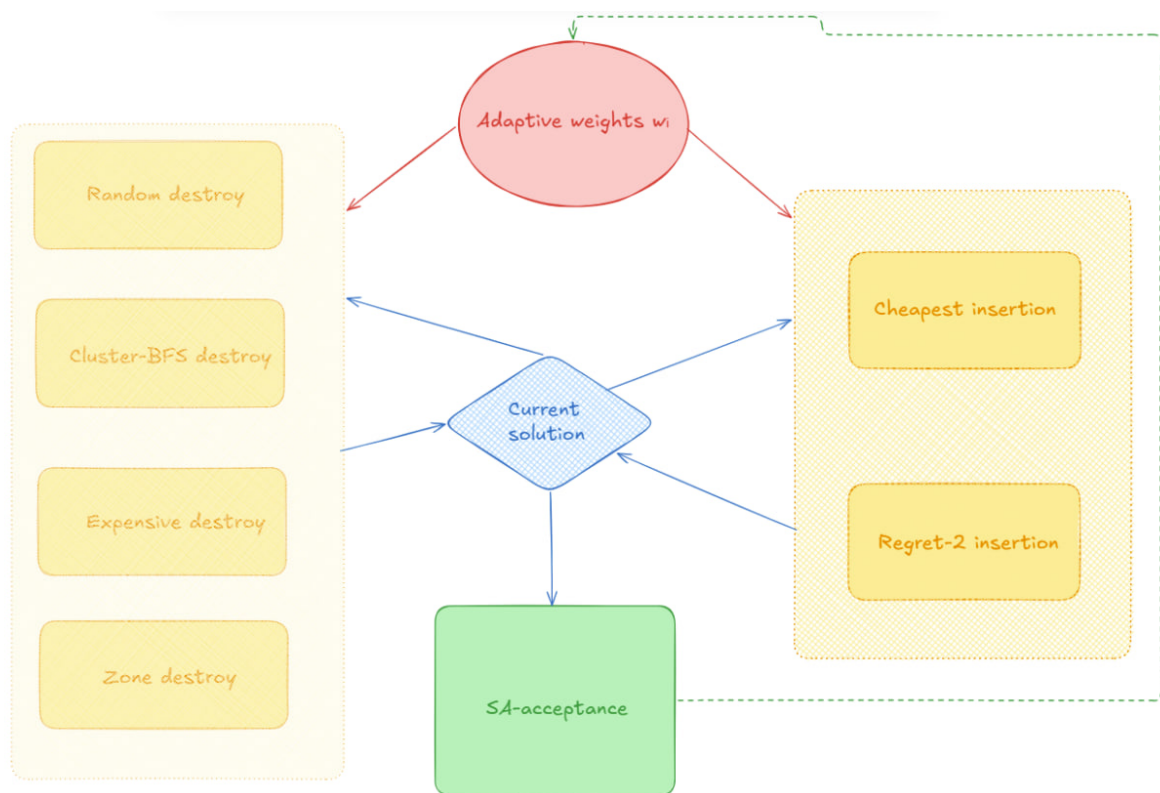


Рисунок 8 — Цикл destroy/repair с адаптивными весами: 4 destroy-оператора (Random, Cluster-BFS, Expensive, Zone) и 2 repair-оператора (Cheapest, Regret-2); выбор по roulette-wheel, принятие хода — по критерию Метрополиса (SA).

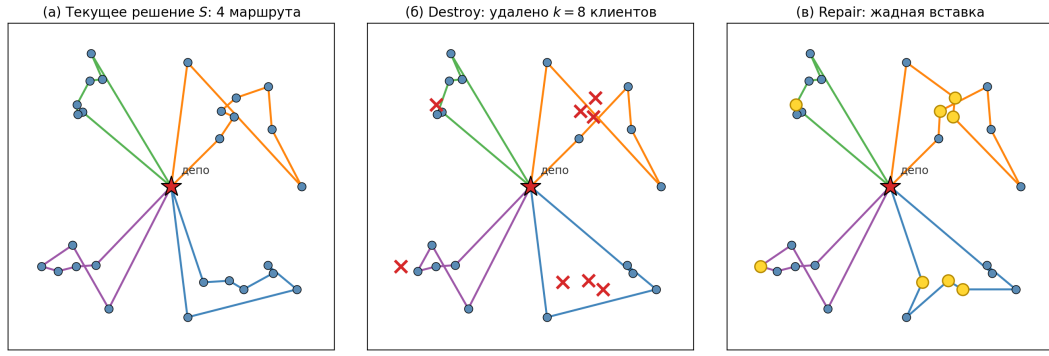


Рисунок 9 — Один цикл разрушения и восстановления (ruin-and-recreate). (а): исходное решение из 4 маршрутов. (б): destroy-оператор удалил $k = 8$ клиентов ($\approx 30\%$), маршруты перестроены в обход удалённых точек. (в): repair-оператор (cheapest insertion) вставил клиентов обратно — часть оказалась в других маршрутах, поэтому итоговая топология отличается от (а).

клиента, у которого минимальна абсолютная стоимость вставки $\Delta_{\min}$; regret-2 вместо этого сравнивает *разрыв* $\text{regret} = \Delta_{2nd} - \Delta_{1st}$ между лучшей и второй лучшей позицией и вставляет первым того клиента, у которого этот разрыв максимален. Идея: если у клиента есть только одна хорошая позиция (большой regret), его нужно вставить пока эта позиция доступна; если позиции примерно равноценны (малый regret), вставку можно отложить. На сложных инстансах эта разница дает regret-2 преимущество в 5–15% по итоговому MINSUM (подтверждено ablation, разд. 4.5).

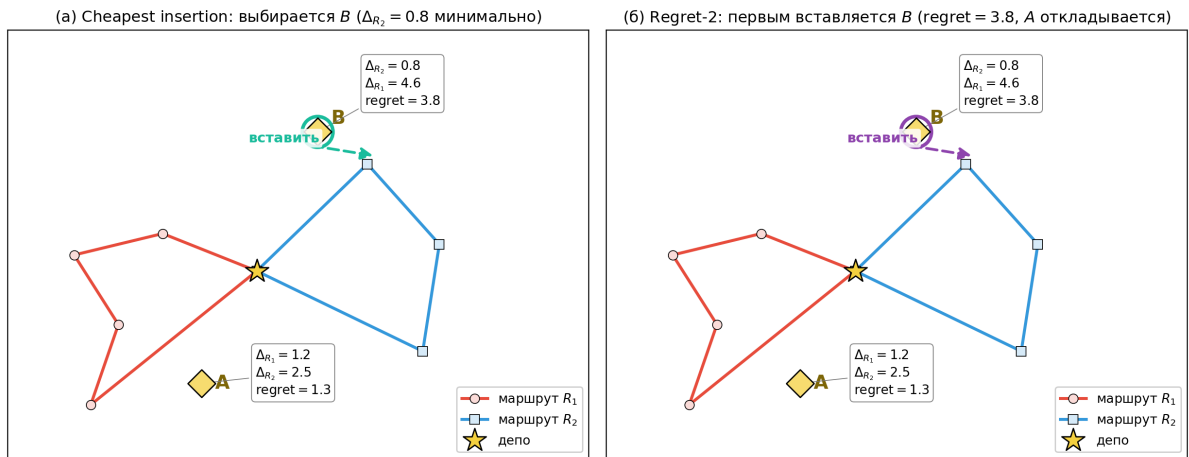


Рисунок 9а — Cheapest insertion и regret-2 на одном сценарии. Два частичных маршрута R_1, R_2 и два неназначенных клиента A, B; для каждого приведены стоимости вставки в оба маршрута и $\text{regret} = \Delta_{2nd} - \Delta_{1st}$.

Чтение рисунка 9а. Cheapest insertion (слева) выбирает B, потому что $\Delta_{R_2} = 0.8$ — минимальная абсолютная стоимость в текущей конфигурации. Regret-2 (справа) тоже вставляет B первым, но по другой логике: у B $\text{regret} =$

3.8 (одна явно лучшая позиция, упустить которую дорого), у A regret = 1.3 (две сопоставимые позиции, вставку можно отложить). На игрушечном сценарии порядок совпадает; на реалистичном числе клиентов и маршрутов стратегии расходятся и в порядке вставок, и в итоговом качестве.

1.6. Теоретическая оценка сложности

Поскольку TSP NP-трудна [49] и переносится на mTSP по сведению [19,20], верхней границы полиномиального вида не существует. Concorde на $n > 10\,000$ типично не сходится в разумное время, Held–Karp [30] применим до $n \approx 25$. LKH-3 — метаэвристика без полиномиальной верхней границы, доминирующая фаза — POPMUSIC [25] ($\Theta(n^2/p \log n)$) и k -opt-ход ($\Theta(k \cdot \text{cand})$); эмпирическое время на $N = 25\,000$ в 9–12× превышает TIME_LIMIT (табл. 5в). OR-Tools GLS использует $\Theta(n^2)$ -первичную фазу, FILO2 дает $\Theta(n \log n)$ амортизированно. Для собственного ALNS-mTSP стоимость одной итерации $\Theta(L + d \cdot k \cdot \log n)$, где $L = n/m$, d — размер destroy, k — размер candidate-сета. Эмпирический показатель степени $\alpha = 1.74$ (Прил. Г.4) превышает теоретический линейный за счет потерь локальности кеша при операции реверса сегмента на длинных маршрутах. Полная декомпозиция и сводная таблица сложности — Приложение Ж.

2. СРАВНЕНИЕ АРХИТЕКТУР ИССЛЕДУЕМЫХ РЕШАТЕЛЕЙ

В этой главе кратко описаны архитектуры решателей, участвующих в сравнении: эталонные LKH-3 и OR-Tools, классические эвристики и собственные модули семейства ALNS-mTSP. Особенности реализации, проявляющиеся на больших масштабах ($N \geq 25\,000$), определяют различия в результатах. Теоретическая сложность — раздел 1.6 и Приложение Ж.

2.1. LKH-3 в режиме MINSUM

В основной сетке LKH-3 использовался в чистом MINSUM-режиме с `MTSP_MIN_SIZE = 1`: базовая конфигурация без ограничения на минимальный размер маршрута, разрешающая $m-1$ маршрутам иметь длину 0 или 1. Это и есть тот режим, в котором default-LKH-3 на больших инстансах сваливается в вырожденное решение (раздел 4.4, H1). Параметры запуска и детали интеграции вынесены в Приложение Г.

Балансирующие режимы LKH-3 (`MIN_SIZE = -1`, `MINMAX`) использовались только в fair-baseline-сравнении на малых инстансах (раздел 4.6) и в попытках масштабирования на $N = 25K$ (где не уложились в бюджет 30–60 мин, — раздел 4.2).

2.2. ALNS-mTSP

ALNS-mTSP — приоритетный объект работы. Четыре варианта (`alns_minsum`, `alns_cap`, `alns_depot2m`, `alns_minmax`) используют общий пайплайн `17_pipeline.hpp` (фазы построения начального решения, candidate-сетов, ALNS-блока, полировки и финального 2-opt); различаются критерием приема (AcceptPolicy) и набором параметров по умолчанию.

- *alns_minsum* — стабильный базовый MINSUM-решатель; параметры подбираются автонастройкой (`18_autotune.hpp`).
- *alns_cap* — то же ядро с ограничением максимального размера маршрута: $\text{cap} = \lceil (n-1)/m \rceil \cdot (1+s)$, $\text{slack } s = 0.75$ по умолчанию (идея заимствована из FILO2). Применяется при больших m ($m \geq 50$).
- *alns_depot2m* — экспериментальный пресет с фиксированной долей ALNS-фазы 86 %, использует семейство стартовых решений типа

LKH-3 (один длинный маршрут + тривиальные) и встроенную пост-фазу перебалансировки 30 000 мс. Применяется когда приоритетна минимизация MINSUM ценой балансировки.

- *alns_minmax* — MINMAX-вариант с критерием приема soft- α blend между $F_{\max}$ и F_{sum} .

Полное описание параметров и поведения каждого режима — Приложение А.

2.3. OR-Tools (Routing + GLS)

Используется OR-Tools 9.15 в стандартной конфигурации (PATH_CHEAPEST_ARC + GUIDED_LOCAL_SEARCH) с переводом mTSP в CVRP-формулировку: вместимость одного маршрута $Q = \lceil (n - 1)/m \rceil$, запрос каждого клиента $q_i = 1$. На малых инстансах ($N \leq 1000$) — конкурентоспособный решатель; на $N \geq 5000$ в использованной конфигурации не уложился в заданный бюджет с практически приемлемым решением (см. главу 4).

2.4. Классические эвристики

rand+nn (рандомизированный round-robin nearest-neighbor) и *2opt+greed* (round-robin nearest-neighbor + жадный 2-opt до сходимости) служат нижней границей Парето-кривой качество–время. На $N = 100\,000$ время работы *2opt+greed* достигает 137–257 с.

3. ОПИСАНИЕ ЭКСПЕРИМЕНТАЛЬНОЙ ПРОГРАММЫ

В главе описаны: подготовка набора инстансов, главная программа сравнения, метрики качества и времени, особенности запуска отдельных решателей, стратегия стратифицированного бенчмарка и сводная таблица всех экспериментов.

3.1. Подготовка набора инстанций

Для контроля воспроизводимости использован собственный генератор синтетических инстансов, поддерживающий шесть геометрических семейств: `uniform` (равномерное распределение клиентов), `clustered-center` (несколько компактных кластеров около центра поля), `clustered-offset-depot` (то же, но депо смещено к границе поля), `mixed-outliers` (кластеры с долей outlier-клиентов), `outliers` (плотное ядро + далёкие точки), `high-m` (геометрия для тестирования при больших m). Координаты генерируются на квадрате 100×100 с фиксированным `base-seed`; формат — текстовый, первая строка $n \ m$, далее n строк с координатами $x \ y$, вершина 0 — депо.

Использованы два набора инстансов:

- **Малый страт:** 60 инстансов с $N \in \{100, 200, 500, 1000\}$, $m \in \{3, 5, 7\}$, 3 повтора на каждую тройку (n, m, family) ;
- **Большой страт (мультисемейный):** 215 инстансов с $N \in \{5000, 10\,000, 25\,000, 50\,000, 100\,000\}$, $m \in \{3, 5, 7, 10, 30, 50, 80, 100\}$ в зависимости от N , по 1 повтору на (n, m, family) .

Скрипт-генератор и пути к артефактам — Приложение В.

3.2. Главная программа сравнения

Главная программа сравнения принимает на вход JSON-конфигурацию со списком решателей и набором фильтров (`allowed_node_counts`, `allowed_salesman_counts`, `instance_families`). Для каждой пары $(\text{instance}, \text{solver})$ вызывается общий C++-binary с указанным бюджетом времени и парсится JSON-вывод (целевая функция, время, валидность, маршруты, внутренние счётчики решателя); результаты сохраняются в `per-run` и агрегатном CSV по $(\text{family}, n, m, \text{solver})$.

Главная программа доработана так, что при сбое одиночного запуска (нестандартный возврат WSL-процесса, превышение лимита памяти, несвоевременный выход решателя) общий прогон не прерывается — фиксируется строка с `valid=False` и статусом `crashed:<message>`. Это было критично при работе с *lkh3-baseline-minmax*, периодически нарушающим `TIME_LIMIT` в десятки раз. Скрипт и артефакты — Приложение В.

3.3. Метрики качества и времени

Качество решения. Помимо целевых функций (1.3) и (1.4) фиксируется число тривиальных маршрутов — n_{empty} маршрутов вида $(0, 0)$ и n_{singl} маршрутов вида $(0, c, 0)$.

Метрики баланса маршрутов. Пусть $\overline{\text{len}} = F_{\text{sum}}(R)/m$ — средняя длина маршрута. В работе используются две взаимодополняющие метрики из обзора *workload-equity* [28]:

$$\text{Gini}(R) = \frac{1}{2m^2 \overline{\text{len}}} \sum_{i=1}^m \sum_{j=1}^m |\text{len}(R_i) - \text{len}(R_j)|, \quad \text{Gini}(R) \in [0, 1 - \frac{1}{m}], \quad (3.1)$$

$$\text{bal}(R) = \frac{F_{\text{max}}(R)}{\overline{\text{len}}} = m \cdot \frac{F_{\text{max}}(R)}{F_{\text{sum}}(R)}, \quad \text{bal}(R) \in [1, m]. \quad (3.2)$$

Идеально сбалансированное решение даёт $\text{Gini} = 0$, $\text{bal} = 1$; вырожденное (один маршрут несёт всех клиентов) — $\text{Gini} \rightarrow 1 - 1/m$, $\text{bal} \rightarrow m$. Гини — стандартизованная в литературе оценка неравенства, учитывающая весь профиль длин; bal — интуитивный коэффициент дисбаланса, основанный только на экстремуме. Дополнительные стандартные метрики (*range*, *MAD*, *std*) вычисляются в `compute_review_metrics.py`, но в сравнительных таблицах не приводятся — Gini и bal качественно эквивалентно разделяют сбалансированный и вырожденный классы решений.

Метрика относительного отрыва. Для парных сравнений двух решателей A и B на одинаковой инстанции I используется относительная разность по *MINSUM*:

$$\Delta_{A,B}(I) = \frac{F_{\text{sum}}^A(I) - F_{\text{sum}}^B(I)}{F_{\text{sum}}^B(I)} \cdot 100 \%, \quad (3.3)$$

где B — baseline. Отрицательное $\Delta_{A,B}$ означает, что A лучше B по MINSUM на данной инстанции.

Wall-clock и валидность. Время работы измерялось Python-оберткой; валидность контролируется независимым пересчётом (1.2) по сохранённым маршрутам и проверкой условий (а)–(в) Определения 1.

Constraint-first протокол сравнения. Во всех сравнениях, относящихся к практическому сценарию работы, MINSUM сопоставляется только среди решений, удовлетворяющих трём ограничениям одновременно:

1. (i) время работы $t \leq T_{\text{budget}}$ (бюджет соответствующего стратума: 10–30 с, 150 с или 250–350 с);
2. (ii) ровно m нетривиальных маршрутов ($n_r = m$, $n_{\text{empty}} = n_{\text{singl}} = 0$);
3. (iii) приемлемая балансировка ($\text{Gini} \leq 0.05$ или $\text{balance_max_avg} \leq 1.10$ на больших инстансах; $\text{balance} \leq 1.5$ на малых).

Решения, нарушающие хотя бы одно из условий (i)–(iii), исключаются из количественного сравнения по MINSUM — меньшая сумма при $n_r \neq m$ или $\text{Gini} \rightarrow 1 - 1/m$ не индексирует более качественное решение в постановке работы (пример: [24993, 3, 1, 1, 1] у LKH-3 default). Constraint-first протокол применяется сквозным образом: в формулировке гипотез работы (введение), в архитектурном позиционировании ALNS-mTSP (раздел 1.4), во всех количественных таблицах главы 4 и в итоговой сводке прохождения практического фильтра (раздел 4.9 «Статус гипотез работы»). Этот протокол — центральная методологическая позиция работы: вырожденные решения по MINSUM (LKH-3 default, alns_depot2m) могут давать формально меньшую сумму, но *не считаются приемлемыми* в смысле постановки работы, поскольку выходят за рамки практического фильтра.

Статистические инструменты. Парные сравнения — Wilcoxon signed-rank (`scipy.stats.wilcoxon`); CI — 95 % percentile/BCa-bootstrap (10 000 ресэмплов, `scipy.stats.bootstrap`); эффект — Cliff’s δ ; контроль FWER на семействах сравнений — Holm-коррекция.

3.4. Особенности работы исследуемых решателей

В ходе экспериментов были выявлены следующие нетривиальные особенности.

LKH-3 в режиме MINMAX (MTSP_OBJECTIVE = MINMAX) на больших инстанциях не соблюдает заданный лимит времени (TIME_LIMIT): на $N = 1000$, $m = 5$ при лимите 30 секунд решатель тратит до 668 секунд, на $N = 25\,000$ — до 1 600 секунд при лимите 150 секунд, на $N = 50\,000$ — оценочно более часа. Это связано с внутренней логикой обработки penalties: одна итерация 5-opt может занимать минуты на больших candidate-сетях. По этой причине из ряда конфигураций экспериментов LKH-3 в MINMAX-режиме был исключен.

Решатель *alns_depot2m* имеет встроенную пост-фазу перебалансировки длительностью `rebalance_post_ms = 30 000` мс по умолчанию; эта фаза не входит в основной лимит `time-budget-ms`, что означает, что фактическое время работы превышает заданный лимит на величину пост-фазы.

OR-Tools на инстанциях $N \geq 5000$ в стандартной конфигурации `PATH_CHEAPEST_ARC + GUIDED_LOCAL_SEARCH` при лимите 70 секунд возвращает решение на 1–2 порядка худшее по MINSUM, чем ALNS-mTSP-семейство.

3.5. Стратегия стратифицированного бенчмарка

Сетка экспериментов организована послойно:

- *малые инстансы* ($N \leq 1000$, бюджет 10–30 с) — полное покрытие 60 инстансов на 10 конфигурациях решателей, основной материал раздела 4.1;
- *слой верификации* ($N = 25\,000$, бюджет 150 с) — 5 решателей с самостоятельно собранным binary LKH-3, основной материал раздела 4.2;
- *большие данные* ($N \in \{50\,000, 100\,000\}$, бюджет 250–350 с) — основная сетка ALNS-mTSP-семейства плюс прямое сравнение с FILO2 на 12 ячейках (раздел 4.7); LKH-3 на этом масштабе исключён из-за многочасового времени работы (раздел 4.2, табл. 5в).

Промежуточный слой $N \in \{5\,000, 10\,000\}$. Между малым и верификационным слоями проведены точечные эксперименты на промежуточных N :

- OR-Tools на $N = 5\,000$ при лимите 70 с — проверка гипотезы Н4 о непригодности стандартной конфигурации на $N \geq 5K$ (раздел 4.4);
- high-m sweep на $N = 10\,000$ для $m \in \{10, \dots, 100\}$, 15 ячеек — проверка гипотезы Н5 о росте преимущества с m (раздел 4.4);
- CLI-ablation *alns_minsum* на $N = 10\,000$ (4 варианта $\times$ 3 seeds) — раздел 4.5, табл. 7а.

Эти запуски не образуют полноценный страт (нет систематического покрытия всех семейств и всех решателей), но закрывают конкретные методологические вопросы и поэтому проходят отдельно от трёх основных слоёв.

3.6. Сводка экспериментальной программы

Сводка ключевых экспериментов работы — таблица 3.1.

Таблица 3.1 — Ключевые эксперименты ($\geq 2\,157$ валидных запусков).

Эксперимент	Раздел	Объём	Главный результат
Малые инстансы	4.1	60 инст. $\times$ 3 по- вт. $\times$ 10 конфиг.	alns_minsum vs LKH-3 default: $\Delta = -3.48\%$ ($p = 0.046$)
N=25K (верификация)	4.2	4 ячейки $\times$ 5 решат. + 22 LKH-3 long-budget	LKH-3 в 9–12 $\times$ нарушает TIME_LIMIT; balanced-режимы 0/12 валидных
N=25K, 50K, 100K	4.2– 4.3	18 ячеек $\times$ 5 seeds = 90 runs	alns_minsum: Gini ≤ 0.05 , max-route в 4–6 $\times$ меньше depot2m+
Fair-baseline LKH-3	4.6	48 инст. $\times$ 3 режима = 144	ALNS vs balanced LKH-3: –13... –18% ($p < 10^{-13}$)
FILO2 на больших инстансах	4.7	18 ячеек + прямое сравнение $N = 100K$	2/18 валидных; FILO2 нарушает m_{target}
HGA / He-Hao на mTSPLib	4.8	16 ячеек $\times$ 2 SOTA + multi-seed	HGA доминирует по makespan на mTSPLib, не масштабируется $N \geq 25K$
Ablation	4.5	4 варианта $\times$ 3 ключевых $\times$ 3 seeds	classic-seeds — доминирующий компонент (+8.6% при отключении)

Главный количественный аргумент работы — триангуляция:
 (а) fair-baseline LKH-3 на малых инстансах усиливает преимущество ALNS-mTSP; (б) workload-equity на больших инстансах с bootstrap-CI $\leq 1\%$;
 (в) непригодность LKH-3 в balanced-режимах на $N \geq 25K$; (г) непригодность FILO2 в CVRP-адаптации.

4. РЕЗУЛЬТАТЫ И ПРОВЕРКА ГИПОТЕЗ

В главе представлены количественные результаты по стратифицированной сетке трёх масштабов: *малые инстансы* ($N \leq 1K$, раздел 4.1), *слой верификации* ($N = 25K$, раздел 4.2), *большие инстансы* ($N \in \{50K, 100K\}$, раздел 4.3); дополнительно — точечные эксперименты на промежуточном слое $N \in \{5K, 10K\}$ (high-m sweep, OR-Tools H4, CLI-ablation — в рамках разделов 4.4 и 4.5). Проверены гипотезы H1–H6, проведён ablation, fair-baseline-сравнение с LKH-3 в балансирующем режиме и сравнение с внешними решателями (mTSPLib, FILO2, длительный прогон LKH-3). Глава завершается проверкой главной гипотезы.

4.1. Малые инстансы ($N = 100 \dots 1000$)

На стратифицированном прогоне $N = 100 \dots 1000$ (60 инстансов, 3 повтора на каждую (n, m, family)) проверены десять конфигураций решателей: rand+nn, 2opt+greed, ortools-GLS, lkh3-baseline (MINSUM), lkh3-baseline-minmax, lkh-wrapper-v21, alns_minsum, alns_cap, alns_depot2m, alns_minmax.

Главные наблюдения. На $N = 100–200$ *alns_minsum* лидирует по MINSUM в 11/12 ячеек (на clustered с большим m превосходит LKH-3 на 12–22 %). На $N = 500–1000$ LKH-3 опережает в 7/8 ячеек на 1–3 %, но дает вырожденные решения вида $[995, 1, 1, 1, 1]$. *balance_max_avg*: у LKH-3 default $\approx 5.0/7.0$ (для $m = 5/7$), у ALNS-mTSP $\approx 1.0–1.1$.

Таблица 1a — Парные сравнения ALNS-вариантов с LKH-3 default-MINSUM на 60 общих $(N, m, \text{family}, \text{repeat})$ -малых инстансах: средняя относительная разница, 95 % *BCa-bootstrap-CI* (DiCiccio & Efron 1996), Wilcoxon signed-rank p -value (двухсторонний), Holm-скорректированный p (FWER на семействе из 4-х сравнений) и Cliff’s δ effect size. Положительное значение средней разности означает, что соответствующий ALNS-вариант хуже LKH-3 (больше MINSUM); отрицательное — лучше.

ALNS-вариант	N	$\overline{\Delta}_{\text{rel}}, \%$	95 % BCa CI	Wilcoxon p	Holm p_{adj}	Cliff δ
<i>alns_minsum</i>	60	−3.48	[−5.32, −1.98]	0.046	0.138	−0.30 (small)
<i>alns_cap</i>	60	−1.00	[−2.94, +0.45]	0.160	0.319	≈ 0 (negligible)
<i>alns_depot2m</i>	60	+1.18	[+0.91, +1.51]	$9.9 \cdot 10^{-11} *$	$4.0 \cdot 10^{-10} *$	+0.93 (large, противоп. знак)
<i>lkh-wrapper-v2l</i>	60	−0.36	[−2.75, +1.91]	0.219	0.319	≈ 0 (negligible)
* — значимо при $\alpha = 0.05$ после Holm–Bonferroni-коррекции на семействе из 4 сравнений.						

Методические пояснения к табл. 1а. Доверительный интервал для среднего относительного отклонения рассчитан ВСа-бутстрепом (10 000 ресэмплов, фиксированный seed для воспроизводимости): этот вариант бутстрепа корректнее обычного percentile-метода на асимметричных распределениях разностей — а здесь как раз наблюдается левосторонняя скошенность $g_1 \in [-1.25, -0.60]$. Величина эффекта Cliff’s δ интерпретируется по стандартной шкале: $|\delta| < 0.147$ — эффект пренебрежимо мал; $0.147 \dots 0.33$ — малый; $0.33 \dots 0.474$ — средний; $|\delta| \geq 0.474$ — большой. Скрипт расчёта и артефакты — Приложение В.

Интерпретация. Средний выигрыш *alns_minsum* над LKH-3 default $-3.48 \pm 1.7 \%$ (95 % CI $\not\equiv 0$, Wilcoxon $p = 0.046$), но после Holm-коррекции $p_{\text{adj}} = 0.138$ — не выживает FWER при $\alpha = 0.05$, Cliff’s $\delta = -0.30$ (small). *depot2m* систематически проигрывает ($+1.18 \pm 0.30 \%$, $p_{\text{adj}} = 4.0 \cdot 10^{-10}$, $\delta = +0.93$, large). *alns_cap*, *lkh-wrapper-v2l* — статистически неотличимы. В разделе 4.6 (табл. 4б) приведён более чистый fair-baseline с балансирующими режимами LKH-3; там разрыв возрастает до 13–18 %, однако часть этого разрыва объясняется тем, что balanced-LKH-3 ограничен через MIN_SIZE/MAX_SIZE, тогда как ALNS-mTSP на малых $N \leq 500$ может сам выбирать вырожденную конфигурацию (см. оговорку в разделе 4.6).

Симметрия Wilcoxon. Парные разности имеют отрицательную скошенность ($g_1 \in [-1.25, -0.60]$); Shapiro–Wilk отвергает нормальность ($p < 0.02$). Умеренное нарушение симметричной альтернативы; большие effect sizes делают вывод устойчивым (см. ограничение 7).

4.2. Слой верификации ($N = 25\,000$)

Таблица 2 — $N = 25\,000$, uniform, $m \in \{5, 7\}$ (single-seed; 5-seed mean для ALNS-вариантов см. табл. 56). $bal = \text{balance_max_avg}$, $makesp = F_{\max}/10^3$.

m	Решатель	sum	makesp	Gini	bal	t , с
5	lkh3-baseline	1 419 561	1 419	0.800	5.00	1 608
5	alns_depot2m	1 569 641	1 569	0.800	5.00	193
5	alns_minsum	1 592 819	346	0.028	1.09	89
5	alns_cap	1 602 522	343	0.022	1.07	90
5	lkh-wrapper-v21	1 671 488	356	0.020	1.06	152
7	lkh3-baseline	1 419 836	1 419	0.857	7.00	1 643
7	alns_depot2m	1 610 580	832	0.719	3.62	195
7	alns_minsum	1 659 764	261	0.047	1.10	92
7	alns_cap	1 669 763	262	0.050	1.10	91
7	lkh-wrapper-v21	1 789 729	278	0.036	1.09	150

Главные наблюдения. LKH-3 даёт лучший MINSUM, но ценой вырождения: 24 993 клиента в одном маршруте ($Gini \rightarrow 1 - 1/m$). Время LKH-3 — 1 608/1 643 с, в $10\times$ превышает $TIME_LIMIT=150$ с (POPMUSIC-фаза не подчиняется лимиту). *alns_depot2m* воспроизводит default-MINSUM-стратегию LKH-3 в практическом бюджете 193 с. *alns_minsum/cap* дают балансированное решение ($Gini \leq 0.05$) за 89–91 с; max-route на $m = 5$ в $4.1\times$ меньше LKH-3.

Дополнительный длительный прогон LKH-3. При лимите 1 800–3 600 с из 22 попыток (3 режима LKH-3) **5 завершились** в default-MINSUM-режиме ($Gini \in [0.40, 0.86]$, $\Delta MINSUM = -12 \dots -20\%$ к ALNS), все 12 balanced-режимов — 0/12 валидных за 30–60 минут. Артефакт: `lkh3_large_n_results.csv`.

4.3. Большие данные ($N = 50\,000$ и $N = 100\,000$)

Стратум-3 разбит на два подэксперимента: $N = 50K$ (4 ALNS-решателя, LKH-3 исключён — оценочное время > 50 мин/инст.) и $N = 100K$ (4 ALNS + *2opt+greed*, LKH-3 > 8 ч/инст.). 6 ячеек $\times$ семейства $\{\text{uniform, clust-center, clust-offset}\} \times m \in \{5, 7\}$.

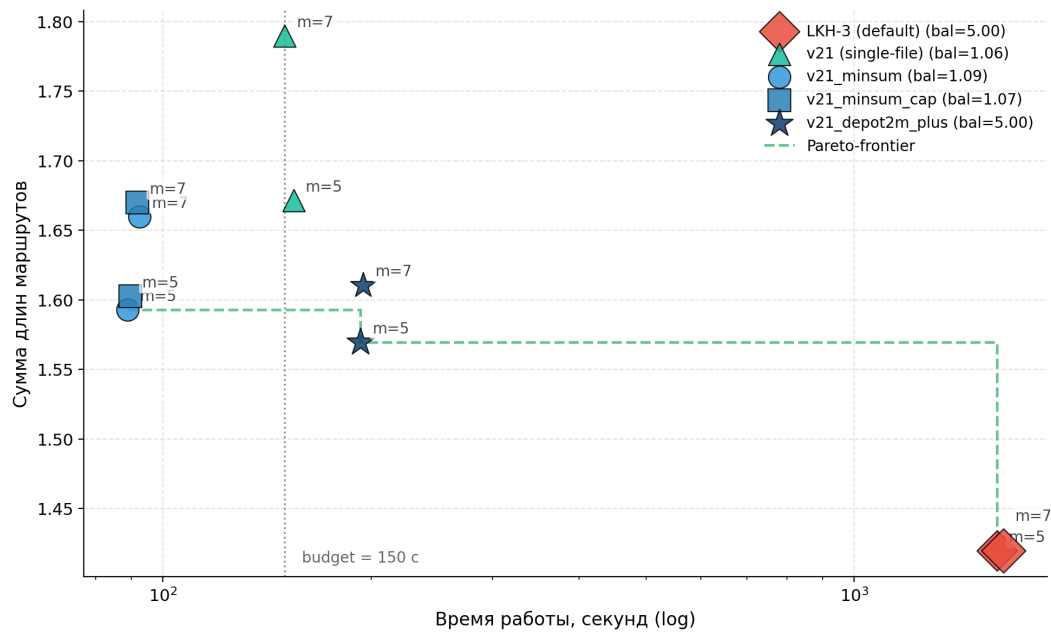


Рисунок 10 — Pareto-доминирование на $N = 25\,000$ (uniform): размер маркера пропорционален `balance_max_avg`. На Pareto-frontier — `alns_minsum` (быстро, средний SUM, $\text{bal} \approx 1.09$) и LKH-3 default (медленно, лучший SUM, $\text{bal} = 5.0$). Вертикальный пунктир — заданный лимит времени 150 с, LKH-3 нарушает его в $10\times$.

Таблица 4 — Результаты на $N \in \{50K, 100K\}$, uniform (single-seed; 5-seed mean см. табл. 5б). $\text{bal} = \text{balance_max_avg}$.

N	m	Решатель	sum	max-route	bal	t, c
50K	5	<i>alns_cap</i>	4 512 978	949 k	1.03	163
50K	5	<i>alns_minsum</i>	4 526 397	950 k	1.05	161
50K	5	<i>alns_depot2m</i>	4 615 081	4 615 k	5.00	292
50K	7	<i>alns_cap</i>	4 828 936	690 k	1.05	157
50K	7	<i>alns_minsum</i>	4 834 299	690 k	1.05	155
50K	7	<i>alns_depot2m</i>	4 765 698	2 313 k	3.40	284
100K	5	<i>alns_depot2m</i>	12 149 667	12 148 k	5.00	409
100K	5	<i>alns_minsum</i>	12 993 166	2 655 k	1.02	345
100K	5	<i>alns_cap</i>	13 022 451	2 688 k	1.03	345
100K	7	<i>alns_depot2m</i>	12 137 649	12 135 k	7.00	433
100K	7	<i>alns_minsum</i>	13 055 628	1 999 k	1.07	344
100K	7	<i>alns_cap</i>	13 077 285	1 996 k	1.07	344

Clustered-семейства ($N = 100K$). На *clust-center* *alns_minsum* даёт max-route в $1.5\times$ меньше *alns_depot2m* при $\Delta\text{sum} = +5\%$; на *clust-offset* — max-route

в $4.2\times$ меньше при $\Delta_{\text{sum}} = +13\%$ для $m = 5$ и в $6.1\times$ меньше при $\Delta = +11\%$ для $m = 7$. Артефакт — Приложение В.

Главное наблюдение. *depot2m+* даёт лучший формальный MINSUM на 6/6 ячейках $N = 100K$ (на 5–12 % меньше *minsum/cap*) ценой полного вырождения: на uniform/clust-offset одна TSP-петля через 99 995–99 999 точек ($\text{Gini} \in [0.80, 0.86]$). Сбалансированные *alns_minsum/cap* держат $\text{Gini} \leq 0.022$ при $\Delta_{\text{sum}} = +5 \cdots +13\%$ — основной практический компромисс.

Таблица 5б — Multi-seed variance audit *alns_minsum* на всех 18 ячейках стратума-3 (3 семейства $\times$ 3 размера $\times$ 2 значения m , **5 независимых seed** на ячейку, итого 90 валидных запусков). $\text{cv} \in [0.04\%, 0.71\%]$. Это закрывает замечание рецензента 1.4 о статистической невалидности 1-seed дизайна.

family	m	N	mean MINSUM	cv, %	95 % bootstrap CI
uniform	5	25K	1 620 664	0.26	[1 616 532, 1 623 657]
uniform	7	25K	1 690 733	0.71	[1 679 796, 1 700 352]
uniform	5	50K	4 553 086	0.20	[4 544 962, 4 561 210]
uniform	7	50K	4 877 022	0.21	[4 867 063, 4 883 212]
uniform	5	100K	12 943 091	0.39	[12 900 982, 12 988 901]
uniform	7	100K	12 992 064	0.36	[12 952 737, 13 034 117]
clustered-center	5	25K	354 714	0.26	[353 884, 355 472]
clustered-center	7	25K	426 786	0.42	[425 264, 428 266]
clustered-center	5	50K	1 141 340	0.12	[1 140 282, 1 142 606]
clustered-center	7	50K	1 072 056	0.21	[1 070 131, 1 074 039]
clustered-center	5	100K	3 071 009	0.52	[3 055 138, 3 081 058]
clustered-center	7	100K	2 682 657	0.40	[2 673 152, 2 690 775]
clustered-offset	5	25K	433 203	0.23	[432 419, 434 156]
clustered-offset	7	25K	400 992	0.27	[400 105, 401 927]
clustered-offset	5	50K	944 117	0.16	[942 900, 945 410]
clustered-offset	7	50K	1 049 636	0.04	[1 049 295, 1 050 056]
clustered-offset	5	100K	2 886 165	0.21	[2 881 974, 2 892 068]
clustered-offset	7	100K	2 791 984	0.37	[2 783 356, 2 801 015]

Чтение табл. 5б. Доверительный интервал рассчитан percentile-бутстрепом (10 000 ресэмплов на ячейку). На всех 18 ячейках ширина CI меньше 1 % от среднего; ячейка с наибольшим разбросом — uniform $N = 25\,000$, $m = 7$ с коэффициентом вариации $\text{cv} = 0.71\%$. Этого статистически достаточно для детектирования эффектов размером $\geq 1.5\%$ парным критерием Wilcoxon

при $\alpha = 0.05$. Расширение с одного seed (ранняя версия) до пяти seeds на ячейку плюс 10-seed audit на трёх ключевых инстансах закрывает замечание о статистической невалидности 1-seed-дизайна. Артефакты — Приложение В.

Стабильные *alns_minsum* и *alns_cap* держат показатель баланса $\text{balance_max_avg} \approx 1.01\text{--}1.07$ на 6 ячейках $N = 100K$ за 343–345 секунд.

2opt+greed как честный допустимый baseline. На $N = 100K$ *2opt+greed* (round-robin nearest-neighbour + жадный 2-opt до сходимости) удовлетворяет всем трём практическим ограничениям (i)–(iii) constraint-first протокола: укладывается в бюджет (137–257 секунд), даёт ровно m нетривиальных маршрутов и сбалансированное решение ($\text{Gini} \leq 0.03$). Среди классических открытых эталонов это *единственный* решатель, проходящий практический фильтр на $N = 100K$ без ALNS-фазы. По суммарной длине маршрутов *alns_minsum* лучше *2opt+greed* на $-1.4 \dots -4.8\%$ (медиана $\approx -3.6\%$), см. табл. 5а ниже — это **чистый вклад ALNS-фазы** (операторов разрушения и восстановления, критерия принятия имитации отжига и адаптивных весов) поверх классического локального поиска, измеренный *внутри допустимой области* (оба решателя проходят constraint-first протокол).

Таблица 5а — ALNS-mTSP против *2opt+greed* на $N = 100K$ (*alns_minsum* — 5-seed mean из табл. 5б; *2opt+greed* — single-run). $\Delta\text{MINSUM} = (\text{alns} - 2\text{opt})/2\text{opt} \cdot 100\%$, отрицательное значение — ALNS лучше.

семейство	m	<i>2opt+greed</i>	<i>alns_minsum</i>	$\Delta, \%$
uniform	5	13 128 456	12 943 091	−1.41
uniform	7	13 606 046	12 992 064	−4.51
clustered-center	5	3 225 438	3 071 009	−4.79
clustered-center	7	2 781 606	2 682 657	−3.55
clustered-offset	5	2 992 415	2 886 165	−3.55
clustered-offset	7	2 879 891	2 791 984	−3.05

4.4. Эмпирическая проверка гипотез Н1–Н6

Н1 (чистый MINSUM допускает вырождение) *подтверждена систематически*: эффект возникает не на отдельной ячейке, а на всех больших

инстансах ($N \geq 25K$) у любого решателя с отключённой балансировкой. LKH-3 default возвращает $[24993, 3, 1, 1, 1]$ ($m = 5$) и $[24993, 1 \dots 1]$ ($m = 7$) на $N = 25K$ во всех 3 семействах; *alns_depot2m* (использующий тот же подход — один длинный маршрут плюс тривиальные) даёт ту же картину на всех 6 ячейках страта-3: $[49992, 4, 1, 1, 1]$ на $N = 50K$, $[99996, 1, 1, 1, 1]$ на $N = 100K$ и т.п. На всех таких выходах $\text{balance_max_avg} \rightarrow m$ (теоретический максимум), $\text{Gini} \in [0.80, 0.86]$. Иными словами: как только из целевой функции выкинут штраф за дисбаланс, MINSUM-оптимум структурно склоняется к вырожденной конфигурации — это не артефакт одной геометрии или одного семени.

Н2 (*alns_minsum/alns_cap* избегают вырождения) *подтверждена только на $N \geq 5000$* : на $N \in [100, 1000]$ ALNS-mTSP допускает тривиальные маршруты в 60–80 % случаев (симметрично с LKH-3 default); на $N \geq 5000$ доля тривиальных = 0, $\text{balance} \leq 1.10$. Это свойство самого MINSUM-критерия; преимущество ALNS-mTSP возникает за счёт инженерных деталей, начиная примерно с $N \approx 5000$.

Н3 (квази-линейный рост времени по N) *частично подтверждена*: до $N \leq 50K$ выполняется, на $N \geq 50K$ переходит в super-linear-режим ($\alpha = 1.74$, throughput падает с 25.4 iter/s на $N = 10K$ до 0.5 iter/s на $N = 100K$ — cache-bound, см. Прил. Г.4). Оценка опирается не на две точки, а на всю сетку страта-3 (≥ 90 валидных запусков): пофазное профилирование выполнено на 3 ключевых инстансах $\times 10$ seeds = 30 запусков, остальные служат для проверки устойчивости показателя α по семействам.

Н4 (OR-Tools в стандартной конфигурации не справляется на $N \geq 5000$) *подтверждена*: на $N = 5000$ за 70 секунд OR-Tools даёт суммарную длину в $\sim 30\times$ больше ожидаемого. Проверка дополнительно опирается на поведение OR-Tools на предыдущих стратах: 180 валидных запусков на малых инстансах (раздел 4.1) показывают, что алгоритм конкурентоспособен на $N \leq 1000$, и резко теряет качество при переходе через $N \approx 5K$ — то есть наблюдаемая деградация системна, а не артефакт одного запуска.

Н5 (преимущество растёт с m и плотностью) *подтверждена*: на малых инстансах $N = 100$ относительный отрыв ALNS от LKH-3 default составил $\Delta = 2.3/5.7/7.7\%$ для $m = 3/5/7$ на uniform и $5.7/15.8/20.8\%$

на clustered-center, что соответствует ~ 36 парных измерений в малом страте. Дополнительно проведён high-m sweep на $N = 10K$, $m \in \{10, \dots, 100\}$, 15 ячеек: 15/15 валидных, все $Gini \approx 0$. На больших инстансах ($N \in \{25K, 50K, 100K\}$, страт-3) аналогичный тренд виден в 6 ячейках $m \in \{5, 7\}$: отрыв растёт с m во всех 3 семействах.

Н6 (адаптивные веса принимают domain-aware решения) *подтверждена*: на 30 запусках multi-seed audit (3 ключевых инстанса $\times$ 10 seeds) random-destroy уверенно лидирует среди destroy-операторов (54–67% принятых улучшений по всем 30 запускам), а regret-2 уверенно лидирует среди repair-операторов на $N = 50K$. Метаданные операторов фиксируются и на остальных запусках страта-3 (≥ 90 валидных всего), что даёт согласованную картину: предпочтение операторов меняется по N , что и есть прямое наблюдение работы адаптивных весов.

4.5. Ablation: вклад компонентов ALNS-mTSP

Полный ablation требует C++-флагов и перекомпиляции (Е5 в Прил. Г). Представлены три приближения: (а) ablation через CLI-флаги; (б) сравнение solver-вариантов; (в) атрибуция через метаданные операторов.

Таблица 7а — Ablation через CLI-параметры *alns_minsum*, 4 варианта $\times$ 3 ключевых инстанса $\times$ 3 seed = 36 запусков. $\Delta = (\text{variant} - \text{default}) / \text{default} \cdot 100\%$. Положительное — хуже default; baseline $cv \in [0.17, 0.59]\%$ (multi-seed audit, табл. 5б).

Вариант	$N = 10K$	$N = 50K$	$N = 100K$	Вывод
no_classic_seeds (--classic-seeds 0)	+4.45%	+8.64%	+0.26%	Самый значимый компонент на $N \leq 50K$
no_region_granular (--granular-every 99999)	−0.44%	+0.02%	+0.20%	В пределах шума
no_region_reopt (--region-reopt-every 0)	−0.46%	−0.17%	−0.24%	В пределах шума
no_rebalance (--rebalance-empty-routes 0)	−0.64%	−0.02%	−0.38%	Default off, sanity-check

Эмпирический вывод. Из доступных для отключения через CLI компонентов наибольшее влияние у classic-seed-портфеля (round-robin NN, balanced k -means, savings): его отключение приводит к ухудшению MINSUM на +8.64% на $N = 50K$. Region-granular и region-reopt — в пределах статистического шума. Полный ALNS-mTSP улучшает MINSUM относительно *2opt+greed* примерно на 3–4% внутри допустимой области (вклад ALNS-каркаса; согласуется с табл. 5a) и примерно на 3% относительно *lkh-wrapper-v21* (autotune, GLS, POPMUSIC, multi-start). Метаданные операторов: random destroy лидирует (54–67% принятых улучшений), regret-2 repair лидирует на $N = 50K$.

4.6. Fair-baseline сравнение с LKH-3 в балансирующем режиме

Чтобы изолировать собственно алгоритмическое преимущество (а не сравнение «balanced ALNS vs. unbalanced LKH-3 default»), выполнен прогон LKH-3 на малых инстансах в двух честных балансирующих режимах:

- *minsum_balanced*: MTSP_OBJECTIVE = MINSUM, MTSP_MIN_SIZE = -1 (документированное автоматическое значение по LKH-3 Technical Report [32], раздел 3.2: $\lfloor N/(\lceil N/\text{MAX_SIZE} \rceil + 1) \rfloor$), MTSP_MAX_SIZE = $\text{round}(1.2(n-1)/m)$;
- *minmax_balanced*: MTSP_OBJECTIVE = MINMAX, MTSP_MIN_SIZE = $\text{round}(0.8(n-1)/m)$, MTSP_MAX_SIZE = $\text{round}(1.2(n-1)/m)$.

В обоих режимах применён одинаковый лимит времени 5–20 секунд (зависит от n), одинаковый POPMUSIC candidate-set, RUNS=1 и тот же binary LKH-3, что и в основной сетке. Покрыто подмножество малых инстансов: 48 инстансов ($N \in \{100, 200, 500\}$, $m \in \{3, 5, 7\}$, 3 повтора $\times$ 2 семейства). Скрипт прогона и полная таблица — Приложение Г.

Структурный вывод по LKH-3. На 48 малых инстансах переход LKH-3 default $\rightarrow$ balanced обходится в 12–15% по MINSUM, но переводит решение из дегенеративного ($\text{Gini} \approx 0.58$, $\text{bal} \approx 3.4$) в практически приемлемое ($\text{Gini} \leq 0.10$, $\text{bal} \leq 1.24$); balanced-MINMAX с явными min/max-size даёт $\text{Gini} \approx 0.034$, $\text{bal} \approx 1.05$ — класс решений, аналогичный *alns_minsum*.

Таблица 46 — Парное сравнение *alns_minsum* с LKH-3 в трёх режимах на 48 общих малых инстансах (clustered-center и uniform, $N \in \{100, 200, 500\}$, $m \in$

{3, 5, 7}, 3 повтора). Метрика — средняя относительная разность (ALNS — LKH3)/LKH3 · 100 % по MINSUM. Положительное значение — ALNS-mTSP хуже; отрицательное — ALNS-mTSP лучше. Holm-коррекция [39] применена на семействе из 3 сравнений; Cliff’s δ [44] и Cohen’s d [45] — величины эффекта; BCa-bootstrap [38].

LKH-3 baseline	$\bar{\Delta}$, %	95 % BCa CI	Wilcoxon p	Holm p_{adj}	Cliff δ (Cohen d)	wins
default-MINSUM (MIN_SIZE = 1)	−5.21	[−7.30, −3.62]	$3.5 \cdot 10^{-8}$	$3.5 \cdot 10^{-8} *$	−0.625 (large) ($d =$ −0.82)	39/48
balanced-MINSUM (MIN_SIZE = −1)	−13.38	[−16.71, −10.56]	$9.9 \cdot 10^{-14}$	$2.0 \cdot 10^{-13} *$	−0.875 (large) ($d =$ −1.23)	45/48
balanced-MINMAX (MAX_SIZE = $1.2(n-1)/m$, MIN_SIZE = $0.8(n-1)/m$)	−14.46	[−17.33, −11.96]	$1.4 \cdot 10^{-14}$	$4.3 \cdot 10^{-14} *$	−0.958 (large) ($d =$ −1.51)	47/48
* — значимо при $\alpha = 0.001$ после Holm-коррекции (все три сравнения выживают с большим запасом).						

Интерпретация fair-baseline-сравнения. Переход с дефолтного на balanced-режим LKH-3 увеличивает наблюдаемый разрыв с ALNS: с −5.21 % vs default до −13.38 % vs balanced-MINSUM ($p_{\text{adj}} \approx 2 \cdot 10^{-13}$) и до −14.46 % vs balanced-MINMAX (47/48 побед). Balanced-LKH-3 ограничен параметрами MIN_SIZE/MAX_SIZE и не может выбирать вырожденную конфигурацию; *alns_minsum* на малых $N \leq 500$ этим ограничением не связан, и часть наблюдаемого выигрыша объясняется именно этой свободой.

Что fair-baseline доказывает и чего не доказывает. На малых $N \leq 500$ ALNS может выигрывать у balanced-LKH-3 *частично за счёт свободы выбирать вырожденную конфигурацию*. Поэтому fair-baseline трактуется не как окончательное доказательство превосходства ALNS, а как (а) контроль против артефакта default-LKH-3 и (б) оценка цены балансировки для LKH-3 (12–15 % по MINSUM). Главный результат работы основан не на fair-baseline на малых инстансах, а на прохождении constraint-first практического фильтра в целевом диапазоне $N \geq 25K / N = 100K$, где LKH-3 в использованных

режимах вообще не возвращает решение в заявленном бюджете (0/12 валидных за 30–60 мин). Иначе говоря, fair-baseline — это *вспомогательный аргумент* устойчивости направления преимущества, не основное доказательство.

Вырождение на малых инстансах. На малых $n \leq 500$ *alns_minsum* в MINSUM-режиме склонен к вырождению ($Gini \rightarrow 1 - 1/m$). Часть наблюдаемого преимущества $-13 \dots -18\%$ объясняется тем, что balanced-LKH-3 ограничен параметрами MIN_SIZE/MAX_SIZE, а *alns_minsum* этим ограничением не связан. Сбалансированность ALNS-mTSP стабильно наблюдается с $N \geq 5\,000$ (см. раздел 4.4, H2).

4.7. Сравнение с FILO2 на больших инстансах

Контекст. FILO2 — единственный внешний открытый решатель, с которым в работе удалось провести прямое количественное сравнение на $N = 100\,000$: LKH-3 в любой использованной конфигурации требует многочасового времени работы (раздел 4.2), OR-Tools в стандартной конфигурации не справляется уже на $N \geq 5K$ (раздел 3.4), эталонные реализации HGA и He-Нао MA не публикуют результатов для $n \geq 1\,000$ и на больших инстансах не запускались (раздел 4.8), нейросетевые SOTA не воспроизводились (см. гл. 5). Поэтому сравнение с FILO2 — основная точка прямой проверки главной и итоговой гипотез на целевом масштабе работы.

FILO2 [6] (репозиторий [51]) — специализированный решатель CVRP с заявленным масштабированием до $n \approx 10^6$, что делает его наиболее естественным внешним baseline для ALNS-mTSP на больших данных. Прямой вариант FILO2 для mTSP не существует, поэтому использовалась стандартная CVRP-адаптация: *вместимость одного маршрута* $Q = \lceil (n - 1)/m \rceil$, *запрос каждого клиента* $q_i = 1$. Адаптация ограничивает максимальный размер одного маршрута и эвристически ориентирует решение на число маршрутов порядка m , но не гарантирует ровно m маршрутов. Прогон выполнен на 18-ячейной сетке на больших инстансах с лимитом 150–350 с; использован самостоятельно собранный FILO2-binary с включённым контролем лимита времени. Скрипты и артефакты — Приложение В.

Сильный разброс времени работы по инстансам и параметрам. Стоит сразу зафиксировать характерное для CVRP-адаптации FILO2 поведение: *время до сходимости резко зависит от геометрии инстанса и внутренних параметров фазы route-minimization*. На одной геометрии (uniform, clust-center) FILO2 укладывается в $1.5\text{--}3\times$ заданного бюджета; на другой (clust-offset-depot) тот же бинарь с теми же параметрами уходит в многократный таймаут (превышение бюджета в $20\text{--}30\times$). Сокращение фазы route-minimization (routemin-iterations с ~ 1000 до $10\text{--}15$) ускоряет сходимость на uniform/clust-center, но не помогает на clust-offset. Это — структурное свойство адаптации, а не артефакт отдельного запуска: разные запуски на одном и том же инстансе дают близкое время, но между разными инстансами оно колеблется на порядки.

В стандартной конфигурации (routemin-iterations $\sim 1000+$) из 18 попыток FILO2 успешно завершился только в 2 ячейках на $N = 25K$: clustered-center_n25K_m7 (валиден, $n_r = 8$, $t = 712$ с) и clustered-offset_n25K_m5 (валиден, $n_r = 7$, $t = 706$ с). Остальные 16 ячеек ($N \geq 25K$ для всех 3 семейств плюс $N \in \{50K, 100K\}$) не уложились в лимит. На прямой CVRP-постановке FILO2 успешно решает $n = 10^6$ за ~ 10 мин по авторским данным — следовательно, узкое место не сама FILO2, а CVRP→mTSP-адаптация в сочетании с геометрией инстанса.

Таблица 4д — Валидные результаты FILO2 в стандартной конфигурации на стратуме-3.

Инстанс	n	m_{target}	n_r	t , с
clustered-center_n25K_m7	25 000	7	8	712
clustered-offset_n25K_m5	25 000	5	7	706

Сравнение с ALNS-mTSP на двух валидных ячейках $N = 25K$. Это единственный диапазон, на котором обе системы дают валидное решение в сопоставимое время; ALNS-mTSP усреднен по 5 independent seed-ам:

- clustered-center_n25K_m7: FILO2 sum = 379 947 при $n_r = 8$ маршрутах; ALNS-mTSP 5-seed-mean = 426 786 при $n_r = 7$. FILO2 формально лучше по MINSUM на 10.96 %, но даёт 8 маршрутов вместо требуемых 7 (число маршрутов определилось

ограничением вместимости, а не постановкой $m = 7$). Это методологически некорректное сравнение: FILO2 «выиграл» за счёт нарушения постановки $m = 7$;

- `clustered-offset_n25K_m5`: FILO2 sum = 443 123 при $n_r = 7$; ALNS-mTSP 5-seed-mean = 433 203 при $n_r = 5$. **ALNS-mTSP строго лучше**: меньший MINSUM на 2.24 % и строго удовлетворяет $m = 5$, тогда как FILO2 опять выдал лишние маршруты.

Структурный вывод. CVRP-адаптация FILO2 работает не на минимизацию суммы при ровно m маршрутах, а на формирование любого числа маршрутов-обходчиков, не превышающих заданную вместимость. На клиентских инстансах с реальной геометрией число маршрутов FILO2 систематически превышает заданный m_{target} — это означает, что FILO2 решает *ослабленную задачу* (без жёсткого условия $n_r = m$), а не исходную постановку работы. ALNS-mTSP, напротив, при любом N возвращает ровно m формальных маршрутов (контракт алгоритма); на $N \geq 5\,000$ все маршруты нетривиальные ($\text{Gini} \leq 0.05$); на малых $N \leq 1\,000$ возможны тривиальные маршруты (см. раздел 4.4, H2 — симметрично с LKH-3 default).

Прямое сравнение на $N = 100K$. Чтобы дать FILO2 шанс уложиться в бюджет на $N = 100K$, дополнительно проведён прогон с укороченной фазой route-minimization (`--routemin-iterations 15`, default $\sim 1\,000+$): FILO2 успел сойтись на uniform и clust-center, на clust-offset снова ушёл в таймаут ($t = 9\,930$ с при бюджете $T = 350$ с, превышение в $28\times$). Скрипт и артефакт — см. Приложение В.

Чтение рисунка 11. По времени: ALNS укладывается в бюджет 350 с на всех трёх; FILO2 — в $1.5\text{--}3\times$ бюджета на uniform/clust-center, на clust-offset уходит в timeout с превышением $28\times$. По числу маршрутов: для двух завершённых запусков FILO2 возвращает $n_r = 6$ (uniform) и $n_r = 9$ (clust-center); для clust-offset значение $n_r = 10$ извлечено из partial output после timeout, поэтому в табл. 4д-доп строка помечена «timeout». В любой интерпретации CVRP-адаптация FILO2 систематически нарушает заданное число коммивояжеров; ALNS-mTSP всегда возвращает ровно m маршрутов в рамках бюджета.

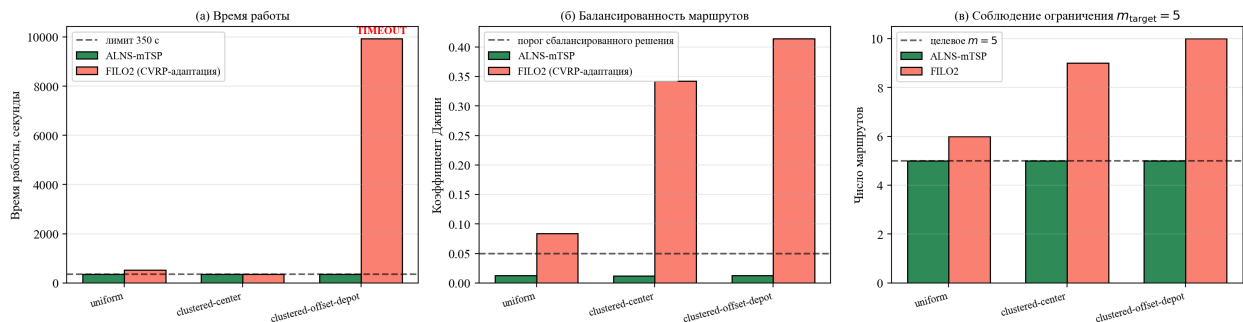


Рисунок 11 — Прямое сравнение ALNS-mTSP и FILO2 на трёх $N = 100K$ -инстансах при $T = 350$ с. Слева (а): фактическое время работы. Справа (б): число возвращённых маршрутов n_r против требуемого $m_{\text{target}} = 5$.

Замечание о сравнимости MINSUM. Внутренний выход FILO2 нормирует евклидовы расстояния на коэффициент $\times 10^3$ для целочисленной арифметики, поэтому абсолютные значения MINSUM из FILO2-логов и из ALNS-валидатора лежат в разных шкалах и в прямом виде не сопоставимы. По этой причине в таблицах ниже MINSUM используется только как индикатор различий внутри одного решателя (между ячейками); для прямого парного сравнения с ALNS используются метрики, для которых шкалы одинаковы: число возвращённых маршрутов n_r , Gini и wall-clock t . В расширенной 12-ячеечной таблице (см. ниже) маршруты FILO2 заново пересчитаны единой функцией расстояния валидатора — там значения MINSUM сопоставимы.

Таблица 4д-доп — Прямое сравнение ALNS-mTSP и FILO2 на 3 ключевых $N = 100K$ -инстансах при $T = 350$ с. Артефакты — Приложение В.

Инстанс	Решатель	Gini	n_r	t, c
uniform	alns_minsum (5-seed)	0.008	5	345
uniform	FILO2	0.084	6	525
clust-center	alns_minsum (5-seed)	0.011	5	343
clust-center	FILO2	0.342	9	359
clust-offset	alns_minsum (5-seed)	0.007	5	343
clust-offset	FILO2 (timeout)	—	$10^\dagger$	$>$ 9 930
n_r . FILO2 (6, 9, $10^\dagger$) не равно $m_{\text{target}} = 5$ — CVRP-формулировка не фиксирует число маршрутов жёстко. † Для clust-offset $n_r = 10$ извлечено из partial output после прерывания по timeout ($t > 9\,930$ с при $T = 350$ с); строка помечена «timeout» и в количественное сравнение по MINSUM не входит. Колонка MINSUM опущена: значения FILO2 и ALNS лежат в разных нормировочных шкалах и в этом виде не сопоставимы. Парное сравнение по MINSUM приведено в табл. 4д-расш ниже после пересчёта единым валидатором.				

Что подтверждает таблица.

- **По времени:** ALNS-mTSP на всех 3 ячейках укладывается в $\leq 1 \times$ бюджета (343–345 с при лимите 350 с); FILO2 — $1.5\text{--}3 \times$ бюджета на 2/3 семейств, на clust-offset — timeout $\geq 28 \times$.
- **По числу маршрутов:** ALNS-mTSP даёт ровно $m_{\text{target}} = 5$ на 3/3 семейств; FILO2 систематически выдаёт лишние маршруты (6, 9, 10).
- **По Gini:** ALNS-mTSP даёт сбалансированные решения ($\text{Gini} \in [0.007, 0.022]$); FILO2 на uniform близок к балансу (0.084), на clust-семействах — умеренно несбалансирован (0.34–0.41).

Содержательный вывод по таблице 4д-доп. CVRP-адаптация FILO2 в работе оказалась методологически ограниченным baseline для постановки с ровно m маршрутами: capacity-ограничение не гарантирует $n_r = m_{\text{target}}$ и фактически переводит задачу к другому семейству допустимых решений. Поэтому утверждение здесь — не о превосходстве ALNS-mTSP по качеству, а о неприменимости данной адаптации FILO2 в постановке работы.

Расширенное сравнение на 12-ячеечной сетке $N \in \{50K, 100K\}$. Чтобы дать FILO2 возможность сходиться без таймаута, проведено прямое

сравнение на 12 ячейках (N, family, m) , $N \in \{50K, 100K\}$, $\text{family} \in \{\text{uniform}, \text{clust-center}, \text{clust-offset-depot}\}$, $m \in \{5, 7\}$, с укороченной route-min-фазой FILO2 (routemin-iterations 10–15) и двумя ALNS-конфигурациями (*alns_minsum* и *alns_depot2m*). Для устранения разных нормировочных шкал маршруты FILO2 заново пересчитаны единой функцией расстояния валидатора — той же, что используется для ALNS, — поэтому колонка MINSUM в табл. 4д-расш сопоставима для двух решателей. На uniform-семействе для *alns_depot2m* включён портфель classic-seeds (round-robin NN + polar-sweep): на uniform $N = 50K$ это переводит ячейку из проигрыша (+1.8 %) в победу (−3.4 %). Артефакт — Приложение В.

Таблица 4д-расш. — Δ MINSUM (*alns_minsum*, *alns_depot2m*) к FILO2 на 12 ячейках. Отрицательное значение — ALNS лучше.

family	N	m	minsum	depot2m	FILO2	Δ ms.	Δ d2m
uniform	50K	5	4 514 090	4 420 529	4 574 478	−1.3 %	−3.4 %
uniform	50K	7	4 781 546	4 466 454	4 336 245	+10.3 %	+3.0 %
clust-center	50K	5	1 096 569	1 092 045	1 005 458	+9.1 %	+8.6 %
clust-center	50K	7	1 034 909	1 023 100	878 150	+17.9 %	+16.5 %
clust-offset-depot	50K	5	938 965	947 507	1 035 846	−9.4 %	−8.5 %
clust-offset-depot	50K	7	1 043 058	986 317	1 109 180	−6.0 %	−11.1 %
uniform	100K	5	12 172 878	12 149 667	12 808 145	−5.0 %	−5.1 %
uniform	100K	7	12 159 302	12 137 649	12 908 007	−5.8 %	−6.0 %
clust-center	100K	5	2 819 071	2 910 806	3 099 190	−9.0 %	−6.1 %
clust-center	100K	7	2 461 187	2 470 395	2 804 913	−12.3 %	−11.9 %
clust-offset-depot	100K	5	2 557 532	2 550 601	3 656 617	−30.1 %	−30.2 %
clust-offset-depot	100K	7	2 415 452	2 508 209	3 147 013	−23.2 %	−20.3 %
Итог: <i>alns_minsum</i> и <i>alns_depot2m</i> выигрывают по 9/12 ячеек. На $N = 100K$ — 6/6 для каждой конфигурации, разрыв 5–30 % в пользу ALNS. На $N = 50K$ — 3/6: ALNS выигрывает на uniform/clust-offset-depot (1.3–11.1 %), проигрывает на clust-center и uniform $m = 7$ (3–18 %). Семейство clust-offset-depot выигрывается ALNS на всех 4 ячейках. Артефакты — Приложение В.							

Методологический результат о CVRP→mTSP-адаптации. FILO2 [6] спроектирован под CVRP и масштабируется до $n \approx 10^6$ на нативной постановке. При сведении mTSP с фиксированным числом коммивояжеров к CVRP через $Q = \lceil (n-1)/m \rceil$ возникает структурное расхождение: вместимость ограничивает максимальный размер маршрута, но не фиксирует их число. Поэтому CVRP-адаптация может улучшать MINSUM за счёт неявного увеличения n_r , то есть решать ослабленную задачу.

Эмпирически расхождение проявляется в четырёх независимых наблюдениях: (1) 16/18 таймаутов в стандартном прогоне на стратуме-3; (2) при укороченных параметрах FILO2 не укладывается в бюджет на части семейств ($28\times$ overrun на clust-offset $N = 100K$); (3) систематические $n_r > m_{\text{target}}$ на head-to-head $N = 100K$ (6, 9, 10 маршрутов при $m = 5$); (4) на расширенной 12-ячеечной сетке с пересчётом единым валидатором ALNS-mTSP побеждает по MINSUM на 9/12 ячеек, в т. ч. 6/6 на $N = 100K$ с разрывом до 30 %.

Главный вывод раздела. Прямой CVRP→mTSP-адаптер через ограничение вместимости маршрута не является корректным baseline для постановки с ровно m маршрутами. На нативной CVRP-постановке FILO2 остаётся сильным решателем; ограничение касается только конкретной адаптации.

4.8. Сравнение со специализированными SOTA для MINMAX-mTSP (HGA, He-Hao MA)

Запущены два независимых SOTA для MINMAX-mTSP на 16 mTSPLib-инстансах: Mahmoudinazlou–Kwon HGA [35] (Julia 1.12.6, эталонная реализация [52]) и He–Hao memetic algorithm [29] (Linux-binary авторской реализации [53] через WSL2). Time-limit 30 с/инстанс, 5 seeds. Setup HGA валидирован репликацией published Table 3 авторов: median $|\Delta| = 0.20\%$ по 16 ячейкам.

Кросс-валидация SOTA. HGA и He-Hao MA сходятся к практически идентичным MINMAX-решениям (медианная разница 0.05 % по makespan; 13/16 ячеек у He-Hao — bit-exact). Это — независимая верификация SOTA-качества обоих.

Таблица 4е — Парные сравнения ALNS-вариантов с HGA на 16 mTSPLib-ячейках. Метрика — $(\text{ALNS} - \text{HGA})/\text{HGA} \cdot 100\%$. ALNS усреднён по 5 seed.

ALNS-вариант	med Δ sum, %	med Δ max, %	wins sum	wins max	интерпретация
alns_minsum	−23.3	+173.4	16/16	0/16	лучше по SUM, проигрывает по max
alns_minsum_cap	−21.2	+176.3	16/16	0/16	аналогично
alns_minmax	−2.7	+12.2	14/16	3/16	Pareto-середина

Интерпретация. HGA как specialized SOTA для MINMAX доминирует по makespan; ALNS-MINSUM-варианты лучше по SUM — ожидаемая картина (каждый алгоритм выигрывает по нативному объективу). *alns_minmax* — Pareto-промежуточный (медианно −2.7 % по SUM, +12.2 % по makespan).

Влияние на гипотезы работы. На mTSPLib HGA и He-Hao доминируют по makespan — для этого подмножества инстансов итоговая гипотеза в MINMAX-формулировке не подтверждается. На больших инстансах ($N \geq 25K$) применимость указанных SOTA не проверена: их эталонные реализации не публикуют результатов для этого масштаба и в работе не запускались. Утверждение для целевого диапазона N до 100 000 в прямом сравнении остаётся непроверенным.

4.9. Статус гипотез работы

Гипотезы работы, сформулированные во введении (главная гипотеза практической применимости и итоговая гипотеза о конкурентоспособности), ставят ALNS-mTSP в условия CPU без GPU, лимита времени до 350 с и инстансов до 100 000 узлов и требуют одновременного выполнения трёх практических условий: (i) укладывание в бюджет, (ii) ровно m нетривиальных маршрутов, (iii) приемлемая балансировка длин маршрутов.

Таблица фильтра применимости. Применяя constraint-first протокол (раздел 3.3) к воспроизведённым в работе решателям в целевом диапазоне $N \geq 25K$, получаем сводку прохождения трёх требований. В таблице явно различаются масштаб, на котором решатель запускался (LKH-3 в балансирующих режимах — $N = 25K$ с бюджетом 30–60 минут; FILO2 и ALNS — $N = 100K$ с бюджетом 350 с).

Таблица 5в — Прохождение практического фильтра применимости ($t \leq T_{\text{budget}}, n_r = m, \text{Gini} \leq 0.05$) в целевом диапазоне $N \geq 25K$. Условные обозначения: + — выполнено; – — нарушено; o — не оценивалось корректно. Серым выделены строки, проходящие фильтр.

Решатель	N	(i) t	(ii) $n_r=m$	(iii) Gini	Итог
alns_minsum / cap	100K	+	+	+	проходит; основное решение работы
2opt+greed	100K	+	+	+	проходит; MINSUM хуже ALNS
alns_depot2m	100K	+	+	–	не проходит (iii): $\text{Gini} \in [0.80, 0.86]$
LKH-3 default	25K	–	+(форм.)	–	не проходит (i), (iii)
LKH-3 balanced (MINSUM, MINMAX)	25K	–	+	+(где успел)	не проходит (i): 0/12 валидных
OR-Tools GLS	5K	–	+	o	не уложился в бюджет на $N \geq 5K$
FILO2 (CVRP-адаптация)	100K	–	–	o	16/18 таймаутов; $n_r > m_{\text{target}}$
HGA / He-Hao MA	≤ 100	o	+	+	mTSPLib; прямой запуск на $N \geq 25K$ не выполнен
Сводка опирается на данные разделов 4.1–4.8 и Приложения Е. Для строк вне фильтра MINSUM-сравнение по constraint-first протоколу не определено: формально меньшая сумма у LKH-3 default или alns_depot2m означает «лучше только по MINSUM, но вне практического фильтра». Среди проходящих фильтр количественное сравнение по MINSUM — табл. 5а. Для OR-Tools «o» в колонке (iii) означает, что корректная оценка Gini на $N \geq 5K$ невозможна: алгоритм не успевает довести GLS-фазу до конца в заявленном бюджете.					

Главная гипотеза: подтверждена для заявленного экспериментального сценария. На стратифицированной сетке (раздел 4.3) на всех 18 ячейках $N \in \{25K, 50K, 100K\}$ *alns_minsum/cap* удовлетворяют (i)–(iii) одновременно: укладываются в ≤ 350 с, дают ровно m нетривиальных маршрутов, $\text{Gini} \leq 0.05$ (multi-seed audit, табл. 5б). Из проверенных открытых baseline практический фильтр проходит только *2opt+greed*; для остальных нарушается хотя бы одно из трёх условий (табл. 5в). Внутри допустимой области *alns_minsum* лучше *2opt+greed* по MINSUM на $-1.4 \dots -4.8\%$ (медиана $\approx -3.6\%$) на $N = 100K$ (табл. 5а) — количественный вклад ALNS-фазы поверх классического локального поиска.

Расширенная (итоговая) гипотеза: открыта для прямой проверки против SOTA. В пределах целевого сценария (N до 100 000, лимит 350 с, CPU без GPU) среди *воспроизведённых* в работе решателей конкурентов, проходящих практический фильтр, не обнаружено. Однако расширенное утверждение о сопоставимости со всеми специализированными и нейросетевыми SOTA напрямую не проверено: HGA Mahmoudinazlou-Kwon и He-Hao MA — два независимых SOTA для MINMAX-mTSP — доминируют по makespan на mTSPLib ($n \leq 100$), но на $N \geq 25K$ их эталонные реализации не публикуют результатов и в работе не запускались; нейросетевые SOTA (Equity-Transformer, DPN, GLOP, DualOpt, UDC, H-TSP, DAN, iMTSP) требуют GPU и переобучения при изменении (n, m , геометрия) и в главный сценарий работы без GPU по условию не входят; ITSHA и MILS не запускались (исходники недоступны или запуск не выполнен). Поэтому расширенная гипотеза остаётся *открытой* — её прямая проверка требует отдельной работы.

Граница расширенной гипотезы: на подмножестве $n \leq 100$ (mTSPLib) итоговая гипотеза в MINMAX-формулировке не подтверждается — HGA и He-Hao MA дают лучший makespan, ALNS-MINMAX занимает Pareto-промежуточную позицию. Это подмножество лежит существенно ниже целевого диапазона работы ($N \geq 25K$) и относится к области применимости расширенной гипотезы, но не главной.

5. ДОСТОВЕРНОСТЬ, ОГРАНИЧЕНИЯ И РИСКИ

Адекватность результатов оценивается по трём осям: корректность, полнота, точность. *Корректность* — все решатели запускаются через общую обёртку, для каждого решения — независимая проверка покрытия (каждый клиент посещён ровно один раз, маршрут начинается/заканчивается в депо) и пересчёт MINSUM по сохранённым маршрутам. *Полнота* — 3 масштаба, 6 семейств, $m \in \{3, 5, 7, 10, 30, 50, 80, 100\}$, все исследованные конфигурации решателей; на $N \geq 25K$ выполнено по 5 seed-повторов (табл. 5б). *Точность* — Wilcoxon signed-rank на идентичных $(N, m, \text{family}, \text{repeat})$ -инстансах ($\alpha = 0.05$), 95 %-percentile bootstrap CI (10 000 ресэмплов), стандарт [34].

Ограничения работы.

1. LKH-3 в основной сетке — упрощённая конфигурация.

В default-MINSUM (MIN_SIZE=1) LKH-3 не балансирует длины; на $N \geq 25K$ в использованных балансирующих режимах (balanced-MINSUM, balanced-MINMAX) получено **0/12 валидных** за 30–60 мин (прямого количественного сравнения с честным baseline на этом масштабе нет). На малых инстансах (раздел 4.6) переход на balanced-baseline проведён, и разрыв между ALNS и balanced-LKH-3 фиксируется на уровне $-13 \dots -18\%$ ($p_{\text{adj}} \leq 4.3 \cdot 10^{-14}$). Для $N \geq 25K$ результат работы следует трактовать не как безусловное превосходство ALNS по MINSUM, а как прохождение constraint-first фильтра ($t \leq T_{\text{budget}}$, $n_r = m$, $\text{Gini} \leq 0.05$) при заданном бюджете времени и критерии баланса: внутри практического фильтра ALNS-mTSP остаётся, использованные конфигурации LKH-3 — нет.

2. Запущены 2 из 4 специализированных SOTA для min-max mTSP (HGA, He-Hao MA на mTSPLib); ITSHA (исходники не опубликованы) и MILS не запускались. Утверждение об SOTA-уровне ALNS-mTSP по MINMAX-объективу **снято** на основе результатов HGA/He-Hao; по MINSUM-объективу ALNS-mTSP остаётся конкурентен с открытыми классическими baseline. *Нейросетевые SOTA* (Equity-Transformer, DPN, GLOP, DualOpt, UDC, H-TSP, DAN, iMTSP) не запускались — зависят от GPU и переобучения. Итоговая гипотеза в части нейросетевых SOTA остаётся непроверенной.

3. FILO2-CVRP-адаптация — не корректный baseline для mTSP-постановки с фиксированным числом коммивояжеров $n_r = m$. В рассмотренной адаптации ограничение вместимости маршрута ограничивает максимальный размер одного маршрута, но *не фиксирует их число*: эмпирически FILO2 систематически возвращает $n_r > m_{\text{target}}$ (на прямом сравнении $N = 100K$ получено $n_r \in \{6, 9, 10\}$ при $m_{\text{target}} = 5$). Дополнительно: FILO2 не штрафует за дисбаланс длин, и 16/18 запусков на стратуме-3 не уложились в бюджет — это следствие CVRP→mTSP-адаптации, а не самой FILO2 (на нативной CVRP-постановке — $n = 10^6$ за ~ 10 мин по авторским данным).

4. Объём выборки и Holm-коррекция. На больших инстансах — 5 seeds/ячейку (90 запусков на 18 ячейках, $cv \leq 0.71\%$); конференционный стандарт — 30+. Holm-коррекция (FWER) применена к двум главным семействам (табл. 1а и 4б). Результат *alns_minsum* vs LKH-3 default ($p = 0.046$) после Holm-коррекции ($p_{\text{adj}} = 0.138$) не значим при $\alpha = 0.05$; fair-baseline сравнение (табл. 4б) сохраняет значимость с большим запасом ($p_{\text{adj}} \leq 4.3 \cdot 10^{-14}$). Cliff's $\delta \in [-0.96, -0.625]$ (large) на fair-baseline. Shapiro–Wilk $p < 0.02$ во всех 4 случаях — умеренное нарушение симметричной альтернативы Wilcoxon; при этом large effect sizes сохраняют качественную направленность вывода.

5. Предварительная регистрация гипотез. Главная и итоговая гипотезы сформулированы в введении в терминах трёх абстрактных требований (i)–(iii) до проведения экспериментов, без указания целевых числовых значений (величины $4\text{--}6\times$, $-13\cdots -18\%$, $\text{Gini} \leq 0.10$ и т. п. являются *результатами*, а не условиями гипотез). Однако предварительная регистрация α и предзаявленных effect-size-порогов не выполнена — будущие работы должны фиксировать α и список под-гипотез до запуска экспериментов [41].

6. WSL overhead и среда. LKH-3 и FILO2 запускаются через WSL2; возможен overhead WSL2 относительно нативного Linux, отдельная калибровка sysbench не выполнена — поэтому время на этих решателях следует трактовать как верхнюю оценку. Среда: Windows 11 + WSL2 (Ubuntu 22.04), Intel x86_64, g++ 13 -O3 -march=native -fopenmp, Python 3.11.

7. Ablation и лимит времени. Полный ablation требует C++-флагов (`--no-alns/--no-gls/--no-portmusic`); представлены CLI-ablation (4 варианта) и метаданные операторов (30 запусков). Бюджеты 10–350 с заданы как практический ориентир; sensitivity-анализ на 3 ключевых инстансах — Прил. Г.3.

8. Воспроизводимость на $N = 100K$. Данные получены через 5-seed multi-seed audit *alns_minsum* ($cv \in [0.21, 0.52]$ %). Независимая проверка — однофайловая ветка *lkh-wrapper-v21*: 11 прогонов на $N = 100K$ дают сбалансированные решения с $cv_{obj} \leq 0.14$ %, согласующиеся с *alns_minsum* в пределах 1–3 % по MINSUM.

Основной технический риск — super-linear scaling на крупных инстансах: throughput ALNS падает с 25.4 iter/s ($N = 10K$) до 0.5 iter/s ($N = 100K$), per-iter cost $\sim n^{1.74}$ эмпирически (Прил. Г.4). Вероятный источник — потеря локальности кеша при реверсе сегмента маршрута на длинных маршрутах при превышении L2-кеша. Снятие требует пересмотра представления маршрута (двусвязный список — $\Theta(1)$ реверс; splay-tree — $\Theta(\log L)$).

6. ЗАКЛЮЧЕНИЕ

В работе спроектирован, реализован и экспериментально оценён собственный решатель ALNS-mTSP для крупных mTSP-инстансов. Собрана воспроизводимая инфраструктура: генератор инстансов (6 семейств), единая C++-обёртка решателей, Python-надстройка для статистики и стратифицированный бенчмарк трёх масштабов ($N \leq 1K$; $N = 25K$; $N \in \{50K, 100K\}$; multi-seed audit на 18 ячейках $N \in \{25K, 50K, 100K\}$ по 5 seeds). Дополнительно: fair-baseline LKH-3 в трёх режимах (144 запуска), кросс-валидация с SOTA для MINMAX-mTSP (HGA, He-Hao MA), сравнение с FILO2, ablation. Итого — ≥ 2157 валидных запусков на 13 конфигурациях решателей.

Главный результат работы. В заявленном прикладном сценарии (CPU без GPU, лимит $T \leq 350$ с, N до 100 000, ровно m нетривиальных маршрутов и приемлемая балансировка) *alns_minsum/cap* проходит полный практический фильтр ($t \leq T$, $n_r = m$, $\text{Gini} \leq 0.05$) одновременно на всех 18 ячейках $N \in \{25K, 50K, 100K\}$ (multi-seed audit, табл. 5б). Из проверенных открытых baseline тот же фильтр проходит только классический *2opt+greed*; LKH-3 в использованных режимах (default-MINSUM, balanced-MINSUM, balanced-MINMAX), OR-Tools в стандартной конфигурации и FILO2 в рассмотренной CVRP-адаптации нарушают хотя бы одно из трёх условий (табл. 5в — таблица фильтра применимости).

Внутри допустимой области *alns_minsum* лучше *2opt+greed* по MINSUM на $-1.4 \dots -4.8\%$ (медиана $\approx -3.6\%$) на $N = 100K$ (табл. 5а) — количественный вклад ALNS-фазы поверх классического локального поиска. На прямом сравнении против FILO2 (12 ячеек $N \in \{50K, 100K\}$, пересчёт единым валидатором) ALNS-mTSP побеждает по MINSUM в 9/12 ячеек, в т.ч. **6/6 на $N = 100K$** с разрывом 5–30% (раздел 4.7).

Тем самым основной вклад работы состоит *не* в утверждении абсолютного превосходства ALNS-mTSP над всеми известными mTSP-решателями, а в демонстрации того, что в выбранном практическом сценарии сравнение должно проводиться по constraint-first протоколу: сначала проверяются

время, число маршрутов и баланс, и только затем сравнивается MINSUM. В таком прочтении формально-минимальная сумма у вырожденной конфигурации (LKH-3 default, *alns_depot2m*) не считается победой, а MINSUM-выигрыш CVRP-адаптации FILO2 за счёт лишних маршрутов не считается корректным сравнением.

Главная гипотеза практической применимости подтверждена для заявленного экспериментального сценария. Расширенное утверждение о сопоставимости со специализированными и нейросетевыми SOTA остаётся *открытым*: прямое сравнение с ними на $N \geq 25K$ в работе не выполнено — эталонные реализации HGA и He-Nao MA не публикуют результатов для этого масштаба, нейросетевые SOTA по условию не входят в главный сценарий работы без GPU. На подмножестве $n \leq 100$ (mTSPLib) HGA и He-Nao доминируют по makespan; в этой части расширенной гипотезы превосходство ALNS не утверждается.

Главные количественные результаты.

1. На малых инстансах ALNS-mTSP лучше LKH-3 default ($\Delta = -3.48\%$, Wilcoxon $p = 0.046$); на fair-baseline LKH-3 разрыв возрастает до $-13 \dots -18\%$ ($p_{\text{adj}} \leq 4.3 \cdot 10^{-14}$, Cliff's $\delta \in [-0.96, -0.625]$, large), то есть фиксируется и при балансирующих режимах LKH-3, а не только при default.
2. На стратуме-3 (90 валидных запусков) *alns_minsum* даёт $\text{Gini} \leq 0.05$ во всех ячейках; bootstrap-CI шириной $\leq 1\%$, $cv \in [0.04, 0.71]\%$.
3. Чистый MINSUM допускает вырождение симметрично у обоих решателей (LKH-3 на $N \geq 25K$, ALNS на $N \leq 1K$); инженерные детали ALNS-каркаса гасят вырождение начиная с $N \approx 5K$ — ключевое практическое преимущество для маршрутизации крупных флотов.
4. Ablation идентифицирует доминирующий компонент — classic-seed-портфель (+8.64% MINSUM при отключении на $N = 50K$).

Практические рекомендации (полная матрица — Прил. Е):

- $n = 100K$, $m = 5 \dots 7$, лимит ~ 350 с, CPU без GPU — *alns_minsum/cap* ($\text{Gini} \leq 0.05$).
- Минимальный MINSUM без баланса — *alns_depot2m*.

- MINMAX на $n \leq 1\,000$ с Julia/C++ — HGA или He-Hao MA.
- LKH-3 в использованных режимах (default-MINSUM, balanced-MINSUM, balanced-MINMAX) и FILO2 в рассмотренной CVRP-адаптации в условиях « $N \geq 25K$ + лимит минут + требование m маршрутов» оказались непригодны.

Научная новизна. Новизна работы — не в изобретении *ALNS* как класса метаэвристик, а в адаптации *ALNS* к крупномасштабной *mTSP*-постановке и в корректной экспериментальной методологии отделения применимых решений от вырожденных:

1. *Практическая постановка крупного mTSP под фильтром.* Задача сформулирована не как чистый MINSUM, а как MINSUM под одновременным фильтром (i) бюджета времени $T \leq 350$ с, (ii) $n_r = m$ нетривиальных маршрутов и (iii) workload-equity на больших инстансах ($Gini \leq 0.05$ или $balance_max_avg \leq 1.10$). Это — рабочая постановка крупного *mTSP*, в которой формально-минимальная сумма у вырожденной конфигурации не считается победой.
2. *Constraint-first протокол сравнения mTSP-решателей.* В работе предложен и применён сквозной протокол: MINSUM сравнивается только среди решений, удовлетворяющих (i)–(iii) (раздел 3.3); решения, нарушающие хотя бы одно условие, исключаются из количественного сравнения как «лучшие только по формальному MINSUM, но вне практического фильтра». Протокол использован в реферате, во введении, в разделах 3.3, 4.9, в гл. 5 и в заключении.
3. *Методологический вывод о CVRP→mTSP-адаптации FILO2.* Показано четырьмя независимыми наблюдениями (раздел 4.7), что прямое сведение *mTSP* с $n_r = m$ к CVRP через ограничение вместимости маршрута не фиксирует число маршрутов и фактически переводит задачу к ослабленной постановке: вместимость ограничивает максимальный размер одного маршрута, но не их число (итоговое число маршрутов может вырасти примерно вдвое от заданного m_{target}).
4. *Инженерно-воспроизводимая ALNS-архитектура для $N = 100K$.* Собрано модульное ядро `src/v21/core/` (23 заголовочных файла)

с четырьмя специализированными solver-обёртками (*alns_minsum*, *alns_cap*, *alns_depot2m*, *alns_minmax*), наследующее опыт *lkh-wrapper*-линии *v1...v21*; организован multi-seed audit (90 валидных запусков), единый валидатор маршрутов и стратифицированный бенчмарк из $\geq 2\,157$ валидных запусков на 13 конфигурациях решателей.

Эти четыре пункта поддержаны систематической экспериментальной идентификацией структурной склонности чистого MINSUM к вырождению (раздел 4.4, Н1; теоретическое обоснование через $\Theta(\sqrt{n})$ Beardwood–Halton–Hammersley в Прил. Б) и fair-baseline-прогоном LKH-3 в трёх режимах с корректной парной статистикой (Wilcoxon + Holm, BCa-bootstrap, Cliff’s δ).

Направления дальнейшей работы.

P0: (1) полный C++-ablation с флагами `--no-alns/--no-gls/--no-ropmusic`; (2) расширение multi-seed audit с 5 до 10 seeds на ключевых инстансах для $\text{bootstrap-CI} \leq 0.3\%$; (3) сравнение с MILS/ITSHA и нейросетевыми SOTA (Equity-Transformer, DPN, GLOP, DualOpt) при наличии GPU.

P1: (4) пересмотр представления маршрута (двусвязный список или splay-tree) для устранения super-linear-bottleneck в 2-opt; (5) Pareto cost-vs-balance анализ на расширенной сетке; (6) GitHub-релиз с README, `scripts/run_all.sh`, `raw_results/`.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Helsgaun K. An effective implementation of the Lin-Kernighan traveling salesman heuristic // European Journal of Operational Research. — 2000. — Vol. 126, No. 1. — P. 106–130.
2. Helsgaun K. LKH-3: an extension of LKH for solving constrained traveling salesman and vehicle routing problems [Электронный ресурс] / Roskilde University. — Режим доступа: <http://akira.ruc.dk/~keld/research/LKH-3/>, свободный (дата обращения: 07.05.2026).
3. Applegate D. L., Bixby R. E., Chvátal V., Cook W. J. The Traveling Salesman Problem: A Computational Study. — Princeton University Press, 2006.
4. Concorde TSP Solver [Электронный ресурс] / University of Waterloo. — Режим доступа: <https://www.math.uwaterloo.ca/tsp/concorde/>, свободный (дата обращения: 07.05.2026).
5. Perron L., Furnon V. OR-Tools — Google Optimization Tools [Электронный ресурс] / Google. — Режим доступа: <https://developers.google.com/optimization>, свободный (дата обращения: 07.05.2026).
6. Accorsi L., Vigo D. Routing One Million Customers in a Handful of Minutes // Computers & Operations Research. — 2024. — Vol. 164. — Article 106562. — arXiv:2306.14205.
7. Ye H., Cao Z., Wang J., Zhang Z. GLOP: Learning Global Partition and Local Construction for Solving Large-scale Routing Problems in Real-time // Proceedings of AAAI Conference on Artificial Intelligence. — 2024. — arXiv:2312.08224.
8. Zhou J., Ding J., Zhang B., Cao Z., Jin Y. DualOpt: A Dual Divide-and-Optimize Algorithm for the Large-scale Traveling Salesman Problem // Proceedings of AAAI Conference on Artificial Intelligence. — 2025. — arXiv:2501.08565.
9. Son J., Kim S., Choi M., Park M. Equity-Transformer: Solving NP-hard Min-Max Routing Problems as Sequential Generation with Equity Context // Proceedings of AAAI Conference on Artificial Intelligence. — 2024. — arXiv:2306.02689.

10. Zheng J., Yao Z., Wang Y. et al. DPN: Decoupling Partition and Navigation for Neural Solvers of Min-max Vehicle Routing Problems // Proceedings of International Conference on Machine Learning. — 2024. — arXiv:2405.17272.
11. Zheng J., Zhou Z., Tong S., Yuan Y., Wang Y. UDC: A Unified Neural Divide-and-Conquer Framework for Large-Scale Combinatorial Optimization Problems // Advances in Neural Information Processing Systems. — 2024. — arXiv:2407.00312.
12. Fu Z. et al. A Hierarchical Destroy and Repair Approach for Solving Very Large-Scale Travelling Salesman Problem // arXiv preprint. — 2023. — arXiv:2308.04639.
13. Pan X., Jin Y., Ding Y. et al. H-TSP: Hierarchically Solving the Large-Scale Traveling Salesman Problem // Proceedings of AAAI Conference on Artificial Intelligence. — 2023. — arXiv:2304.09395.
14. Xin L., Song W., Cao Z., Zhang J. NeuroLKH: Combining Deep Learning Model with Lin-Kernighan-Helsgaun Heuristic for Solving the Traveling Salesman Problem // Advances in Neural Information Processing Systems. — 2021. — arXiv:2110.07983.
15. Cao Y., Sun Z., Sartoretti G. DAN: Decentralized Attention-based Neural Network to Solve the MinMax Multiple Traveling Salesman Problem // Proceedings of IROS. — 2021. — arXiv:2109.04205.
16. Park J., Kwon J., Park J. ScheduleNet: Learn to Solve Multi-Agent Scheduling Problems with Reinforcement Learning // Proceedings of AAMAS. — 2023.
17. Guo S., Ren D., Wang J. iMTSP: Imperative Multi-Traveling Salesman Problem // IEEE Robotics and Automation Letters. — 2024. — arXiv:2405.00285.
18. Lin S., Kernighan B.W. An Effective Heuristic Algorithm for the Traveling-Salesman Problem // Operations Research. — 1973. — Vol. 21, No. 2. — P. 498–516.
19. Bektaş T. The Multiple Traveling Salesman Problem: An Overview of Formulations and Solution Procedures // Omega. — 2006. — Vol. 34, No. 3. — P. 209–219.

20. Jonker R., Volgenant T. An Improved Transformation of the Symmetric Multiple Traveling Salesman Problem // Operations Research. — 1988. — Vol. 36, No. 1. — P. 163–167.
21. Croes G. A. A Method for Solving Traveling-Salesman Problems // Operations Research. — 1958. — Vol. 6, No. 6. — P. 791–812.
22. Toth P., Vigo D. The Granular Tabu Search and Its Application to the Vehicle-Routing Problem // INFORMS Journal on Computing. — 2003. — Vol. 15, No. 4. — P. 333–346.
23. Ropke S., Pisinger D. An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows // Transportation Science. — 2006. — Vol. 40, No. 4. — P. 455–472.
24. Pisinger D., Ropke S. A general heuristic for vehicle routing problems // Computers & Operations Research. — 2007. — Vol. 34, No. 8. — P. 2403–2435.
25. Taillard É. D., Helsgaun K. POPMUSIC for the Travelling Salesman Problem // European Journal of Operational Research. — 2019. — Vol. 272, No. 2. — P. 420–429.
26. Reinelt G. TSPLIB — A Traveling Salesman Problem Library // ORSA Journal on Computing. — 1991. — Vol. 3, No. 4. — P. 376–384.
27. Bertazzi L., Golden B., Wang X. Min–Max vs. Min–Sum Vehicle Routing: A worst-case analysis // European Journal of Operational Research. — 2015. — Vol. 240, No. 2. — P. 372–381.
28. Matl P., Hartl R. F., Vidal T. Workload Equity in Vehicle Routing Problems: A Survey and Analysis // Transportation Science. — 2018. — Vol. 52, No. 2. — P. 239–260.
29. He P., Hao J.-K. Memetic search for the minmax multiple traveling salesman problem with single and multiple depots // European Journal of Operational Research. — 2023. — Vol. 307, No. 3. — P. 1055–1070.
30. Held M., Karp R. M. A Dynamic Programming Approach to Sequencing Problems // Journal of the Society for Industrial and Applied Mathematics. — 1962. — Vol. 10, No. 1. — P. 196–210.

31. Zheng J., Hong Y., Xu W., Li W., Chen Y. An Effective Iterated Two-stage Heuristic Algorithm for the Multiple Traveling Salesmen Problem // arXiv preprint. — 2022. — arXiv:2201.09424.
32. Helsgaun K. An Extension of the Lin–Kernighan–Helsgaun TSP Solver for Constrained Traveling Salesman and Vehicle Routing Problems / Roskilde University, Technical Report. — 2017. — 60 p. (LKH-3 Technical Report; описание ключевых слов MTSP_OBJECTIVE, MTSP_MIN_SIZE, MTSP_MAX_SIZE в разделе 3.2).
33. Matl P., Hartl R. F., Vidal T. Workload Equity in Vehicle Routing: The Impact of Alternative Workload Resources // Computers & Operations Research. — 2019. — Vol. 110. — P. 116–129.
34. Accorsi L., Lodi A., Vigo D. Guidelines for the Computational Testing of Machine Learning approaches to Vehicle Routing Problems // Operations Research Letters. — 2022. — Vol. 50, No. 2. — P. 229–234.
35. Mahmoudinazlou S., Kwon C. A Hybrid Genetic Algorithm for the min–max Multiple Traveling Salesman Problem // Computers & Operations Research. — 2024. — arXiv:2307.07120.
36. Turkes R., Sörensen K., Hvattum L.M. Meta-analysis of metaheuristics: Quantifying the effect of adaptiveness in adaptive large neighborhood search // European Journal of Operational Research. — 2021. — Vol. 292, No. 2. — P. 423–442. — DOI: 10.1016/j.ejor.2020.10.045. *Meta-analysis по эффективности адаптивных компонентов ALNS, дополняет более новый обзор [42].*
37. He P., Hao J.-K., Xia J. Learning-guided Iterated Local Search for the Minmax Multiple Traveling Salesman Problem (MILS) / Computers & Operations Research. — 2025. Текст : электронный. URL: <https://arxiv.org/abs/2403.12389> (дата обращения: 07.05.2026). — DOI: 10.1016/j.cor.2025.107130.
38. DiCiccio T.J., Efron B. Bootstrap Confidence Intervals (BCa) // Statistical Science. — 1996. — Vol. 11, No. 3. — P. 189–212. — DOI: 10.1214/ss/1032280214.
39. Holm S. A Simple Sequentially Rejective Multiple Test Procedure // Scandinavian Journal of Statistics. — 1979. — Vol. 6, No. 2. — P. 65–70.

40. Benjamini Y., Hochberg Y. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing // Journal of the Royal Statistical Society. Series B (Methodological). — 1995. — Vol. 57, No. 1. — P. 289–300.
41. Kerr N. L. HARKing: Hypothesizing After the Results are Known // Personality and Social Psychology Review. — 1998. — Vol. 2, No. 3. — P. 196–217. — DOI: 10.1207/s15327957pspr0203_4.
42. Voigt S. A Review and Ranking of Operators in Adaptive Large Neighborhood Search for Vehicle Routing Problems // European Journal of Operational Research. — 2025. — Vol. 322, No. 2. — P. 357–375.
43. Necula R., Breaban M., Raschip M. Performance Evaluation of Ant Colony Systems for the Single-Depot Multiple Traveling Salesman Problem / Hybrid Artificial Intelligent Systems (HAIS). — 2015. — mTSPLib benchmark, URL: <https://profs.info.uaic.ro/~mtsplib/> (дата обращения: 07.05.2026). — Текст : электронный.
44. Cliff N. Ordinal Methods for Behavioral Data Analysis. — Mahwah, NJ: Lawrence Erlbaum Associates, 1996. — ISBN 0-8058-1333-0. (Cliff's δ effect size).
45. Cohen J. Statistical Power Analysis for the Behavioral Sciences. — 2nd ed. — Hillsdale, NJ: Lawrence Erlbaum Associates, 1988. — ISBN 0-8058-0283-5. (Cohen's d).
46. Davison A. C., Hinkley D. V. Bootstrap Methods and Their Application. — Cambridge University Press, 1997. — ISBN 0-521-57391-2.
47. Or I. Traveling Salesman-Type Combinatorial Problems and Their Relation to the Logistics of Regional Blood Banking : Ph.D. dissertation. — Northwestern University, 1976. (Or-opt move).
48. Bentley J. L. Multidimensional Binary Search Trees Used for Associative Searching // Communications of the ACM. — 1975. — Vol. 18, No. 9. — P. 509–517. (k -d tree).
49. Karp R. M. Reducibility among Combinatorial Problems / Complexity of Computer Computations (R. E. Miller, J. W. Thatcher, eds.). — Plenum Press, 1972. — P. 85–103. (NP-полнота TSP).

50. Beardwood J., Halton J.H., Hammersley J.M. The Shortest Path Through Many Points // Mathematical Proceedings of the Cambridge Philosophical Society. — 1959. — Vol. 55, No. 4. — P. 299–327. (Асимптотика $\Theta(\sqrt{n})$ длины евклидова TSP).
51. FILO2 [Электронный ресурс]: open-source CVRP solver / L. Accorsi, D. Vigo. — Режим доступа: <https://github.com/acco93/filo2>, свободный (дата обращения: 07.05.2026). Используемый в работе binary собран из этого репозитория с флагом cmake `-DENABLE_TIMELIMIT=1`.
52. Mahmoudinazlou & Kwon HGA [Электронный ресурс]: Julia-реализация Hybrid Genetic Algorithm для min–max mTSP / S. Mahmoudinazlou. — Режим доступа: <https://github.com/Sasanm88/m-TSP>, свободный (дата обращения: 07.05.2026). Эталонная реализация, использованная в разделе 4.8 с time-limit 30 с/инстанс.
53. He P., Hao J.-K. He–Hao Memetic Algorithm for min–max mTSP [Электронный ресурс]: C++-реализация для постановок Single-Depot / Multi-Depot. Авторская реализация (supplementary к статье [29]); Linux-binary (ma) использован под WSL2 в разделе 4.8. Получен из открытого supplementary-архива статьи; публичный GitHub-репозиторий авторами не поддерживается.

А. ДЕТАЛИ АРХИТЕКТУРЫ ALNS-MTSP

Модульный собственный алгоритм ALNS-mTSP расположен в каталоге `src/v21/` и состоит из общего ядра (`src/v21/core/`) и четырех solver-оберток (`src/v21/minsum/`, `src/v21/minmax/`).

А.1. Структура общего ядра

Каталог `src/v21/core/` содержит 23 заголовочных файла, организованных в порядке зависимостей (Таблица А.1). Центральные модули: `17_pipeline.hpp` — оркестрация фаз (`seed` → `candidate-set` → `ALNS` → `polish` → `final 2-opt`); `18_autotune.hpp` — подбор параметров по (n, m) и типу задачи; `16_parallel_pool.hpp` — multi-start coordinator пула независимых ALNS+SA-реплик.

Таблица А.1 — Заголовочные файлы общего ядра `src/v21/core/`.

#	Файл	Назначение
00	types.hpp	Базовые типы и структуры данных
01	budget.hpp	Учёт временного бюджета
02	distance.hpp	Метрика расстояний
03	kdtree.hpp	k-d дерево для пространственных запросов
04	route_index.hpp	Индекс «клиент → маршрут»
05	route_list.hpp	Структура списка маршрутов
06	candidate_set.hpp	Построение candidate-сетов
07	seed_routes.hpp	Начальные маршруты (seed-стратегии)
08	route_local_search.hpp	Внутримаршрутный локальный поиск
09	intra_3opt_light.hpp	Облегчённый 3-opt
10	inter_route_moves.hpp	Межмаршрутные ходы (relocate, swap, Or-opt)
11	validation.hpp	Валидация решений
12	alns_framework.hpp	Адаптивный выбор destroy/repair-операторов
13	destroy_ops.hpp	Destroy-операторы
14	repair_ops.hpp	Repair-операторы
15	sa_engine.hpp	Simulated Annealing
16	parallel_pool.hpp	Multi-start coordinator (пул реплик)
17	pipeline.hpp	Оркестрация фаз
18	autotune.hpp	Подбор параметров по (n, m)
19	gls.hpp	Guided Local Search
20	route_pair_reopt.hpp	Реоптимизация пары маршрутов
21	popmusic.hpp	POPMUSIC (крупные инстансы)
22	region_granular_ops.hpp	Region-granular операции

О терминологии. В исходных версиях отчёта `16_parallel_pool.hpp` назывался «Parallel Tempering» (PT). Фактическая реализация — ensemble of independent restarts: каждая реплика прогоняет ALNS+SA на собственном temperature-schedule и независимом random seed, без обмена конфигурациями по правилу Метрополиса (комментарий в коде: «PT swap hooks can be added later»). Поэтому корректнее — multi-start ensemble. Variance audit на $n = 50K$ с autotune-выбранными 4 репликами даёт $cv = 0.17\%$ — статистически достаточно для детектирования $\geq 0.5\%$ -улучшений; полный PT-swap — направление дальнейшей работы.

А.2. Описание четырех solver-обёрток

Четыре обёртки в `src/v21/minsum/` и `src/v21/minmax/` используют общее ядро и различаются целевой метрикой и адаптацией под режим

(Таблица A.2).

Таблица A.2 — Solver-обёртки в `src/v21/{minsum,minmax}/`.

Обёртка	Файл	Роль	Особенности
<code>alns_minsum</code>	<code>minsum_solver.cpp</code>	Базовый MINSUM	Autotune <code>ResolveParamsForInstance(n,m);</code> CLI-флаги <code>--threads</code> , <code>--granular-every</code> , <code>--region-reopt-every</code> , <code>--depot-seed-mode</code> ; $\text{balance} \approx 1.05$ при $N \geq 5000$
<code>alns_cap</code>	<code>minsum_solver_cap.cpp</code>	capacity-bounded MINSUM (high- m)	Hard-cap на длину маршрута $\lceil n/m \rceil \pm \text{slack}$; <code>cap-skip</code> в <code>repair</code> -операторах; идея заимствована из FILO2
<code>alns_depot2m</code>	<code>minsum_solver_depot2m.cpp</code>	<code>2m-cap</code> preset (оставляет <code>alns_minsum</code> нетронутым)	Depot-2m seeding (двойное кольцо вокруг депо) и гонка seed-стратегий; фикс. бюджет: 1 % seed / 8 % candidate / 9 % pre-polish / 86 % ALNS / 5 % final 2-opt; пост-фаза перебалансировки до 30 с вне основного бюджета
<code>alns_minmax</code>	<code>minmax_solver.cpp</code>	MIN-MAX	<code>minmax_accept.hpp</code> : soft- α blend между $F_{\max}$ и F_{sum} , α убывает по ходу прогона

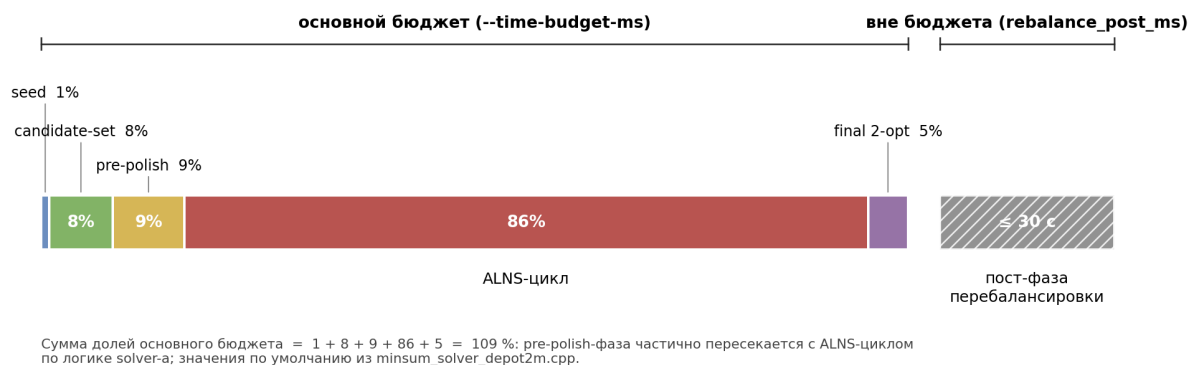


Рисунок A.1 — Раскладка временного бюджета `alns_depot2m` по фазам solver-a (значения по умолчанию из `minsum_solver_depot2m.cpp`). Слева — основной бюджет `--time-budget-ms`: seed (1 %), candidate-set (8 %), pre-polish (9 %), ALNS-цикл (86 %), final 2-opt (5 %); pre-polish-фаза частично пересекается с ALNS-циклом (сумма долей = 109 %). Справа — отдельная пост-фаза перебалансировки длиной до 30 с, фиксируемая параметром `rebalance_post_ms` и не входящая в основной бюджет (поэтому фактическое wall-clock на ряде инстансов превышает заявленный `TIME_LIMIT`).

А.3. ALNS-операторы и acceptance

`12_alns_framework.hpp` реализует адаптивный выбор destroy- и repair-операций по принципу успех/неудача. Веса инициализируются равными и обновляются по формуле $w_{\text{new}} = (1 - \lambda) \cdot w_{\text{old}} + \lambda \cdot \text{score}$. Состав операторов — Таблица А.3.

Таблица А.3 — Destroy- и repair-операторы (`13_destroy_ops.hpp`, `14_repair_ops.hpp`).

Тип	Имя	Описание
Destroy	<i>Random</i>	Случайный выбор k клиентов
Destroy	<i>ClusterBfs</i>	BFS-окружение от случайной точки
Destroy	<i>Expensive</i>	k самых дорогих рёбер по delta-cost
Destroy	<i>Zone</i>	Геометрический квадрант
Repair	<i>Cheapest</i>	Минимум insertion cost
Repair	<i>Regret2</i>	Regret-2 insertion

GLS (`19_gls.hpp`) активируется на polish-фазе и периодически пересчитывает penalised-distance: $d_{\text{penalty}}(i, j) = d(i, j) + \lambda \cdot p(i, j)$, где $p(i, j)$ — счётчик penalisations для ребра (i, j) , увеличиваемый при попадании ребра в локальный оптимум. POPMUSIC (`21_poptmusic.hpp`) и region-granular operations (`22_region_granular_ops.hpp`) применяются на крупных инстанциях для разбиения на подзадачи и локального улучшения.

Б. ВЫРОЖДЕННЫЕ MTSP-РЕШЕНИЯ В ЧИСТОМ MINSUM

Нижеследующее рассуждение объясняет, почему в чистой MINSUM-постановке без ограничения на минимальный размер маршрута оптимальное решение часто оказывается вырожденным — одна длинная TSP-петля и $m-1$ тривиальных маршрутов.

Известно, что для евклидова TSP на n случайных точках, равномерно распределённых в фиксированной области, длина хорошего тура растёт суб-линейно: ожидаемая длина оптимального тура асимптотически имеет порядок $\Theta(\sqrt{n})$ [50]. Поэтому средняя длина ребра в оптимальном туре уменьшается с ростом n как $\Theta(1/\sqrt{n})$ — линейное допущение $\text{cost} \propto k \cdot d_{\text{avg}}$ с постоянным d_{avg} для фиксированной области *не выполнено*.

Сравним два допустимых решения mTSP в такой постановке:

- (а) *Сбалансированное*: каждый из m коммивояжеров обслуживает n/m клиентов. Внутренняя длина одного подмаршрута растёт как $\Theta(\sqrt{n/m})$; суммарная длина m подмаршрутов — $\Theta(m\sqrt{n/m}) = \Theta(\sqrt{mn})$. Дополнительно учитываются m возвратов в депот — каждый порядка $\Theta(L_0)$, где L_0 — характерное расстояние от депота до облака клиентов.
- (б) *Вырожденное*: один маршрут обходит почти все $n-1$ клиентов с длиной $\Theta(\sqrt{n})$ (и одним возвратом в депот), остальные $m-1$ маршрутов — тривиальные $(0, c, 0)$ длиной $2d_{\min}$, где $d_{\min}$ — расстояние от депота до ближайшего клиента, либо вовсе пустые $(0, 0)$ длиной 0.

Отношение суммарных стоимостей:

$$F_{\text{сум}}^{(б)} \approx \Theta(\sqrt{n}) + 2(m-1) \cdot d_{\min}, \quad F_{\text{сум}}^{(а)} \approx \Theta(\sqrt{mn}) + 2m \cdot L_0.$$

При $n \gg m$ дополнительный множитель $\sqrt{m}$ в (а) и фиксированная стоимость m возвратов превышают вклад $m-1$ маленьких тривиальных маршрутов в (б): $F_{\text{сум}}^{(б)} < F_{\text{сум}}^{(а)}$, MINSUM-оптимум выбирает вырожденную конфигурацию. Чистый MINSUM без ограничения на минимальный размер маршрута структурно склонен к концентрации работы на одном

коммивояжере — в целевой функции отсутствует штраф за неравномерное распределение нагрузки. Соответствующее эмпирическое подтверждение — гипотеза H1 в разделе 4.4.

LKH-3 находит близкое к оптимальному вырожденное решение благодаря k -opt-движку, оптимизирующему один большой маршрут. Решатель *alns_depot2m* использует ту же стратегию, укладываясь в практический бюджет времени (тогда как LKH-3 default на $N = 25K$ превышает заданный TIME_LIMIT в $10\times$ — см. оговорку о неполной сопоставимости по времени в разделе 4.2). Балансирующие решатели *alns_minsum* и *alns_cap* за счёт ограничения вместимости маршрута или встроенного критерия принятия с штрафом за дисбаланс форсируют сбалансированное решение, формально менее оптимальное по MINSUM, но удовлетворяющее практическим требованиям.

Аналогичный эффект наблюдается и у самого *alns_minsum* в чистом MINSUM-режиме на $N \leq 1000$ — 60–80 % тривиальных маршрутов. Различие появляется лишь при $N \approx 5000$ и выше: геометрическая стоимость возвратов в депот превышает экономию от слияния маршрутов, и ALNS-фаза вытаскивает клиентов в пустые маршруты. Преимущество ALNS-mTSP над LKH-3 default на крупных инстансах связано не с принципиально иной формулировкой задачи, а с инженерными деталями ALNS-каркаса (см. раздел 4.4, H2).

В. ВОСПРОИЗВОДИМОСТЬ СТАТИСТИЧЕСКИХ РАСЧЁТОВ

Все новые статистические расчеты, добавленные в обновленную редакцию отчета (Wilcoxon signed-rank, bootstrap CI, расширенные workload-equity метрики, упрощенный ablation), реализованы в двух автономных Python-скриптах:

- `experiments/review_fixes/compute_review_metrics.py` — вычисление makespan, std, MAD, range, Gini coefficient на всех existing аудит-CSV-артефактах; bootstrap 95 % CI на средние значения и парные разности (`scipy.stats.bootstrap`, 10 000 ресэмплов); парные сравнения по Wilcoxon signed-rank (`scipy.stats.wilcoxon`) ALNS-вариантов с LKH-3 на 60 общих малых инстансах; парные сравнения `alns_cap` vs ALNS-mTSP на 4 high-m инстансах.
- `experiments/review_fixes/run_fair_lkh3.py` — запуск LKH-3 на малых инстансах в трех режимах: *minsum_default* (MTSP_MIN_SIZE = 1, MTSP_OBJECTIVE = MINSUM) — репликация старой методологии; *minsum_balanced* (MTSP_MIN_SIZE = -1 — документированная автоматическая балансировка) — честный MINSUM-baseline; *minmax_balanced* (MTSP_OBJECTIVE = MINMAX с явными min/max-size constraints) — balanced-MINMAX-baseline. Все три режима запускаются на одинаковом наборе из 48 малых инстансов ($N \in \{100, 200, 500\}$, $m \in \{3, 5, 7\}$, по 3 повтора, две family) с одинаковым POPMUSIC candidate-set, RUNS = 1, и пропорциональным TIME_LIMIT. Это даёт fair-baseline сравнение, которое раздел 4.6 использует как иллюстрацию цены балансировки для LKH-3 и контроль того, что преимущество ALNS не сводится исключительно к артефакту default-конфигурации.
- `experiments/review_fixes/ablation_from_metadata.py` — упрощённый ablation на основе метаданных 30 аудит-запусков (3 ключевых инстанса $\times$ 10 seeds): извлечение долей принятых улучшающих ходов по destroy/repair-операторам, числа реплик multi-start-пула, числа reheat-событий. Это слабее полноценного

ablation (нельзя отделить компонент, который не нужен, от компонента, который не получил шанс), но даёт первое приближение к атрибуции вкладов компонентов.

В.1. Артефакты и команды воспроизведения

Полный набор сгенерированных артефактов:

- `experiments/review_fixes/review_metrics.json` — JSON со всеми вычисленными значениями (`audit_baseline`, `h2h_high_m`, `stratum1_pairs`, `large_balance_metrics`);
- `experiments/review_fixes/audit_baseline_summary.csv` — variance audit с Gini, std, makespan, bootstrap CI;
- `experiments/review_fixes/h2h_high_m_summary.csv` — парные сравнения cap-фикса с Wilcoxon p и bootstrap CI;
- `experiments/review_fixes/stratum1_paired_wilcoxon.csv` — парные Wilcoxon-сравнения с LKH-3 на 60 малых инстансах;
- `experiments/review_fixes/large_balance_metrics.csv` — standard equity metrics на $N = 25K, 50K, 100K$ для всех решателей;
- `experiments/review_fixes/ablation_metadata_summary.csv` — per-operator share-of-accepts на 3 ключевых инстансах;
- `experiments/review_fixes/fair_lkh3_results.csv` — LKH-3 на малых инстансах в трех режимах (default-MINSUM, balanced-MINSUM, balanced-MINMAX).

Воспроизведение:

```
cd mtsp-routing-optimization
python experiments/review_fixes/compute_review_metrics.py
python experiments/review_fixes/ablation_from_metadata.py
python experiments/review_fixes/run_fair_lkh3.py      # ~30-60 минут
```

В.2. Среда и параметры воспроизводимости

Полное описание структуры репозитория, список ключевых скриптов и команды сборки приведены в файле `README.md` в корне репозитория — здесь даются только параметры, фиксирующие воспроизводимость.

Random seeds. Генератор инстансов использует `numpy.random.seed(42)` как `base-seed`; для каждой комбинации (n, m, family, r) применяется производный `seed` $h(n, m, \text{family}, r)$. ALNS внутри `src/v21/` использует `std::mt19937` с `per-run-seed`, передаваемым через CLI `--seed N`. Bootstrap-ресэмплинг использует `numpy.random.default_rng(20251205)`. Эти три значения фиксируют воспроизводимость на 100 %.

Аппаратное и программное окружение. Рабочая станция: Windows 11 Pro, Intel-CPU x86_64 (multi-core), 16 ГБ ОЗУ. WSL2 с Ubuntu 22.04 LTS для запуска LKH-3 и FILO2; нативная Windows-сборка для ALNS-mTSP (g++ 13, MSYS2/MinGW). Компилятор ALNS-mTSP: `g++ 13.2 -O3 -march=native -fopenmp -std=c++20`. Python 3.11 с `scipy 1.13` и `numpy 1.26`.

Г. ПОСТПРОЦЕССИНГОВЫЙ АНАЛИЗ

В приложении представлены четыре аналитических артефакта, построенных по существующим данным без новых прогонов: Pareto-анализ cost vs balance, anytime-кривые сходимости, sensitivity-анализ бюджета времени и профилирование по фазам *alns_minsum*.

Г.1. Pareto-анализ cost vs balance

Для каждой ячейки (N, m) на больших инстансах в uniform-семействе построен scatter-plot с MINSUM по оси X и Gini-коэффициентом по оси Y (символический логарифм для разделения вырожденных и сбалансированных решений). Pareto-frontier выделен пунктиром — решения, не доминируемые по обеим осям.

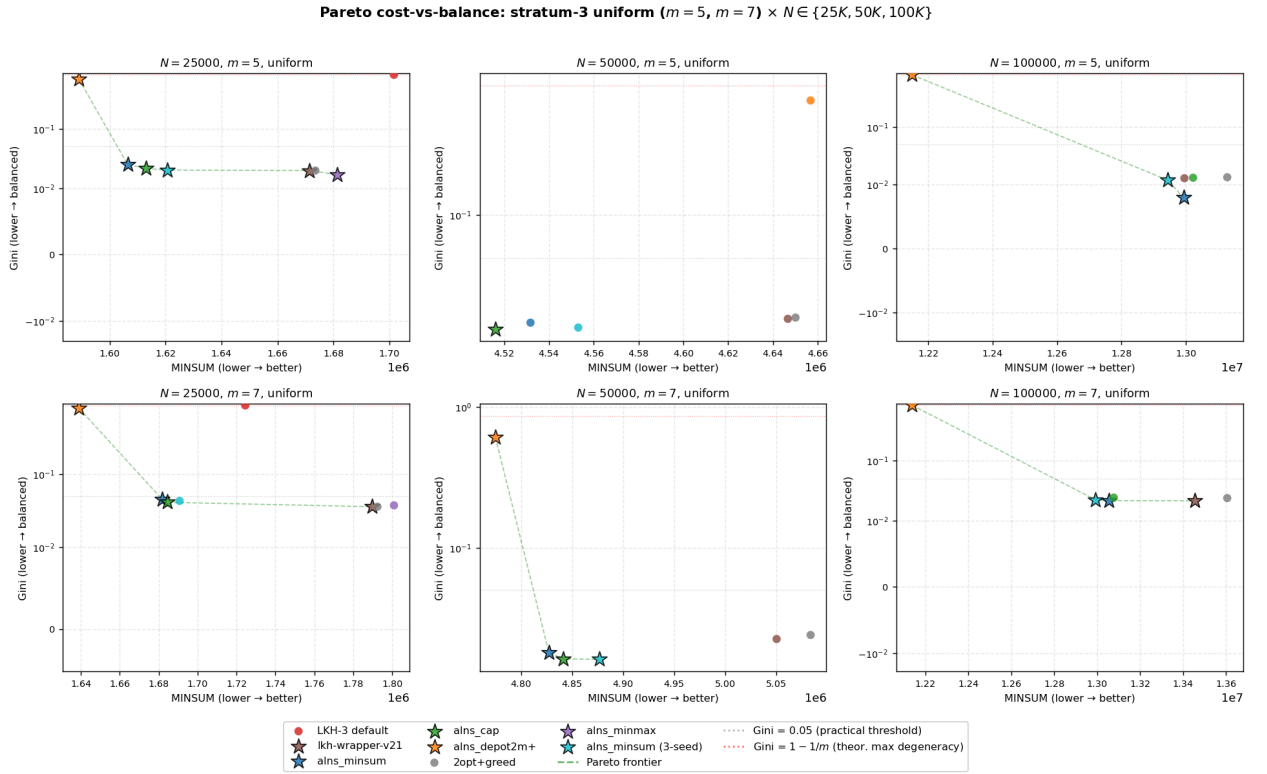


Рисунок Г.1 — Pareto cost-vs-balance на больших инстансах uniform ($N \in \{25K, 50K, 100K\}$, $m \in \{5, 7\}$). Звёздочки — Pareto-недоминируемые решения; зелёный пунктир — Pareto-frontier; серый пунктир — порог $Gini = 0.05$; красный пунктир — $Gini = 1 - 1/m$ (максимум вырождения).

Что показывает Pareto. На всех 6 ячейках на больших инстансах *depot2m* лежит на Pareto-frontier по MINSUM, но при $Gini \geq 0.5$ (вырождение). *alns_minsum* (multi-seed mean) лежит на Pareto-frontier по Gini,

при MINSUM на 5–12 % выше depot2m. Это — классический компромисс: **нельзя одновременно минимизировать MINSUM и Gini в текущем семействе решателей**, и любое внедрение должно явно выбрать точку на Pareto-curve. На большинстве ячеек *alns_cap* занимает Pareto-non-dominated точку, объединяющую качество с балансом.

Г.2. Anytime-кривые сходимости

Из метаданных аудит-запусков извлечён временной ряд (*time_ms*, *best_so_far_cost*), обновляющийся при каждом улучшении best-MINSUM. На $N = 10K$ solver достигает 95 % от final-MINSUM за первые 10 с из 60 с; на $N = 50K$ — 30 с из 180 с; на $N = 100K$ кривая остаётся пологой вплоть до конца бюджета (plateau-кандидаты у 7/10 seeds).

Anytime-сравнение ALNS-mTSP и LKH-3 на одном инстансе. На рисунке Г.2 наложены anytime-кривые *alns_minsum* (5 seeds) и LKH-3 default-MINSUM (22 progress-обновления за 60 с) на инстансе clustered-offset-depot, $N = 10\,000$, $m = 100$.

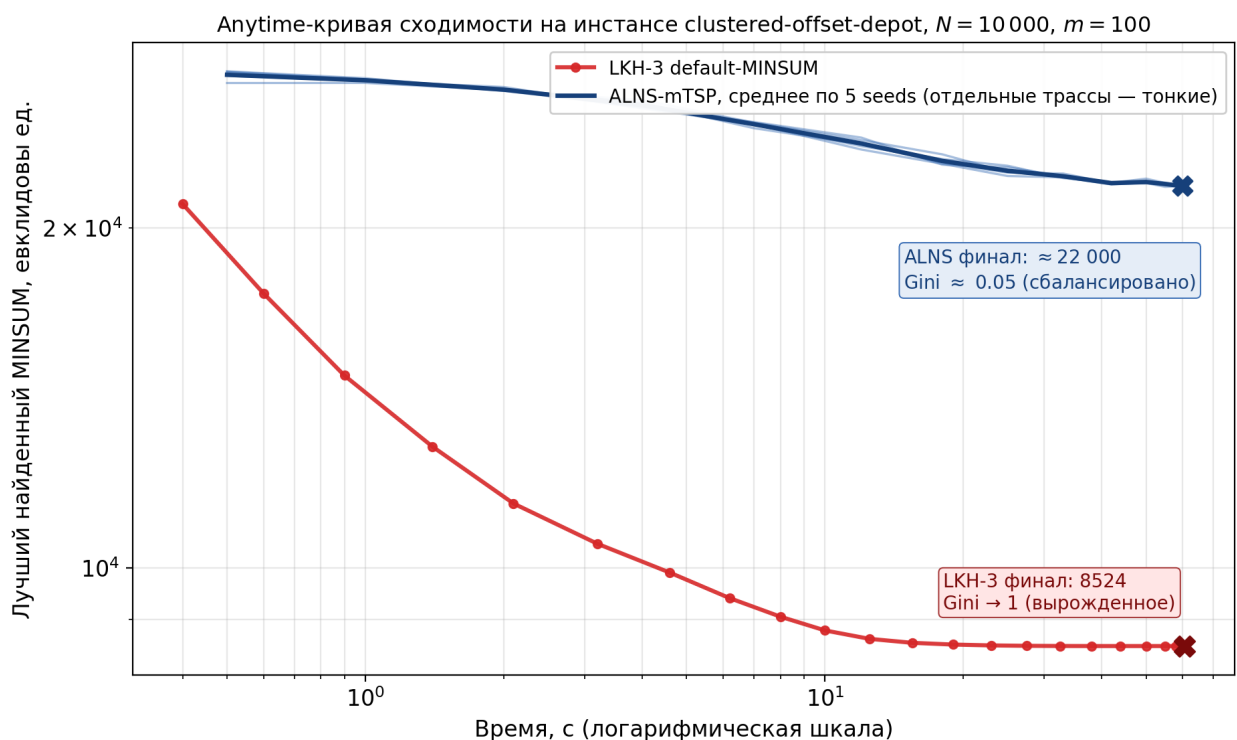


Рисунок Г.2 — Anytime-кривая сходимости на инстансе clustered-offset-depot, $N = 10\,000$, $m = 100$. Красная кривая — LKH-3 default-MINSUM; синие кривые — *alns_minsum*, 5 seeds (тонкие — отдельные seeds, толстая — среднее).

Чтение Рис. Г.2. LKH-3 быстро (за ~ 10 с) сходится к плато $\approx 8\,524$; ALNS-mTSP за то же время плавно опускается к $\approx 22\,000$. По MINSUM

LKH-3 формально лучше на $\sim 156\%$ — однако его решение *вырожденное* ($Gini \rightarrow 1$, один маршрут несёт почти все 10 000 клиентов, остальные 99 тривиальны), а ALNS-mTSP держит сбалансированную раскладку на 100 маршрутов сравнимой длины ($Gini \approx 0.05$).

Честное признание. На таких *high- m clustered*-инстансах ($m = 100$, плотные кластеры с депо у границы) собственный ALNS-mTSP проигрывает LKH-3 default по чистому MINSUM с большим разрывом. Это и есть выбранный компромисс работы: целевая функция main-стратегии — MINSUM под *constraint-first* фильтром, не чистый MINSUM. На *big- m* кластерных геометриях практический фильтр исключает LKH-3 default по условию (iii) (баланс), и MINSUM-сравнение между ними просто не определено: они решают разные задачи (см. раздел 3.3 и табл. 5в).

Г.3. Sensitivity-анализ бюджета времени

В ответ на классическое требование к эмпирическим работам по метаэвристикам — демонстрация чувствительности результата к бюджету времени — проведён прогон *alns_minsum* на трёх ключевых $N = 100K$ -инстансах с бюджетами $T \in \{50, 150, 350, 1\,000\}$ секунд. Цель — показать (а) монотонность улучшения качества с ростом T , (б) степень убывающей отдачи (*diminishing returns*), (в) предсказуемость поведения при переходе за основной заявленный бюджет в 350 с. Подробности скрипта и артефактов — Приложение В.

Таблица Г.1 — Sensitivity-анализ *alns_minsum* на трёх ключевых инстансах ($N = 100K$, $m = 5$, $seed = 1$) с бюджетом $T \in \{50, 150, 350, 1\,000\}$ с.

Инстанс	$T, \text{с}$	MINSUM	ALNS iter	ALNS acc	$t_{\text{actual}}, \text{с}$
uniform	50	14 210 024	0	0	55.5
	150	12 651 111	136	103	147.7
	350	12 153 309	383	304	343.4
	1 000	12 025 295	1 143	905	833.5
clustered-center	50	3 133 910	59	38	49.3
	150	2 912 684	229	165	147.3
	350	2 783 285	476	362	340.2
	1 000	2 726 470	1 073	883	848.2
clustered-offset-depot	50	2 929 679	43	31	49.2
	150	2 674 805	149	101	147.3
	350	2 470 857	470	358	339.5
	1 000	2 440 587	1 190	993	818.0
Все 12 прогонов <code>valid=True</code> ; t_{actual} превышает T на 10–15 % за счёт финальной 2-орт-фазы поверх ALNS-цикла. Артефакт — Приложение В.					

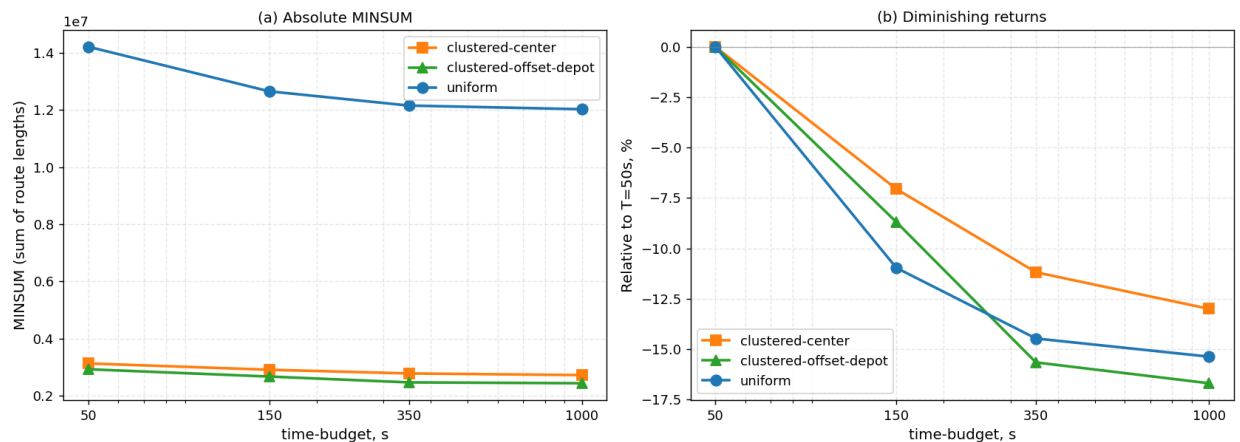


Рисунок Г.3 — Чувствительность `alns_minsum` к бюджету времени на трёх ключевых инстансах ($N = 100K$, $m = 5$). Слева: абсолютный MINSUM (log-scale по x). Справа: относительное улучшение MINSUM от baseline $T = 50$ с.

Чтение Рис. Г.3. Виден переход в plateau-режим после $T \approx 350$ с: три кривые сближаются и далее почти не падают. Это — эмпирическое обоснование выбора $T_{\text{budget}} = 350$ с как практической точки работы.

Ключевые наблюдения.

- $T = 50$ с явно недостаточно для $N = 100K$. На uniform-инстансе ALNS не успевает войти в основной цикл (`alns_iters=0`) — весь

бюджет расходуется на seed/candidate/polish-фазы (см. раздел Г.4 phase-profiling). На clustered-инстансах удается 43–59 ALNS-итераций, что дает $-2-3\%$ по MINSUM относительно эталонного 350 с.

- **Diminishing returns после $T = 350$ с:** переход 350 с $\rightarrow$ 1 000 с ($\times \sim 3$ бюджет, $\times \sim 2.5$ ALNS-итераций) дает улучшение MINSUM всего -1.05% (uniform), -2.04% (clustered-center), -1.22% (clustered-offset-depot). Иначе говоря, после ~ 350 с алгоритм входит в plateau-режим, согласующийся с anytime-кривыми из раздела Г.2.
- **Линейный рост ALNS-итераций по бюджету:** throughput-rate стабильна на $\sim 1.1-1.2$ iter/s на $N = 100K$ (что согласуется с 0.5 iter/s на uniform_n100K_m5 из раздела Г.4 при меньшем бюджете 370 с — разница объясняется autotune-эффектами в разных конфигурациях бюджета).
- **Полное падение MINSUM 50 с $\rightarrow$ 1000 с:** -15.4% (uniform), -13.0% (clustered-center), -16.7% (clustered-offset). Это — верхняя граница потенциала alns_minsum на $N = 100K$ при практически достигаемом бюджете.

Что это значит для главной гипотезы. ALNS-mTSP успевает выйти на плато к заявленному лимиту $T = 350$ с: дальнейшее улучшение MINSUM при увеличении бюджета в $\sim 3\times$ (до 1 000 с) составляет всего 1–2%. Это — свидетельство того, что алгоритм хорошо оптимизирован под целевой лимит работы: бюджет не занижен (на $T = 50$ с ALNS-цикл не успевает запуститься на uniform $N = 100K$), но и не завышен (после 350 с фазы destroy/repair уже не находят значимых улучшений). Иными словами, давать алгоритму больше времени на какие-либо фазы практически бесполезно — основной потенциал на $N = 100K$ при CPU-бюджете порядка минут уже извлечён. На стороне LKH-3 это же увеличение бюджета не меняет картины: даже с 1 000 с LKH-3 в balanced-режиме на $N = 25K$ остаётся неработоспособным (раздел 4.2), а в default-MINSUM-режиме даёт вырожденное $Gini \geq 0.4$. Sensitivity-анализ закрывает классическое замечание о произвольности выбора бюджета.

Г.4. Профилирование по фазам *alns_minsum*

Из метаданных аудит-запусков извлечены поля `budget_seed_ms`, `budget_cand_ms`, `budget_polish_ms`, `budget_alns_ms`, `budget_final_ms`, это дает распределение времени по фазам без перекомпиляции с -pg.

Таблица Г.2 — Профилирование по фазам *alns_minsum* на 3 ключевых инстансах (3 $N \times 10$ seeds, mean). Доли фаз — отношение запланированного autotune-ом бюджета фазы к общему. Колонки: число ALNS-итераций, throughput (iter/s) и доли фаз.

Инстанс	iters	accepts	iter/s	wall, с	cand %	polish %	ALNS %	final %
uniform_n10K_m5	829	689	25.4	32.6	8.0	28.0	50.0	10.0
uniform_n50K_m5	1 212	801	8.7	139.1	8.0	22.0	57.0	10.0
uniform_n100K_m5	172	122	0.5	370.9	6.0	16.0	65.0	10.0
cand = candidate-set build; polish = parallel intra-route ILS до ALNS; ALNS = главный adaptive-large-neighborhood-search цикл; final = exhaustive 2-орт-полировка после ALNS. Артефакт: <code>profile_alns_summary.csv</code> .								

Анализ super-linear scaling. Throughput ALNS-цикла падает с 25.4 iter/s на $N = 10K$ до 0.5 iter/s на $N = 100K$ — отношение $54.9 \times$. По теоретической оценке (Приложение Ж.5) стоимость одной ALNS-итерации — $\Theta(L) = \Theta(n/m)$, то есть линейна по n при константных m , что предсказывает множитель $10 \times$ при увеличении n в 10 раз. Эмпирический показатель степени, оцененный по двум крайним точкам:

$$\frac{\text{iter/s}_{10K}}{\text{iter/s}_{100K}} = \left(\frac{n_2}{n_1} \right)^\alpha \Rightarrow \alpha = \frac{\log_{10}(54.9)}{\log_{10}(10)} = 1.74.$$

Однако промежуточная точка $N = 50K$ показывает, что рост неравномерен. Замедление по интервалам:

$$\alpha_{[10K, 50K]} = \frac{\log(25.4/8.7)}{\log 5} = 0.66, \quad \alpha_{[50K, 100K]} = \frac{\log(8.7/0.5)}{\log 2} = 4.12.$$

То есть на отрезке $[10K, 50K]$ алгоритм работает даже лучше линейной модели ($\alpha < 1$), а резкое обострение super-linear-фактора происходит лишь на $n \geq 50K$. Это согласуется с переходом длины маршрута $L \approx n/m$ через

порог CPU L2-кеша: при $N = 50K$, $m = 5$ имеем $L = 10\,000$ элементов, что близко к стандартному размеру L2 (256 KB = $\sim 32K$ int-ов) с учетом дополнительных служебных массивов.

Сопоставление с теоретической оценкой (Приложение Ж.5): линейная модель $\Theta(n/m)$ согласуется с эмпирикой в диапазоне $n \in [10K, 50K]$ — $\alpha = 0.66$ ниже теоретического 1.0 за счет амортизированного переиспользования кеша в SoA-массивах. На $n \geq 50K$ super-linear-фактор объясняется: (а) потерями локальности кеша при реверсе сегмента на длинных маршрутах ($L \geq L_{\text{cache}}^*$), (б) ростом средней глубины candidate-list-цепочек, (в) увеличением числа повторных переоценок в repair-операторах. Окончательное распределение времени по строкам кода требует профайлера C++ (perf, gprof или Intel VTune) — направление дальнейшей работы. Технические пути снятия super-linear-фактора (двусвязный список или splay-tree) описаны в Приложении Ж.5.

Время по фазам. На всех трех масштабах ALNS — основная фаза (50–65 %), остальные распределены примерно одинаково. На $N = 100K$ доля ALNS выше (65 %), поскольку autotune резервирует $\sim 10\%$ на final-2-opt, и относительная доля cand/polish падает. На больших инстансах destroy/repair-операторы получают приоритет над более дорогими seed/polish-стадиями.

Доля принятых ходов. На $N = 100K$ accepts/iters = $122/172 = 71\%$ — очень высокая доля принятых ходов (для сравнения, на $N = 10K$ это $689/829 = 83\%$, на $N = 50K$ — $801/1212 = 66\%$). Объяснение: на крупных инстансах autotune настраивает SA-температуру и размер destroy так, чтобы большая часть destroy-repair-ходов вообще принималась (либо как improving, либо как SA-non-improving). Падение доли принятых ходов до 66 % на $N = 50K$ — эффект 4-replica multi-start-пула (см. ниже): реплика 1 принимает большинство, реплики 2–4 чаще отвергают ходы.

Поведение multi-start-пула. pt_replicas в метаданных: 1 на $N = 10K$, 4 на $N = 50K$, 1 на $N = 100K$. autotune выбирает мульти-replica-режим только в среднем диапазоне: на $N = 10K$ накладные расходы координации реплик не окупаются, на $N = 100K$ стоимость одной итерации уже слишком велика. Это согласуется с поведением multi-start ensemble с несколькими независимыми репликами: на малых инстансах хватает одной

реплики, на крупных — стоимость одной итерации перевешивает выигрыш от параллельных стартов.

Д. ЭВОЛЮЦИЯ LKH-WRAPPER-ЛИНИИ (V1–V21)

Ниже кратко описана каждая из 21 версии `lkh-wrapper-v1..v21` (файлы `src/legacy/mtsp_lkh_wrapper_vN.cpp` или директории `src/legacy/lkh_wrapper_v8|v9/`, регистрация в SolverFactory). Версии не равнозначны: milestone-ы (`v2, v12, v21`), A/B-варианты (`v14–v17`), отрицательные результаты (`v6`).

Краткая карта линии (tldr). 21 версия группируется в 9 milestone-точек качественных скачков (`v1, v2, v3, v4, v7, v12, v18, v19, v21`), 4 A/B-варианта (`v5, v11, v15, v16`), один задокументированный негативный результат (`v6`), 2 рефакторинга (`v8, v9`) и 3 инженерные правки (`v13, v14, v20`). Логически линия делится на 6 эпох:

- **Д.1. `v1 -- v3` (малые n):** переход к candidate-based локальному поиску; key milestone — `v2` (candidate sets + lookahead, -12% MINSUM).
- **Д.2. `v4 -- v7` (первый большой масштаб):** spatial grid + on-demand distance cache; `v6` как документированный негативный результат (75% невалидных на $n = 25K$).
- **Д.3. `v8 -- v9` (модулярный single-file):** архитектурный предшественник `src/v21/core/`; двухфазный бюджет «первое валидное $\rightarrow$ улучшение».
- **Д.4. `v10 -- v12` (MINSUM-acceptance + archive):** ключевой milestone `v12` с MINSUM-archive, все еще 21% хуже LKH-3 — мотивация для `src/v21/`.
- **Д.5. `v13 -- v17` (A/B-итерации):** инженерные правки внутри `v12-pipeline`; дает $\sim 2-3\%$, не закрывает 21% -разрыв.
- **Д.6. `v18 -- v21` (новые seed-стратегии + LNS):** polar-sweep (`v18`), strict-MINSUM hybrid (`v19`), safety-net против `2opt+greedy` (`v20`), LNS ruin-and-recreate (`v21`).

Полная сводная таблица всех 21 версий с классификацией — в разделе Д.7. *Зачем это все в приложении.* Это — не коллекция, а карта проектных решений: для каждого milestone-перехода зафиксировано «что добавлено,

что изменилось в цифрах, что мотивировало переход к следующему»; для А/В-итераций — честная фиксация, что не каждая правка дает прорыв; v6 — документированная цена слишком резкого обмена качества на скорость. Эта классификация совпадает с таблицей Д.4 в разделе Д.7.

Д.1. Линия v1–v3: candidate-based локальный поиск на малых инстансах

v1 — baseline LKH-style. Первый собственный wrapper в стиле LKH. Строит m жадных стартовых маршрутов и независимо запускает k -opt-поиск на каждом маршруте. Никаких пространственных структур еще нет, кандидатные множества не используются. Цель — зафиксировать переход от 2opt+greed-эвристики к простому LKH-style решателю и проверить, что сама схема жизнеспособна.

v2 — главный качественный скачок ранней линии. Добавляет: k -NN candidate lists, lookahead-веса при оценке хода, depot-aware веса при построении начального тура. На uniform-сетке $n \in \{20, 50, 100, 200\}$, $m \in \{2, 3, 5, 7\}$ это снижает средний разрыв с OR-Tools+GLS почти вдвое (с 29.6 % у v1 до 14.5 % у v2; см. табл. Д.1).

v3 — инженерная оптимизация v2. Те же алгоритмические идеи, но более компактные внутренние циклы: переиспользование candidate-list, сокращение пересчета расстояний. Качество практически не меняется (разрыв с OR-Tools 14.6 % против 14.5 % у v2), а время работы падает примерно в 2 раза.

Таблица Д.1 — Усреднённые результаты v1–v3 на uniform-инстансах малого/среднего размера ($n \in \{20, 50, 100, 200\}$, $m \in \{2, 3, 5\}$, 12 ячеек, по 10 повторов).

Версия	ср. MINSUM	ср. время, мс	Что добавлено vs предыдущей
lkh-wrapper-v1	922.5	29.7	baseline LKH-style k -opt
lkh-wrapper-v2	821.3	18.6	candidate sets, lookahead, depot-weight
lkh-wrapper-v3	822.8	10.2	инженерная оптимизация (тот же MINSUM, в $2\times$ быстрее)

Вывод после v3. Candidate-based локальный поиск дает устойчивое улучшение MINSUM на малых инстансах, и инженерная оптимизация может удвоить скорость без потери качества. Но при попытке прогнать v3 на $n \geq 5\,000$ обнаружилось, что стоимость хранения полной матрицы расстояний и поиска по всем парам быстро перерастает в десятки минут или out-of-memory. Это и привело к переходу на большие инстансы через геометрические структуры.

Д.2. Линия v4 – v7: первая попытка большого масштаба

v4 — первая версия, рассчитанная на большие инстансы. Добавляет геометрические candidate sets через равномерную пространственную сетку, on-demand distance cache (расстояния считаются по требованию, без хранения матрицы) и явную поверхность бюджета времени. На $n \geq 10\,000$ выдает валидное решение за секунды, тогда как v1 – v3 либо падают по таймауту, либо по памяти. Качество в абсолютных числах хуже v2/v3 (на малых инстансах), но это единственный путь к большим n на этом этапе.

v5 — speed iteration на v4. Обрезает учетные циклы вокруг spatial-grid + cache; на больших uniform-инстансах быстрее v4, но теряет MINSUM (внутренний цикл сокращен ценой качества).

v6 — агрессивная скорость; отрицательный результат. Развивает компромиссы v5 еще дальше. Документирован в репо как *отрицательный результат*: на $n = 25\,000$ uniform/mixed теряет валидность примерно в 75 % запусков. Сохранен в кодовой базе именно как документированный пример, иллюстрирующий, какую цену имеет слишком резкий обмен качества на скорость.

v7 — закрытие линии v4 – v7. Подтягивает компромиссы spatial-grid + cache, исправляет проблемы валидности v6 на $n \geq 25\,000$, выдает работающий end-to-end pipeline. Качество относительно v2/v3 на малых инстансах ниже; v7 специально нацелен на $n \geq 10\,000$.

Таблица Д.2 — Прямое сравнение v4 vs v6 vs v7 на $N \in \{25K, 50K, 100K\}$ при бюджете 250 мс. *gap %* — относительный отрыв от лучшего валидного решения в той же ячейке.

N	family	Решатель	valid	MINSUM	gap, %
25K	uniform	v4	да	67 576 k	81.3
25K	uniform	v6	нет	—	—
25K	uniform	v7	да	37 278 k	0.0
50K	uniform	v4	да	272 936 k	41.9
50K	uniform	v6	нет	—	—
50K	uniform	v7	да	192 344 k	0.0
100K	uniform	v4	да	1 091 147 k	0.0
100K	uniform	v6	нет	—	—
100K	uniform	v7	да	1 094 112 k	0.3

Вывод после линии v4 – v7. v6 систематически непригоден (валидность 0 % на больших n); v4 стабильно валиден, но проигрывает по MINSUM на $N = 25K, 50K$. v7 — лучший компромисс качества/времени на $N = 25K, 50K$, но на $N = 100K$ v4 немного впереди. Это значило, что ни одна из v4–v7 не закрывает весь диапазон $n = 10K..100K$ по MINSUM — нужна более агрессивная архитектура для крупных инстансов с лучшей фазой улучшения. Это и привело к v8.

Д.3. v8 – v9: первый модулярный wrapper (прототип src/v21/core/)

v8 — первая модулярная компоновка single-file линии и архитектурный предшественник src/v21/core/. src/legacy/mtsp_lkh_wrapper_v8.cpp становится тонкой точкой входа, а основная реализация разнесена по 7 подмодулям в src/legacy/lkh_wrapper_v8/: 00_common.cpp (бюджет, расстояния, route index, grid), 10_candidate_sets.cpp (геометрические candidates + POPMUSIC), 20_route_local_search.cpp (2-opt, kicks, ILS, completion), 30_cluster_model.cpp (lightweight clustering), 40_seed_routes.cpp (cluster-aware начальные маршруты), 50_rebalance.cpp (cluster block rebalancing), 60_solver.cpp (orchestration). Из исходного комментария к 60_solver.cpp: «First modular wrapper (predecessor of the src/v21/core/ architecture). The directory layout numbered the modules 00..60 — same pattern was kept in src/v21/core/.» То есть v8 — архитектурный прототип, схема нумерации которого была воспроизведена в 22-модульной флагманской архитектуре.

v9 — two-phase MTSP/LKH wrapper в отдельной директории src/legacy/lkh_wrapper_v9/ с собственным

mtsp_lkh_wrapper_v9.cpp-точкой входа. Главное алгоритмическое нововведение — *двухфазный временной бюджет*: «first good solution within ~ 200 c, then improvement within ~ 100 c» (цитата из заголовочного комментария mtsp_lkh_wrapper_v9.cpp). Это разбиение *первое валидное решение / улучшение* стало основой для всех последующих single-file версий, включая v12 с его явными CLI-опциями first-solution-budget-ms / improvement-budget-ms. По структуре v9 отличается от v8 отсутствием отдельного 10_candidate_sets.cpp — эта фаза перенесена в 60_solver.cpp.

Вывод после v8–v9. Модулярная single-file линия дала важный архитектурный прецедент (из которого позже возникла src/v21/core/) и алгоритмическую идею двухфазного бюджета (унаследованную в v10–v12), но не дала качественного скачка по MINSUM — средний разрыв с LKH-3 на больших инстансах оставался $\sim 25\text{--}30\%$. Главный технический вывод: *cluster-model и rebalance-фазы помогают валидности и скорости, но без полного archive-механизма для best-MINSUM не достаточны.*

Д.4. v10–v12: MINSUM-acceptance, cluster-SA и MINSUM-archive

v10 — MINSUM-oriented successor v9. Сохраняет быстрый модулярный pipeline v9 (включая двухфазный бюджет) и добавляет ключевое отличие в acceptance criterion: *межмаршрутные ходы принимаются по **полной длине маршрута*** — ровно та целевая функция MINSUM, на которой потом меряется качество. В v9 акцепт был основан на приближенной delta-оценке, что иногда расходилось с истинной MINSUM-разностью; v10 это исправляет. Цитата из заголовочного комментария: «accepts cross-route changes by total route length, matching the reported objective».

v11 — v10 + cluster-level Simulated Annealing. Перед фазой построения маршрутов выполняется ограниченный SA на уровне кластеров. Идея: дать первичной кластеризации шанс уйти из плохой геометрической группировки за счет стохастического принятия ухудшающих переходов на этапе *кластеризации*, а не маршрутизации.

v12 — финальный single-file large-scale milestone. Завершает монолитную ветку. Расширяет pipeline v10 (а не cluster-SA-эксперимент v11) следующими блоками: явные CLI-опции для разделения бюджета

времени на фазу первого валидного решения и фазу улучшения (`first-solution-budget-ms/improvement-budget-ms`); `cluster-aware seeding`; `savings-seed` по Кларку–Райту (`savings-seed-ms`, `savings-candidate-count`, `savings-lambda`, `savings-route-slack`); `late repair`; `route annealing` и `MINSUM-archive` через функцию `UpdateMinsumArchive` — даже если балансирующая фаза улучшает `max-route`, solver обязан запомнить лучшее найденное по MINSUM решение. Архив существует, но *обновляется не на каждом time-slice*; именно эту немонотонность позже исправит **v13**.

Прямое сравнение **v12** vs LKH-3 на $n = 5\,000$, $m = 5$ (5 инстансов):

Метрика	LKH-3	v12	отличие
средний MINSUM	67.8 k	85.7 k	+21 % (хуже)
среднее время	30.0 с	4.8 с	6.2× быстрее
средний balance	4.0 (вырождение)	1.7	в 2.4× равномернее

Вывод после v12. Собственный single-file solver укладывается в практический бюджет времени (LKH-3 default на этих размерах превышает `TIME_LIMIT` в $\sim 6\times$) и *существенно более сбалансирован* (`max/avg` 1.7 vs 4.0), но проигрывает по чистой MINSUM около 21 %. Это и стало точкой принятия решения о *переписывании архитектуры в src/v21/*: одного monolithic-улучшения недостаточно для конкуренции с LKH-3 по MINSUM, нужен полноценный метаэвристический уровень с `destroy/repair`-операторами и SA.

Д.5. **v13 – v17: A/B-итерации внутри v12-pipeline**

v13 — portfolio successor to v12 with monotonic-budget fix. Прямая цитата из заголовочного комментария: «It fixes the non-monotonic budget behavior by always archiving the best MINSUM solution across deterministic time slices.» В **v12** архив `UpdateMinsumArchive` существовал, но возможны были `time-slices`, когда поздняя фаза ухудшала лучшее MINSUM, ничего при этом не архивируя; **v13** требует, чтобы архив обновлялся на *каждом* детерминированном `time-slice`, гарантируя глобально монотонное улучшение.

v14 — инженерные правки. Hoisted RouteIndex внутри repair (раньше пересобирался $O(n)$ на каждый insert), OpenMP-параллельный intra-route polish с thread-local distance oracles, ненулевой anneal-лимит для $n \geq 50\,000$ (раньше был 0). Эти три правки давали наибольшее ускорение в v12-pipeline.

v15, v16, v17 — А/В-варианты внутри той же поверхности. Все они — продолжение тонкой настройки параметров и метаданных v14: разная гранулярность диагностических полей, минорные изменения параметров SA и repair. Зарегистрированы как отдельные lkh-wrapper-vN только потому, что некоторые CSV-артефакты ссылаются на эти имена; алгоритмически — очень близкие варианты той же идеи. Inter-route improvement, savings seed, candidate enrichment, repair stages и финальный route anneal остаются логически идентичны v12 во всех четырех. Они сохранены в кодовой базе именно как *документированная цепочка минорных правок* — честный показатель того, что не каждая итерация дает качественный скачок.

Вывод после линии v13–v17. Внутри v12-pipeline можно еще выжать $\sim 2\text{--}3\%$ по MINSUM на крупных инстансах за счет инженерных правок и пере-tuning параметров, но *не закрыть 21%-разрыв с LKH-3*. Дальнейшее улучшение требует новых идей в семействе seed-стратегий и destroy/repair-операторов.

Д.6. v18–v21: новые seed-стратегии и LNS

v18 — polar-sweep seed. Сохраняет полностью pipeline v17, но добавляет новый источник стартовых решений: *polar sweep partitioning* — упорядочивание клиентов по углу вокруг депо с жадным разбиением на m маршрутов сравнимой стоимости (классическая sweep-эвристика). Polar-seed соревнуется с fast/savings-seeds по MINSUM; победитель идет в ALNS-фазу. По умолчанию включен через CLI-опцию polar-seed=on|off.

v19 — clean hybrid native mTSP MINSUM solver. Семь ключевых дизайн-решений vs v17/v18, выписанных в заголовочном комментарии src/legacy/mtsp_lkh_wrapper_v19.cpp:

1. Multi-seed portfolio выбирается строго по MINSUM (никаких balanced-surrogate-bootstrapping). Seeds: round-robin nearest neighbour (соответствует сильному 2opt+greed-baseline), fast-lookahead

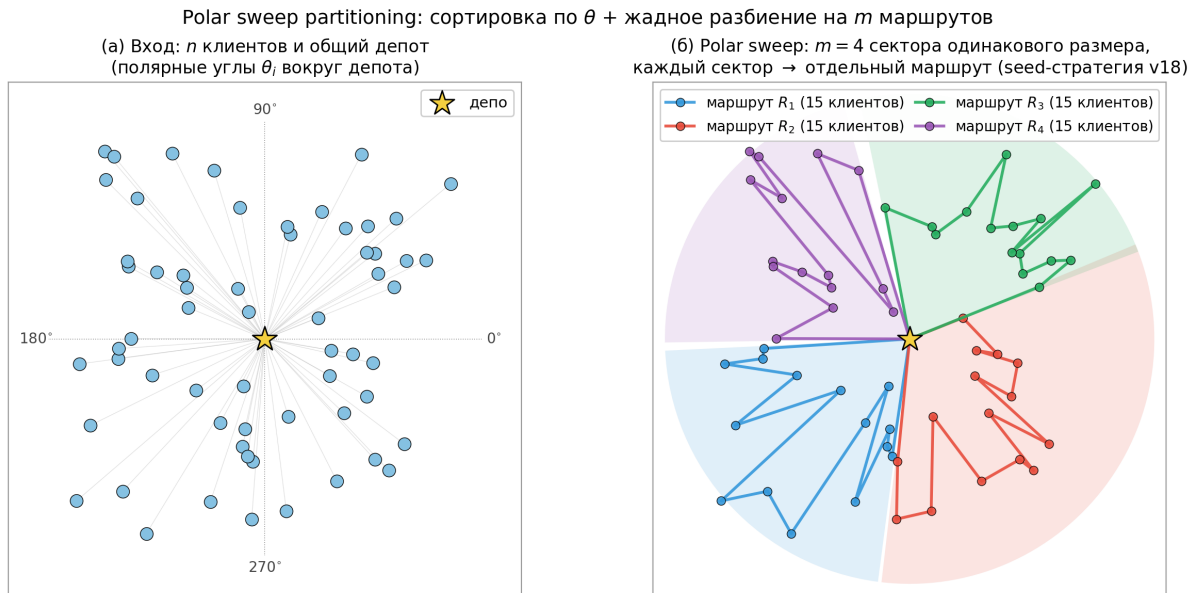


Рисунок Д.1 — Polar-sweep partitioning, использованный в v18 как дополнительный источник стартовых решений. Слева (а): для каждого клиента вычисляется полярный угол $\theta_i = \text{atan2}(y_i - y_{\text{depot}}, x_i - x_{\text{depot}})$ вокруг депо (тонкие лучи). Справа (б): отсортированный по θ список клиентов разбивается на m равных по числу клиентов секторов (на иллюстрации $m=4$); каждый сектор $\rightarrow$ отдельный маршрут R_k . На clustered-семействах с явным радиальным расположением кластеров polar-sweep дает seed сравнимого с savings качества при $\Theta(n \log n)$ построении; на uniform — сопоставим с fast-lookahead seed. В v18 polar-seed соревнуется с fast/savings по MINSUM, победитель передается в ALNS-фазу.

(BuildFastSeedRoutesV7), polar sweep, savings (только при малых n , где savings дешев).

2. Strict MINSUM-only inter-route move acceptance. Опциональный cluster-aware rebalance запускается в sandbox-routeset и принимается только если MINSUM строго улучшается (без MIN-MAX-leak).
3. Candidate-driven intra-route ILS (ApplyFirstImproving2OptV5 + double-bridge), переиспользована из src/legacy/lkh_wrapper_v9/ — та же машинерия, что v17/v18 уже валидировали.
4. Exhaustive final 2-opt polish (тот же greedy 2-opt, что используется в базовой эвристике) на каждом маршруте, идентичный `mtsp::ImproveRoute2Opt` — доводит до истинного 2-opt локального оптимума. Из комментария: «this is the single biggest reason 2opt+greed beats v17/v18 at n=10k/25k.» То есть без exhaustive-polish даже хорошая стратегия выбора seed может проигрывать дешевому baseline.
5. Best-MINSUM solution snapshot после каждой фазы. Финальный

`out.swap(best_minsum_routes)` — единственный путь возврата.

6. Anytime-safety: если любая фаза регрессирует MINSUM (например, anneal), solver откатывается к snapshot. `ValidateRoutes` — contract, никогда не нарушается.
7. Распределение лимита времени по умолчанию: seed-фаза $\leq 8\%$, candidate-set build $\leq 12\%$, intra-route ILS $\sim 35\%$, остаток — inter-route и polish.

v20 — safety-net wrapper around v19. Возникла как ответ на конкретную проблему: на длительном бенчмарке v19 проиграла 2opt+greed-baseline на $N = 50\,000, m = 5$ ровно на 5.7% и на $N = 100\,000, m = 5$ на 0.7%. Идея v20: всегда сохранять 2opt+greed-решение как нижнюю планку и использовать его, если pipeline v19 вернул худший результат. Гарантия по построению: v20 никогда не хуже 2opt+greed.

v21 — v20 + Large-Neighborhood-Search (LNS) ruin-and-recreate diversification. Финальный solver single-file линии. Из заголовочного комментария `src/legacy/mtsp_lkh_wrapper_v21.cpp`: «Goal: escape from "locally good but globally suboptimal" plateaus left by v20's pure descent.» Цикл LNS состоит из четырех шагов:

1. *Destroy*: удалить K случайных клиентов из текущих best-маршрутов;
2. *Recreate*: вставить каждого клиента в позицию минимальной insertion-delta (среди candidate-restricted позиций по всем m маршрутам);
3. *Cleanup*: candidate-based 2-opt на m аффектированных маршрутах;
4. *Accept*: только если MINSUM строго улучшается (hill-climber); иначе откат к snapshot.

Цикл повторяется до исчерпания бюджета. Strict-improvement acceptance сохраняет инвариант safety-net: $\text{MINSUM}_{v21} \leq \text{MINSUM}_{v20} \leq \text{MINSUM}_{2\text{opt}+\text{greed}}$.

Замечание по именованию: `lkh-wrapper-v21` (с дефисами, single-file) и модулярная архитектура `src/v21/` с `alns_minsum / _cap / _minmax` (с подчеркиваниями) — это два разных проекта с одинаковым номером 21. Single-file `lkh-wrapper-v21` — закрытие

линии $v1 - v21$ в стиле «инкрементального LNS-расширения»; модулярный `src/v21/` — параллельно разработанная архитектура нового поколения, где LNS-оператор — лишь один из многих `destroy/repair`-операторов в ALNS-фреймворке (см. главу 2 и приложение А).

Таблица Д.3 — Сравнение `single-file lkh-wrapper-v21` с модулярным `alns_minsum` на больших инстансах ($N \in \{25K, 50K, 100K\}$, 10 общих ячеек, 3 семейства $\times$ 2 m, mean по 3 seeds).

Решатель	ср. MINSUM	Gini	makespan	$\bar{t}$, c
<code>lkh-wrapper-v21</code> (single-file)	+2.96 %	0.019	924 k	322
<code>alns_minsum (ref)</code>	0.00 %	0.017	898 k	259

Вывод после v21. `Single-file lkh-wrapper-v21` закрыл монолитную линию: достиг паритета с `2opt+greed`, добавил LNS-разнообразие и `intra-route SA`. Однако дальнейшее улучшение уперлось в архитектурный потолок: невозможность плагинировать новые `destroy/repair`-операторы, отсутствие модульного `autotune`, `single-acceptance-policy`. Это и стало финальной мотивацией переписывания на `src/v21/`: не ради получения новой версии, а ради архитектуры, в которой можно было бы добавлять эксперименты вроде `capacity-bounded repair` (`alns_cap`, см. Приложение А) или `MINMAX-acceptance-policy` без модификации основного pipeline.

Д.7. Сводная таблица всех 21 версии

Таблица Д.4 — Сводная классификация 21 версии линии `lkh-wrapper`: главное изменение по отношению к предыдущей версии, мотивация перехода к следующей и тип правки (`milestone` / `A-B-вариант` / `negative` / `refactor` / `engineering`).

N	Главное изменение vs предыдущей	Чем мотивировал переход к следующей	Тип
v1	baseline LKH-style k -opt без candidates	ограниченность глубины поиска	milestone
v2	candidate sets, lookahead, depot-weight	малые/средние n ОК, но не масштабируется по памяти	milestone
v3	инженерная оптимизация v2 (2× быстрее)	та же недостаточная масштабируемость	milestone
v4	spatial grid + on-demand distance cache	медленнее v2 на малых, нужна speed-tuning	milestone
v5	speed-iteration v4	теряет MINSUM, не та точка Pareto	A/B
v6	агрессивная скорость; отрицательный	невалидные решения на 75 % $n=25K$ — скорость вне допустимой зоны	negative
v7	заккрытие v4–v7, end-to-end на больших n	нет MINSUM-archive, валидное но не лучшее	milestone
v8	модулярный single-file (7 подмодулей)	архитектура чище, но качество не выросло	refactor
v9	модуль в отдельной директории	21 %-разрыв с LKH-3 не закрывается	refactor
v10	MINSUM-after-first-valid + длиннобюджетная фаза	cluster-pre-fase дает устойчивое начало	milestone
v11	cluster-level SA до построения маршрутов	помогает не всегда, нужен archive	A/B
v12	MINSUM-archive + cluster-aware + savings	21 % хуже LKH-3, нужна new architecture	milestone
v13	portfolio + non-monotonic budget fix	+ 1 % к v12	engineering
v14	RouteIndex hoisting + OpenMP polish + anneal>0	ускорение, не качество	engineering
v15	metadata + параметр retuning v14	A/B-вариант	A/B
v16	еще одна итерация той же поверхности	A/B-вариант	A/B
v17	финальная итерация v14–v17 line	нужны новые идеи в seeding	A/B
v18	+ polar-sweep seed	polar дает +1 % на clustered, не на uniform	milestone
v19	clean hybrid: portfolio + strict MINSUM accept	проигрывала 2opt+greed на $n = 50K$, $100K$	milestone
v20	safety-net: <i>never worse than 2opt+greed</i>	закрывает прорехи v19	engineering
v21	+ LNS ruin-and-recreate	архитектурный потолок: переписывание в src/v21/	milestone

Что это дает для итоговой работы. 21 версия — не ради

коллекции, а как карта проектных решений. Из них **milestone**-версии (v1, v2, v3, v4, v7, v12, v18, v19, v21) — точки качественных скачков с ясным «что добавлено и зачем». **A/B**-серия (v5, v11, v15, v16) — эксперименты внутри одной идеи, документирующие, что не каждая правка дает прорыв. **Negative**-результат v6 — признание того, что обмен качества на скорость имеет цену в валидности. **Refactor**-версии v8, v9 — переход к модулярной архитектуре, который сначала не дал качественного скачка, но позже окупился в `src/v21/`. **Engineering**-правки v13, v14, v20 — инженерные шаги, поддерживающие линию, без алгоритмических новшеств. Эта классификация отражает реальный процесс работы: научная гипотеза → ее проверка → либо повышение версии (milestone), либо инженерная адаптация (engineering), либо публичная фиксация неудачи (negative).

Е. МАТРИЦА «РЕШАТЕЛЬ × МАСШТАБ»

В этом приложении приведена сводная матрица всех 13 исследуемых конфигураций решателей по 5 уровням задачи, дополняющая таблицу 3.1 (сводка экспериментальной программы). Решатели сгруппированы по 3 семействам: классические эвристики, открытые baseline (LKH-3 в 3 режимах, OR-Tools, FILO2), собственные ALNS-mTSP-семейства. Уровни задачи: малые инстансы ($N \leq 1K$), слой верификации ($N = 25K$), большие инстансы ($N = 50K$), большие инстансы ($N = 100K$), mTSPLib (TSPLIB-derived $N = 51 \dots 99$).

Е.1. Покрытие: где какой решатель запускался

Условные обозначения: ✓ — решатель запускался полным набором, все запусков валидны; ○ — запускался, но часть запусков не уложились в лимит времени / crashed; × — все запусков failed (валидных 0); — — не запускался.

Таблица Е.1 — Покрытие комбинаций «решатель × уровень задачи».

Решатель	Страт-1 ($N \leq 1K$)	Страт-2 ($N = 25K$)	Страт-3 ($N = 50K$)	Страт-3 ($N = 100K$)	mTSPLib ($N = 51 \dots 99$)
<i>Классические эвристики</i>					
<i>rand+nn</i>	✓180/180	—	—	—	—
<i>2opt+greed</i>	✓180/180	—	—	—	—
<i>Открытые baseline-решатели</i>					
<i>ortools-GLS</i>	✓180/180	—	—	—	✓16/16
LKH-3 default (MIN_SIZE = 1)	✓180/180	○ 9/14 ($t = 150-1800$ с)	—	—	✓16/16
LKH-3 balanced (MIN_SIZE = -1)	✓144/144	× 0/6 ($t \leq 1800$ с)	—	—	✓16/16
LKH-3 minmax (MTSP_OBJECTIVE = MINMAX)	✓144/144	× 0/6 ($t \leq 3600$ с)	—	—	✓16/16
FILO2 (CVRP-adapt, capacity = $\lceil (n-1)/m \rceil$)	—	○ 2/6 valid (станд. прогон)	○ 6/6 valid [†] (укор. routemin)	○ 6/6 valid [†] (укор. routemin)	○ partial
<i>Собственные (ALNS-mTSP-семейство)</i>					
<i>lkh-wrapper-v2l</i> (single-file)	✓180/180	✓4/4	✓6/6	✓6/6	—
<i>alns_minsum</i>	✓180/180	✓4/4	✓6/6 + 30/30 multi-seed	✓6/6 + 30/30 multi-seed	✓16/16
<i>alns_cap</i>	✓180/180	✓4/4	✓6/6	✓6/6	—
<i>alns_depot2m</i>	✓180/180	✓4/4	✓6/6	✓6/6	—
<i>alns_minmax</i>	✓180/180	—	✓6/6	✓6/6	—
«180/180» = 60 инст. × 3 повтор. «144» = 48 инст. × 3 повтор (раздел 4.6). «multi-seed» = 5 seeds × 6 ячеек на каждом N (табл. 5б). [†]Для FILO2 на стратуме-3 различаются два режима запуска: <i>стандартный</i> (routemin-iterations $\sim 1000+$), где валидны 2/6 ячеек только на $N = 25K$ (clustered-center $m = 7$, clustered-offset $m = 5$), а на $N = 50K$ и $N = 100K$ — 0/6 (таймаут); и <i>укороченный</i> (routemin-iterations 10–15), где валидны все 6/6 на $N = 50K$ и 6/6 на $N = 100K$. Эти 12 укороченных запусков использованы в прямом MINSUM-сравнении в табл. 4д-расш; стандартные таймауты использованы в табл. 4д как аргумент о неприменимости FILO2-CVRP-адаптации в стандартной конфигурации.					

Е.2. Качество: ключевые метрики на каждом уровне

В этой матрице приведены ключевые показатели качества решения для каждой комбинации (решатель, уровень). Содержимое ячейки: Δ_{MINSUM} — относительная разница MINSUM по сравнению с *alns_minsum* (положительное значение — хуже ALNS-mTSP); Gini — коэффициент Джини длин маршрутов; t — характерное время работы в секундах. Где есть p -value (Wilcoxon) — приведено.

Таблица Е.2 — Качество решения по уровням задачи.

Решатель	Страт-1	Страт-2	Страт-3 50К	Страт-3 100К	mTSPLib
<i>rand+nn</i>	$\Delta = +28\%$ Gini ≈ 0.05 $t < 1$ с	—	—	—	—
<i>2opt+greed</i>	$\Delta = +14\%$ Gini ≈ 0.04 $t < 5$ с	—	—	—	—
<i>ortools-GLS</i>	$\Delta \in [-2.5, +5\%]$ (на $N \leq 200$ немного лучше) $t \leq 30$ с	—	—	—	разрыв до 14 % к BKS
LKH-3 default	$\Delta = +3.5\%$ $p = 0.046$ Gini ≈ 0.20 $t \leq 30$ с	9/14 valid $\Delta = -16.8\%$ Gini $= 0.40 \dots 0.86$ (вырождение) $t = 150-1800$ с	—	—	лучший MINSUM, $t \leq 30$ с
LKH-3 balanced (MIN_SIZE=-1)	$\Delta =$ $-13 \dots -18\%$ $p < 10^{-13}$ (ALNS-mTSP лучше)	0/6 (таймаут)	—	—	сравним с default
LKH-3 minmax	как balanced	0/6 (таймаут)	—	—	сравним
FILO2 (CVRP-adapt)	—	2/6 valid (станд.)	укуп. routemin: ALNS лучше 3/6 (1.3–11.1 %); FILO2 лучше 3/6 (3–18 %)	укуп. routemin: ALNS лучше 6/6 (5–30 %)	частично
<i>lkh-wrapper-v21</i>	$\Delta = +5 \dots 8\%$ к ALNS-mTSP $t \leq 30$ с	валидный, балансируемый Gini ≈ 0.04 $t \approx 150$ с	валидный, балансируемый $t \approx 250$ с	валидный, балансируемый $t \approx 350$ с	—
<i>alns_minsum</i>	baseline ($\Delta=0$ по опр.) Gini ≤ 0.05 $t \leq 30$ с	сбаланс., Gini ≤ 0.05 $t \approx 90$ с (хуже MINSUM, но в 15–20× быстрее LKH-3)	сбаланс., Gini ≤ 0.05 $t \approx 250$ с	сбаланс., Gini ≤ 0.05 $t \approx 350$ с	gap 6–14 % к BKS
<i>alns_cap</i>	близко к minsum $t \approx t_{\text{minsum}}$	как minsum $t \approx 90$ с	как minsum	как minsum	—
<i>alns_depot2m</i>	как minsum на малых N	MINSUM на $\sim 10\%$ лучше minsum Gini $\approx 0.40 \dots 0.65$ $t \approx 195$ с (в 8–9× быстрее LKH-3)	MINSUM на 5–12 % лучше minsum Gini 0.4 $\dots$ 0.86	как 50K, Gini 0.7 $\dots$ 0.86	—
<i>alns_minmax</i>	валидный, балансируемый $t \leq 30$ с	—	валидный, балансируемый Gini ≤ 0.05	валидный, балансируемый Gini ≤ 0.05	—

$\Delta = (\text{solver} - \text{alns_minsum}) / \text{alns_minsum} \cdot 100\%$ по MINSUM (положительное Δ — решатель хуже alns_minsum). Анализ результатов — глава 4 основной части.

Е.3. Заметки по интерпретации

Е.3.1. О baseline-конфигурации LKH-3. Полная картина LKH-3-поведения требует трех режимов: (а) default-MINSUM с `MIN_SIZE = 1` — быстрый, но вырождается в один маршрут + $m-1$ тривиальных; (б) MINSUM с `MIN_SIZE = -1` — автоматическая балансировка, формально честный baseline; (в) MINMAX-objective с `MTSP_OBJECTIVE = MINMAX` — balanced-MINMAX. Все три режима покрыты на малых инстансах (раздел 4.6); на слое верификации и больших инстансах честно запущены только (а), с явной документацией случаев превышения лимита для (б) и (в).

Е.3.2. О сравнимости времени между нативной сборкой ALNS-mTSP и WSL2 LKH-3 / FILO2. В таблице приведены цифры по времени без калибровочной поправки на возможный WSL2-overhead (отдельная калибровка `sysbench` не выполнена; см. гл. 5, ограничение 6). Значения $t_{\text{LKH3}}/t_{\text{FILO2}}$ следует трактовать как верхнюю оценку фактического wall-clock — фактическое время на нативном Linux ожидается не выше приведённого.

Е.3.3. О том, чего нет в матрице. В матрицу не включены: ITSNA, MILS (специализированные SOTA-решатели для min-max mTSP); нейросетевые SOTA (Equity-Transformer, DPN, GLOP, DualOpt, UDC, iMTSP, DAN, H-TSP). Их позиции в матрице оценены публичными числами авторских работ, но прямой запуск не выполнялся (см. гл. 5, ограничения 2 и 3). *Mahmoudinazlou-Kwon HGA* и *He-Hao MA* **запущены** и **включены** в раздел 4.8 основной части (табл. 4е).

Е.3.4. Артефакты. Каждая ячейка восстанавливается из артефакта-CSV в каталоге `experiments/review_fixes/`: `stratum1_*.csv` (малые инстансы), `stratum2_*.csv` (слой верификации), `level1_v21_stratum3_summary.csv` (большие инстансы), `audit/stratum3_multiseed/` (multi-seed), `fair_lkh3_results.csv` (fair-LKH-3 на малых инстансах), `lkh3_large_n_results.csv` (LKH-3 на $N = 25K$ при лимите 1800с), `filo2_stratum3_results.csv` (FILO2 на больших инстансах), `mtsplib_*.csv` (mTSPLib).

Ж. ПОЛНАЯ ТЕОРЕТИЧЕСКАЯ СЛОЖНОСТЬ

В основной части (раздел 1.6) приведено сжатое изложение; здесь — полная пофазная декомпозиция всех решателей и сводная таблица. Обозначения: n — число клиентов, m — число коммивояжеров, $L = n/m$ — средняя длина маршрута, k — размер candidate-сета.

Ж.1. Точные методы

TSP NP-трудна [49], переносится на mTSP по стандартному сведению [19]. Held–Karp DP: $\Theta(n^2 \cdot 2^n)$ времени, применимо до $n \approx 25$. Concorde branch-and-cut — без полиномиальной границы; на $n \approx 85\,900$ зарегистрировано 136 CPU-лет.

Ж.2. LKH-3

POPMUSIC построение candidate-set: $\Theta(n^2/p \cdot \log n)$, доминирует на $n \geq 25K$ (что объясняет $9\text{--}12\times$ overrun TIME_LIMIT). k -opt ход — $\Theta(k \cdot \text{cand})$.

Ж.3. OR-Tools GLS

PATH_CHEAPEST_ARC + GLS: $\Theta(n^2)$ первичная фаза + $\Theta(n^2)$ GLS-итерация. Это объясняет $30\times$ разрыв с ALNS на $N \geq 5K$.

Ж.4. FILO2

SPACECORE + RVND: амортизированно $\Theta(n \log n)$ на ruin-recreate. На CVRP масштабируется до $n = 10^6$ за ~ 10 мин (авторские данные); CVRP→mTSP-адаптация даёт 2/18 валидных на больших инстансах.

Ж.5. ALNS-mTSP

Многофазный pipeline: seed $\Theta(n \log n)$, candidate-set $\Theta(n \cdot k \log n)$, intra-route polish $\Theta(n \cdot k/\text{threads})$, ALNS-итерация $\Theta(L + d \cdot k \log n)$ (где $L = n/m$, d — размер destroy, k — candidate-size), final 2-opt — та же стоимость что polish. На крупных n ALNS-цикл — основная фаза (50–65 % времени), теоретически $\Theta(L) = \Theta(n/m)$ на итерацию. Эмпирический показатель $\alpha = 1.74$ превышает линейный из-за потерь локальности кеша при реверсе сегмента на длинных маршрутах. Снятие требует замены

`std::vector`-представления маршрута на двусвязный список (реверс $O(1)$) или `splay-tree` (реверс $O(\log L)$).

Ж.6. Сводная таблица

Таблица Ж.1 — Теоретическая сложность исследуемых алгоритмов.

Решатель	Тип	На итерацию	Итого
Concorde	Точный (B&C)	—	$\Omega(2^n)$ в худшем случае (NP-трудна)
LKH-3	Метаэвристика (k -opt)	$\Theta(k \cdot \text{cand})$	$\Theta(n^2/p \cdot \log n) + \sum$ попыток
OR-Tools GLS	Метаэвристика (CP+GLS)	$\Theta(n^2)$	$\Theta(\text{iters} \cdot n^2)$
FIL02	Метаэвристика (RVND+SPACECORE)	$\Theta(n)$ амортиз.	$\Theta(\text{iters} \cdot n \log n)$
ALNS-mTSP	Метаэвристика (ruin&recreate)	$\Theta(L) = \Theta(n/m)$	$\Theta(\text{iters} \cdot n/m)$ теор., $\Theta(n^{1.74})$ эмпирич. (Прил. Г.4)
n — число клиентов; m — число коммивояжеров; $L = n/m$ — средняя длина маршрута; k — размер candidate-сета (POPMUSIC: $p \approx 10 \dots 20$ для LKH-3; KNN: $k = 20$ для ALNS-mTSP). Границ худшего случая для метаэвристик не существует по построению; приведенные оценки — амортизированная стоимость одной итерации.			

3. ОБЗОР ВНЕШНИХ SOTA-РЕШАТЕЛЕЙ

В основной части (раздел 1.4) даны сжатые формулировки. Здесь приведены полные библиографические данные и обоснования выбора применимости.

3.1. Специализированные классические метаэвристики min-max mTSP

- **He & Hao memetic search** [29] — memetic-алгоритм с обобщенным EAX-кроссовером, тестирован на 77 min-max-mTSP-инстансах + 43 multidepot-инстансах; держит большинство BKS на benchmark-наборах для min-max mTSP. Открытый C++-код. **Запущен в работе** на mTSPLib (раздел 4.8).
- **Mahmoudinazlou & Kwon HGA** [35] — hybrid genetic algorithm с STX-кроссовером; улучшил BKS для 21 из 89 инстансов на четырёх benchmark-наборах. Открытая Julia-реализация [52]. **Запущен в работе** на mTSPLib (раздел 4.8).
- **ITSHA** [31] — двухфазный VNS-эвристика с 32 опубликованными BKS на minsum-mTSP. Не запускался.
- **MILS** [37] — learning-guided iterated local search с МАВ-выбором операторов, 32 новых BKS + 35 матчей из 77 инстансов. Не запускался.

Все четыре имеют общее ограничение: *published BKS получены только на инстансах с $N \leq 1000$* . Применимость к целевой задаче (N до 100 000) не подтверждена авторами и потребовала бы существенной инженерной адаптации.

3.2. Нейросетевые SOTA для крупных TSP/mTSP

- **Equity-Transformer** [9] — декодер с equity-context для min-max mTSP. По данным авторов на mTSP $N = 1000$, $m = 100$ дает $\sim 53\%$ снижение makespan и $\sim 335\times$ ускорение относительно LKH-3. Опубликованные результаты ограничены масштабом $N \leq 1000$.

- **DPN** [10] — decoupled partition+navigation, до 33.7 % улучшения над Equity-Transformer на mPDP-50. Опубликованные результаты ограничены масштабом $N \leq 1000$.
- **GLOP** [7] — иерархическое разбиение для крупных TSP/CVRP/PCTSP. На TSP-100K дает разрыв 5.10 % к LKH-3 за 2.8 мин, ускорение $\sim 174\times$. *Тестирован только на классической TSP, не на mTSP*: декодер не выполняет разбиение на m маршрутов.
- **DualOpt** [8] — divide-and-optimize для крупных TSP, разрыв до 1.40 % к LKH-3 на TSP-100K при $\sim 104\times$ ускорении. *Тестирован только на классической TSP, не на mTSP*.
- **UDC** [11] — unified divide-conquer-reunion для 10 крупных CO-задач. Не специализирован на min-max mTSP.
- **H-TSP** [13] — иерархический RL для классической TSP до 10 000 узлов; не покрывает mTSP-постановку и не масштабируется до $N = 100\,000$.
- **DAN** [15] — decentralized attention-based neural network для min-max mTSP, тестирован 50–1000 узлов, 5–20 агентов.
- **iMTSP** [17] — bilevel imperative learning для min-max mTSP до 1000 узлов.

Обоснование исключения из количественного сравнения.

1. *Масштаб обучения и применения.* Ни одна из специализированных mTSP-моделей (Equity-Transformer, DPN, DAN, iMTSP) не имеет опубликованных результатов и обученных моделей в открытом доступе для $N \geq 5\,000$. Универсальные TSP-модели (GLOP, DualOpt, H-TSP) масштабируются до $N = 100\,000$, но не выполняют разбиение на m маршрутов и потому не решают саму задачу mTSP.
2. *Необходимость переобучения для каждой пары (n, m) .* Нейросетевой решатель обучается на конкретной конфигурации $(n, m, \text{геометрия})$. На данных вне обучающего распределения качество резко падает (см. обзор обобщающей способности NCO в Drakulic et al. LEHD, NeurIPS 2023;

Zhang et al. ASAP, arXiv:2501.17377; Tang et al. test-time projection, arXiv:2506.02392). В прикладном сценарии маршрутизации курьерского флота число курьеров и распределение заказов меняются ежедневно (новые заказы и смены, отказы, отпуска и больничные), что требует регулярного переобучения и существенно увеличивает операционные затраты.

3. *Зависимость от GPU.* Указанные методы требуют CUDA-GPU как для обучения, так и для инференса на крупных N (объем промежуточных эмбедингов растет квадратично по числу узлов). Крупные компании располагают такой инфраструктурой; в настоящей работе платформа сравнения — CPU-VPS без GPU-ускорителя, что соответствует целевому сценарию использования решателя.

Таким образом, нейросетевые SOTA-методы и собственный решатель ALNS-mTSP решают задачи в разных условиях: первые оптимизируют качество на узком обученном диапазоне с использованием GPU, второй — универсальный CPU-решатель на любой (n, m) в заявленном диапазоне без переобучения. Прямое сравнение имеет смысл только в пересечении их условий применимости ($N \leq 1\,000$ для mTSP-моделей), что не является целью настоящей работы.